

CD 2750

Programspråkssemantik

Operationell semantik · Denotationssemantik
Axiomatisk semantik



Kjell Post

MÄLARDALENS HÖGSKOLA

Innehåll

Kursinformation	3
Föreläsningsbilder	5
1 Introduktion	9
2 Gofer	39
3 Operationell semantik	62
4 Denotationssemantik	100
5 Beräkningsmodeller	123
6 Direct style semantics (forts)	150
7 Försättningar (continuations)	171
8 Statisk analys	185
9 Axiomatisk semantik, programbevis	209
10 Bilagor	229
Programmeringsspråket Gofer	235
1 Introduktion	235
2 Interaktion med Gofer	235
3 Uttryck, värden och typer	236
Laborationsmaterial	238
Introduktion till Gofer-implementationerna	240
Operationell semantik i Gofer	242
Denotationssemantik i Gofer	246
Projektuppgift	248
Appendices of "Semantics with Applications" with Gofer implementations	249
B Introduction to Gofer Implementations	251
C Operational Semantics in Gofer	255
D Denotational Semantics in Gofer	263
E From Miranda to Gofer by Hand	267
Bibliography	269

MÄLARDALENS HÖGSKOLA 1997-10-15
IDT, KJELL POST

CD2750 — Programspråkssemantik

kursinformation period 2 1997/98



Allmän information

Kursen omfattar 38 timmar föreläsningar samt 20 laborationstimmar. Den första föreläsningen äger rum i sal N23, måndagen den 27 oktober kl 13.

Kursen behandlar olika sätt att definiera semantik (betydelse) för programmeringsspråk och är lämplig för personer som konstruerar, implementerar eller analyserar programmeringsspråk. De metoder som vi kommer att studera är operationell semantik, denotationssemantik och axiomatisk semantik. Dessutom kommer en rad matematiska begrepp som domänteori och λ -kalkyl att tas upp som grund för de olika metoderna.

Inlämningsuppgifter kommer att delas ut med cirka en veckas mellanrum. På laborationerna används de teoretiska kunskaperna genom att en interpretator implementeras utgående från semantiska definitioner.

Förkunskapskrav

För tillträde till kursen skall CD2620, Kompilator teknik, *samt* CD 1000, Funktionell Programmering, eller CD 2760, Logikprogrammering, vara avklarade. Särskilda undantag kan göras av examinatorn för studenter som inte läst CD2620 men som klarat av CD1100, Automatater och Beräkningar.

Litteratur och kursmaterial

Läroboken heter "Semantics with Applications", *Nielson* och *Nielson* (J. Wileys förlag) och finns att köpa i bokhandeln. Kurskompendium innehållande föreläsningsanteckningar och laborationsuppgifter finns också att köpa i bokhandeln vid kursstart. En WWW-sida för kursen med skelett för laborationerna finns på <http://www.idt.mdh.se/kurser/cd2750>. Addera den till era bokmärken.

Laborationer

Laborationerna görs i språket Gofer på Sun- eller PC-maskiner i grupper om högst 2 personer. Gofer är installerat på UNIX-maskinerna och finns även tillgänglig för hemmabruk för de flesta maskiner från bl a <ftp.cs.chalmers.se> under `pub/haskell/gofer`.

Examination

För godkänd kurs krävs godkända inlämningsuppgifter och godkända laborationer. Ingen tentamen. Inlämningsuppgifter löses individuellt och lämnas in före angiven tid. Sista dag för redovisning av laborationerna är måndag den 1 jan 1998.

Lärare och administrativ personal

Kjell Post, föreläsare och kursledare. Besök norra huset, plan 1, rum 1260, telefon 10 1687; elektronisk post kjell.post@mdh.se.

Monica Wasell, institutionssekreterare. Besök norra huset, plan 1, rum 1241, telefon 10 1453; elektronisk post monica.wasell@mdh.se.

Roger Jonsson, studierektor. Besök norra huset, plan 1, rum 1242, telefon 10 1454; elektronisk post roger.jonsson@mdh.se.

CD2750

Programspråkssemantik



Kjell Post

Innehåll

1	Introduktion	4
	Olika typer av semantik	8
	Funktioner	9
	Operationell semantik	13
	Denotationssemantik	15
	Axiomatisk semantik	17
	Språket While	19
	Abstrakt syntax	20
	Relationer	22
	Övergångssystem	23
	Semantik för uttryck	24
	Strukturell induktion	30
	Substitution	32
2	Gofer	34
3	Operationell Semantik	57
	Naturlig semantik	59
	Strukturell semantik	72
	Utvidgningar av While	81
4	Denotationssemantik	95

Direct style semantics	96
λ -kalkyl	101
Direct Style Semantics	114
5 Beräkningsmodeller	118
Domänteori	120
6 Direct style semantics (forts)	145
Block och deklarationer	153
7 Fortsättningar (continuations)	166
8 Statisk Analys	180
Funktionella beroenden (kap 5)	188
9 Axiomatisk semantik, programbevis	204
Partiell korrekthet	205
Total korrekthet	216
Proceduranrop och rekursion	220
Sista sidan	223
10 Bilagor	224
10.1 Naturlig semantik för While med block och procedurer	225
10.2 Härledningsträd för ett program med block och procedurer	226

10.3 Denotationssemantik för While med block och procedurer	227
10.4 Partiell korrekthet för fakultetsprogrammet	228
10.5 Total korrekthet för fakultetsprogrammet .	229

1 Introduktion

Mål med kursen

Att beskriva programmeringsspråks semantik.

Vad menas med semantik?

Från 1800-talet: det område där man studerar [tolkning och innebörd](#), främst för naturligt språk.

- **Syntax:** [Hur symboler relateras och manipuleras utan någon som helst tolkning.](#)
- **Semantik:** [Vad de olika symbolerna betyder \(mening, tolkning\).](#)
- **Pragmatik:** [Hur symbolerna utnyttjas \(används\).](#)

Syntax, semantik och pragmatik

“Syntax is straightforwardly viewed as being concerned with delineating the legal forms (or phrases, expressions, etc) of the language but too often one finds that ‘semantics’ is then taken as being everything else, confusing ‘what an expression means’ with ‘how does one compute an expression’s value’ — two equally important aspects of the study of languages, but nonetheless distinct, called *semantics* and *pragmatics*.”

Chris Wadsworth, 1978

Varför studera semantik?

- Kunna beskriva programmeringsspråk på ett rigoröst sätt.
- Validering av program och översättare.
- Specifikation för kompilator/interpretator.
- Bestämma egenskaper för program.

Exempel: ALGOL 60 definierades i ett mycket välskrivet dokument, skrivet (på engelska) av mycket framstående dataloger. Ändå innehåller dokumentet många tvetydigheter.

[*The remaining trouble spots in ALGOL 60*, Donald Knuth, CACM, 10, 611-618, 1968] Visar tvetydigheter med sidoeffekter, goto-satser, for-satser, ...

Tony Hoare: “When you design a language with so few trouble spots, you can be proud.”

Även definitionen av Pascal innehöll många tvetydigheter trots att den skrevs många år efter ALGOL-rapporten.

[*Ambiguities and insecurities in Pascal*, Welsh et al, SP&E, 7, 685-696, 1977]

Resultat: Olika kompilatorer implementerade olika språk! Ledde till att många ropade högljutt på standardisering. *Men hur ska man standardisera något som man inte kan beskriva?*

Fler exempel: Uppföljningsrapporten av standardkommittén för FORTRAN beskriver 52 olika saker som behöver förtydligas.

Tony Hoare igen: “There are two ways to design a system. Either you make it so simple there are obviously no errors, or you make it so complicated that there are no obvious errors.”

Olika typer av semantik

- **Operationell semantik** (Landin, Plotkin, Milner)

Semantiken för språket definieras med hjälp av en abstrakt maskin. Betydelsen av ett program är de tillstånd som besöks då programmet körs.

- **Denotationssemantik** (Strachey, Scott)

Med hjälp av funktioner översätts syntaktiska fraser (programspråkskonstruktioner) till matematiska objekt (betydelsen). En viktig egenskap är att semantiken för en sammansatt konstruktion alltid definieras som en komposition av semantiken för de ingående komponenterna.

- **Axiomatisk semantik** (Floyd, Hoare)

Använder logik för att med hjälp av pre- och postvillkor visa att ett program satisfierar en specifikation.

Funktioner

Vi skriver $f : X \rightarrow Y$ för att beteckna en *total funktion* från X till Y .

Funktionsapplikation skrivs $f(x)$ eller ibland fx .

Exempel: $g(x) = x^3 : \mathbf{Z} \rightarrow \mathbf{Z}$. $g(3) = 27$.

En total funktion $f : X \rightarrow Y$ avbildar *varje* $x \in X$ på *exakt ett* element $y \in Y$.

- $f(x) = y$ och $f(x) = y' \Rightarrow y = y'$
- $(\forall x \in X, \exists y \in Y) f(x) = y$

Alla par $\langle x, y \rangle$ representeras av $graph(f)$:

$$graph(f) = \{\langle x, y \rangle \in X \times Y \mid f(x) = y\}$$

En *partiell funktion* $g : X \hookrightarrow Y$ mappar varje $x \in X_f \subset X$ till ett exakt ett element $y \in Y$.

Vi skriver $g(x) = y$ om $x \in X_f$ och $g(x) = \text{undef}$ om $x \notin X_f$.

Exempel: $g(x) = \sqrt{x} : \mathbf{Z} \hookrightarrow \mathbf{Z}$

Operationer på funktioner

Komposition: $(f \circ g)(x) = f(g(x))$.

Uppdatering: Om $f : X \rightarrow Y$, $x \in X$ och $y \in Y$ så är

$$f[x \mapsto y](x') = \begin{cases} y & \text{om } x = x' \\ f(x') & \text{annars} \end{cases}$$

Funktioner med exakt ett argument

I Gofer tar funktionerna alltid exakt *ett* argument.

```
? :t nand where nand(x,y) = not(x && y)
let {...} in nand :: (Bool,Bool) -> Bool
```

Funktionen `nand` tar ett argument: en tupel $\langle x, y \rangle$.

Betrakta nu följande definition

```
? :t nand where nand x y = not(x && y)
let {...} in nand :: Bool -> Bool -> Bool
```

vilket råkar vara ekvivalent med

```
? :t nand where nand = \x y -> not(x && y)
let {...} in nand :: Bool -> Bool -> Bool
```

Funktionspilen ” $\rightarrow$ ” är högerassociativ, dvs

$$\text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \equiv \text{Bool} \rightarrow (\text{Bool} \rightarrow \text{Bool})$$

`nand` tar en `Bool` och returnerar en funktion!

```
? :l /dev/tty
nand x y = not(x && y)
mynot = nand True
^D
? mynot(True)
False
? (nand True) True
False
```

Funktionsapplikation är vänsterassociativ så

```
? nand True True
False
```

Ett litet smakprov

Givet ett enkelt språk med *sekvenser av tilldelningar*.

Exempel: $z := x; x := y; y := z$

Operationell semantik

En operationell specifikation anger hur exekveringen ska utföras:

- En sekvens av satser $S_1; S_2; \dots; S_n$ exekveras var för sig från vänster till höger.
- En sats $x := y$ tilldelar x värdet av y .

För att representera *variablers värden* använder vi ett sk *tillstånd*. Initialt tillstånd: $[x \mapsto 5, y \mapsto 7, z \mapsto 0]$

En *härledning* är en *abstraktion* av exekveringen:

$$\begin{aligned} & \langle z := x; x := y; y := z, [x \mapsto 5, y \mapsto 7, z \mapsto 0] \rangle \\ \Rightarrow & \langle x := y; y := z, [x \mapsto 5, y \mapsto 7, z \mapsto 5] \rangle \\ \Rightarrow & \langle y := z, [x \mapsto 7, y \mapsto 7, z \mapsto 5] \rangle \\ \Rightarrow & [x \mapsto 7, y \mapsto 5, z \mapsto 5] \end{aligned}$$

Kan också åskådliggöras med ett *s k härledningsträd*:

$$\frac{\frac{\langle z:=x, s_0 \rangle \rightarrow s_1 \quad \langle x:=y, s_1 \rangle \rightarrow s_2}{\langle z:=x; x:=y, s_0 \rangle \rightarrow s_2} \quad \langle y:=z, s_2 \rangle \rightarrow s_3}{\langle z:=x; x:=y; y:=z, s_0 \rangle \rightarrow s_3}$$

där

$$\begin{aligned} s_0 &= [x \mapsto 5, y \mapsto 7, z \mapsto 0] \\ s_1 &= [x \mapsto 5, y \mapsto 7, z \mapsto 5] \\ s_2 &= [x \mapsto 7, y \mapsto 7, z \mapsto 5] \\ s_3 &= [x \mapsto 7, y \mapsto 5, z \mapsto 5] \end{aligned}$$

Denotationssemantik

- Betydelsen av en sekvens av satser $S_1; S_2; \dots; S_n$ är funktionssammansättningen av betydelsen av de enskilda satserna:

$$\mathcal{S}[[S_1; \dots; S_n]] = \mathcal{S}[[S_n]] \circ \dots \circ \mathcal{S}[[S_1]]$$

Kom ihåg att $(f \circ g)(x) = f(g(x))$.

- Betydelsen av en tilldelning $x := y$ är en funktion som, givet ett tillstånd s_1 , producerar ett nytt tillstånd s_2 , där s_2 är likadant som s_1 med undantag för x som i s_2 mappas till värdet av y :

$$\mathcal{S}[[x := y]]s = s[x \mapsto \mathcal{V}[[y]]]$$

Syntaxparanteserna $[[\dots]]$ används som vanliga paranteser och innesluter syntaktiska konstruktioner.

Typen på $\mathcal{S}$ är **Stm** $\rightarrow$ **State** $\rightarrow$ **State**.

$$\begin{aligned} & \mathcal{S}[\![z:=x; x:=y; y:=z]\!]([x \mapsto 5, y \mapsto 7, z \mapsto 0]) \\ = & \mathcal{S}[\![y:=z]\!] \circ \mathcal{S}[\![x:=y]\!] \circ \mathcal{S}[\![z:=x]\!]([x \mapsto 5, y \mapsto 7, z \mapsto 0]) \\ = & \mathcal{S}[\![y:=z]\!](\mathcal{S}[\![x:=y]\!](\mathcal{S}[\![z:=x]\!]([x \mapsto 5, y \mapsto 7, z \mapsto 0]))) \\ = & \mathcal{S}[\![y:=z]\!](\mathcal{S}[\![x:=y]\!]([x \mapsto 5, y \mapsto 7, z \mapsto 5])) \\ = & \mathcal{S}[\![y:=z]\!]([x \mapsto 7, y \mapsto 7, z \mapsto 5]) \\ = & [x \mapsto 7, y \mapsto 5, z \mapsto 5] \end{aligned}$$

Att observera:

- Ingen exekvering, bara manipulation av matematiska objekt.
- Binder sig inte till någon särskild exekveringsordning.
- Mer ”abstrakt” än operationell semantik.

Axiomatisk semantik

Ett program sägs vara *partiellt korrekt* i förhållande till ett *initialt villkor* pre och ett *slutgiltigt villkor* $post$ om

- *initialtillståndet* uppfyller pre ...
- och programmet *terminerar*...
- så uppfyller $post$ det *slutgiltiga tillståndet*

$$\{x = n \wedge y = m\} z := x; x := y; y := z \{y = n \wedge x = m\}$$

Tillståndet $[x \mapsto 5, y \mapsto 7, z \mapsto 0]$ uppfyller trivialt det initiala villkoret med $n = 5, m = 7$.

Om påståendet är korrekt och programmet *terminerar* kan vi dra slutsatsen att det *slutgiltiga tillståndet* har $y = 5, x = 7$.

Hur bevisar man att påståendet är korrekt? Genom att bygga ett *bevisträd* utav små bevis på formen

$$\{y = n, \dots\} x := y \{x = n, y = n, \dots\}.$$

$$\frac{\frac{\{p_0\}z := x \quad \{p_1\}x := y \quad \{p_2\}}{\{p_0\}z := x; x := y \quad \{p_2\}y := z \quad \{p_3\}}}{\{p_0\}z := x; x := y; y := z \quad \{p_3\}}$$

Där

$$p_0 = x = n \wedge y = m$$

$$p_1 = z = n \wedge y = m$$

$$p_2 = z = n \wedge x = m$$

$$p_3 = y = n \wedge x = m$$

Språket While

$$\begin{aligned} a &\rightarrow n \mid x \mid a_1 + a_2 \mid a_1 * a_2 \mid a_1 - a_2 \\ b &\rightarrow \text{true} \mid \text{false} \mid a_1 = a_2 \mid a_1 \leq a_2 \mid \neg b \mid b_1 \wedge b_2 \\ S &\rightarrow x := a \mid \text{skip} \mid S_1 ; S_2 \mid \text{if } b \text{ then } S_1 \text{ else } S_2 \\ &\quad \mid \text{while } b \text{ do } S \end{aligned}$$

n	$\in$	Num	numeraler
x	$\in$	Var	variabler
a	$\in$	Aexp	aritmetiska uttryck
b	$\in$	Bexp	booleska uttryck
S	$\in$	Stm	satser

Exempel:

$y := 1; \text{while } \neg(x = 1) \text{ do } (y := y * x; x := x - 1)$

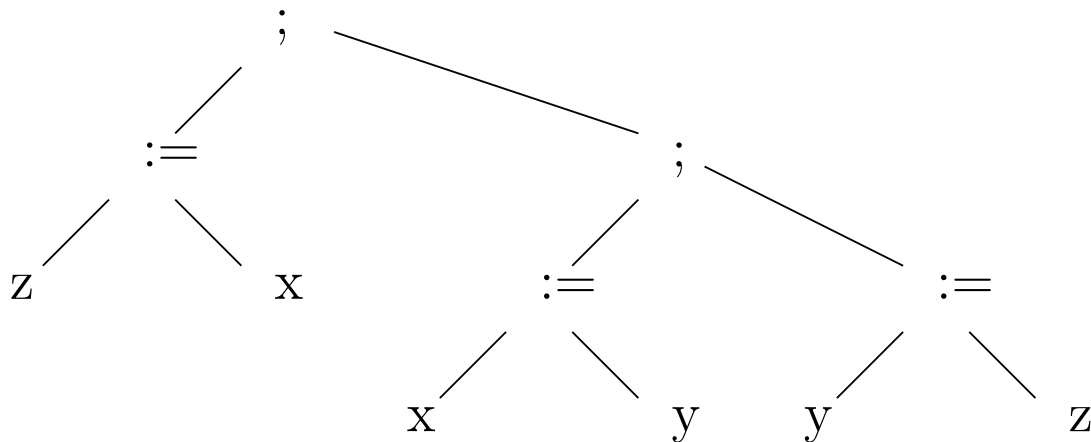
Abstrakt syntax

Ett *abstrakt syntaxträd* bevarar *strukturen* på programmet.

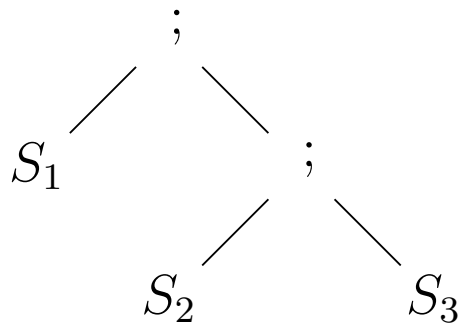
Observera att bilden på sid 8 i boken för

$z := x; x := y; y := z$

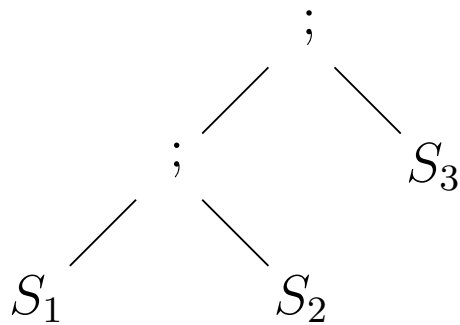
borde se ut enligt nedan:



Vi kommer att använda paranteser för att visa abstrakt syntax i s k *linjär notation*:



Linjärt: $S_1; (S_2; S_3)$



Linjärt: $(S_1; S_2); S_3$

Relationer

En relation R över X och Y är en delmängd av $X \times Y$:

$$X \ R \ Y \subseteq \{\langle x, y \rangle \mid x \in X, y \in Y\}$$

Exempel: Relationen $\leq$ över $\mathbf{N} \times \mathbf{N}$ definieras av mängden

$$\{\langle x, y \rangle \mid x, y \in \mathbf{N}, x \leq y\} = \{\langle 0, 1 \rangle, \langle 0, 2 \rangle, \langle 0, 3 \rangle, \dots\}$$

Övergångssystem

Ett *övergångssystem* är en trippel $\langle \Gamma, T, \triangleright \rangle$ där Γ är en mängd tillstånd, $T \subseteq \Gamma$ är en mängd sluttillstånd och $\triangleright \subseteq \Gamma \times \Gamma$ är en relation som måste uppfylla

$$(\forall \gamma \in T)(\forall \gamma' \in \Gamma) \neg(\gamma \triangleright \gamma')$$

Dvs, har man väl kommit in i ett sluttillstånd kan man inte fortsätta.

För ett tillstånd $\gamma \in \Gamma, \gamma \notin T$ sägs γ ha *fastnat* om det inte finns något γ' så att $\gamma \triangleright \gamma'$.

Tillstånd kallas ibland för *konfiguration*.

Semantik för uttryck

Semantiken för aritmetiska uttryck $n \in \mathbf{Num}$

$$n \rightarrow d \mid nd$$

$$d \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

kan specificeras med en s k *semantisk funktion*

$$\mathcal{N} : \mathbf{Num} \rightarrow \mathbf{Z}$$

så att t ex $\mathcal{N}[[314]] = 314$.

Observera att **314** är syntax och 314 är ett tal!

$\mathcal{N}$ definieras induktivt över syntaxen för n :

$$\begin{aligned}\mathcal{N}[[d]] &= \mathcal{D}[[d]] \\ \mathcal{N}[[nd]] &= 10 * \mathcal{N}[[n]] + \mathcal{D}[[d]] \\ \mathcal{D}[[0]] &= 0 \\ &\vdots \\ \mathcal{D}[[9]] &= 9\end{aligned}$$

Exempel:

$$\begin{aligned}\mathcal{N}[[314]] &= 10 * \mathcal{N}[[31]] + \mathcal{D}[[4]] \\ &= 10 * (10 * \mathcal{N}[[3]] + \mathcal{D}[[1]]) + \mathcal{D}[[4]] \\ &= 10 * (10 * \mathcal{D}[[3]] + \mathcal{D}[[1]]) + \mathcal{D}[[4]] \\ &= 10 * (10 * 3 + 1) + 4 \\ &= 314\end{aligned}$$

Att observera:

1. Semantiken definieras med en regel för varje produktion.
2. Värdet av ett sammansatt objekt ges av värdet av delarna.

Tillstånd

Värdet av ett uttryck beror på de ingående variablernas värden.

Definition: Ett *tillstånd* (eng. *state*) är en funktion

$$\text{State} : \text{Var} \rightarrow \mathbb{Z}$$

Om s är ett tillstånd där x har värdet 4 så är $s(x) = 4$.

```
? :load /dev/tty
i_state x = 0
update s y v x = if x==y then v else s x
state = update i_state "x" 4
^D
? newstate "x"
4
```

Betydelsen av aritmetiska uttryck $a \in \mathbf{Aexp}$ ges nu av

$$\mathcal{A} : \mathbf{Aexp} \rightarrow (\mathbf{State} \rightarrow \mathbf{Z})$$

Dvs, $\mathcal{A}$ tar sina argument i sänder, dvs $\mathcal{A}[\mathbf{x}+1]$ ger en funktion som tar ett tillstånd och producerar ett heltal.

$$\begin{aligned}\mathcal{A}[n]s &= \mathcal{N}[n] \\ \mathcal{A}[x]s &= s(x) \\ \mathcal{A}[a_1+a_2]s &= \mathcal{A}[a_1]s + \mathcal{A}[a_2]s \\ \mathcal{A}[a_1*a_2]s &= \mathcal{A}[a_1]s * \mathcal{A}[a_2]s \\ \mathcal{A}[a_1-a_2]s &= \mathcal{A}[a_1]s - \mathcal{A}[a_2]s\end{aligned}$$

Exempel: Antag att $s(\mathbf{x}) = 3$.

$$\begin{aligned}\mathcal{A}[\mathbf{x}+1]s &= \mathcal{A}[\mathbf{x}]s + \mathcal{A}[1]s \\ &= s(\mathbf{x}) + \mathcal{N}[1] \\ &= 3 + 1 \\ &= 4\end{aligned}$$

Betydelsen av booleska uttryck $b \in \mathbf{Bexp}$ ges nu av

$$\mathcal{B} : \mathbf{Bexp} \rightarrow (\mathbf{State} \rightarrow \mathbf{T})$$

$$\mathcal{B}[\mathbf{true}]s = tt$$

$$\mathcal{B}[\mathbf{false}]s = ff$$

$$\mathcal{B}[a_1 = a_2]s = \mathcal{A}[a_1]s = \mathcal{A}[a_2]s$$

$$\mathcal{B}[a_1 \leq a_2]s = \mathcal{A}[a_1]s \leq \mathcal{A}[a_2]s$$

$$\mathcal{B}[\neg b]s = \neg \mathcal{B}[b]s$$

$$\mathcal{B}[b_1 \wedge b_2]s = \mathcal{B}[b_1]s \wedge \mathcal{B}[b_2]s$$

Fria variabler

Mängden *fria variabler*, $FV(a)$, i ett aritmetiskt uttryck a är de variabler som ingår i a :

$$\begin{aligned}FV(n) &= \emptyset \\FV(x) &= \{x\} \\FV(a_1 + a_2) &= FV(a_1) \cup FV(a_2) \\FV(a_1 * a_2) &= FV(a_1) \cup FV(a_2) \\FV(a_1 - a_2) &= FV(a_1) \cup FV(a_2)\end{aligned}$$

Exempel: $FV(\mathbf{x+y*x}) = \{\mathbf{x, y}\}$.

$FV(b)$ definieras på liknande sätt (se boken).

Strukturell induktion

För att visa en egenskap P :

- Visa att P gäller för alla baselement.
- Antag att induktionshypotesen gäller för alla delar i ett sammansatt objekt; visa sedan att hypotesen gäller för hela objektet.

Lemma: Låt s, s' vara tillstånd så att $s(x) = s'(x)$ för alla $x \in FV(a)$. Då gäller $\mathcal{A}[[a]]s = \mathcal{A}[[a]]s'$.

Bevis: Med strukturell induktion över $\mathbf{Aexp}$.

Basfall:

- $a \rightarrow n$,
$$\mathcal{A}[[n]]s = \mathcal{N}[[n]], \quad \mathcal{A}[[n]]s' = \mathcal{N}[[n]]$$
- $a \rightarrow x$,
$$\mathcal{A}[[x]]s = s(x), \quad \mathcal{A}[[x]]s' = s'(x).$$

Eftersom $x \in FV(a)$ så är dessa lika.

Induktionshypotes: Antag att lemmat gäller för $a_1, a_2 \in \mathbf{Aexp}$.

Induktion:

- $a \rightarrow a_1 + a_2$,

$$\mathcal{A}[[a_1 + a_2]]s = \mathcal{A}[[a_1]]s + \mathcal{A}[[a_2]]s$$

$$\mathcal{A}[[a_1 + a_2]]s' = \mathcal{A}[[a_1]]s' + \mathcal{A}[[a_2]]s'$$

Enligt IH gäller $\mathcal{A}[[a_i]]s = \mathcal{A}[[a_i]]s'$ eftersom $FV(a_i) \subseteq FV(a_1 + a_2)$.

Alltså gäller lemmat också för $a_1 + a_2$.

- $a \rightarrow a_1 * a_2, a \rightarrow a_1 - a_2$,

Samma princip.

Substitution

Syntaktisk substitution

Att (textuellt) ersätta en variabel förekomst y i ett aritmetiskt uttryck a med ett annat uttryck a_0 :

$$a[y \mapsto a_0]$$

$$(x+y*x)[x \mapsto y-5] = (y-5)+y*(y-5).$$

Substitution definieras induktivt över **Aexp**:

$$\begin{aligned} n[y \mapsto a_0] &= n \\ x[y \mapsto a_0] &= \begin{cases} a_0 & \text{om } x = y \\ x & \text{annars} \end{cases} \\ (a_1 + a_2)[y \mapsto a_0] &= (a_1[y \mapsto a_0]) + (a_2[y \mapsto a_0]) \\ (a_1 * a_2)[y \mapsto a_0] &= (a_1[y \mapsto a_0]) * (a_2[y \mapsto a_0]) \\ (a_1 - a_2)[y \mapsto a_0] &= (a_1[y \mapsto a_0]) - (a_2[y \mapsto a_0]) \end{aligned}$$

Substitution för tillstånd

Tillstånd betraktas som funktioner, så

$$(s[y \mapsto v])(x) = \begin{cases} v & \text{om } x = y \\ s(x) & \text{annars} \end{cases}$$

Inlämningsuppgifter

- Uppgift 1.2
- Uppgift 1.8
- Uppgift 1.12

Använd strukturell induktion.

- Uppgift 1.13

Visa $\mathcal{A}[[a[y \mapsto a_0]]]s = \mathcal{A}[[a]](s[y \mapsto \mathcal{A}[[a_0]]s])$

OBS: olika typer av $\mapsto$!

2 Gofer

- Gofer är ett funktionellt språk.

Dvs, funktioner kan

- skickas som argument
- returneras som resultat
- lagras i variabler

Den huvudsakliga kontrollstrukturen är rekursion.

- Gofer är starkt typat.

Varje uttryck har en typ (som härleds av systemet).

- Gofer är interaktivt.
- Gofer har mönstermatchning, garbage collection, lat evaluering, etc.
- Gofer är gratis.

En session med Gofer

```
snorre$ gofer
Gofer version 2.30a...
Type :? for help
? 3+2
5

? if 4*5 == 60/3 then 17 else -42
17
```

Ladda filer

Gofer-program har suffix `.gs` och laddas in med `:l`.

```
? :l mypgm.gs
```

Variabelbindningar

Variabler kan bindas med `let` eller `where`.

```
? let b=False in not(b)
True
? not(b) where b=False
True
```

I ett program kan man också skriva t ex

```
x = 10-5*2
```

Bindningar är *inte* tilldelningar! Variabler får värden när de introduceras och kan sedan inte bindas om.

FunktionsdeklARATIONER

I filen `test.gs`:

```
succ(x) = x+1      -- compute successor
nand(x,y) = not(x && y)
```

Ladda in och testa:

```
? :l test.gs
? if nand(True,False) then succ(3) else 0
4
? nand(0,1)
ERROR: Type error in application
*** expression      : nand (0,1)
*** term            : (0,1)
*** type            : (Int,Int)
*** does not match  : (Bool,Bool)
```

Deklarationerna

```
succ(x) = x+1
nand(x,y) = not(x && y)
```

är ekvivalenta med

```
succ = \x -> x+1
nand = \ (x,y) -> not(x && y)
```

Observera hur namnet på funktionen (t ex **succ**) är separerad från dess värde ($\backslash x \rightarrow x+1$).

Vi kan applicera värdet av en funktion på ett argument utan att referera till funktionens namn:

```
? (\x -> x+1) (3)
4
```

För er som kan λ -kalkyl: $\backslash x \rightarrow x+1 = \lambda x.x + 1$

Typer

- Alla uttryck i Gofer har en typ.
Ett sådant språk kallas *starkt typat*.
- En typinferensmekanism luskar reda på typen för alla uttryck i ditt program.
- Olika slags typer:
 - Primitiva typer: `Int`, `Bool`, ...
 - Sammansatta typer: Funktioner, par, ...
 - Algebraiska typer: träd, poster, ...

Man kan fråga efter typen på ett uttryck med `:t`

```
? :t 1+2
1 + 2 :: Int
```

Symbolen `::` uttalas "har typen".

Primitiva typer

Gofer har bl a heltal, booleans, strängar:

```
? :t 3  
3 :: Int
```

```
? :t True  
True :: Bool
```

```
? :t "hej"  
"hej" :: String
```

Typer skrivs alltid med inledande versal.

Variabelnamn kan därför inte börja med en versal.

Sammanfattning

Funktioner: Om T_1 och T_2 är två typer så är $T_1 \rightarrow T_2$ också en typ.

```
? :t succ  
succ :: Int -> Int
```

Par: Om $T_1, \dots, T_n$ är typer så är $(T_1, \dots, T_n)$ också en typ.

```
? :t (3,True,succ)  
(3,True,succ) :: (Int,Bool,Int -> Int)
```

Listor: Om T är en typ så är $[T]$ också en typ.

```
? :t [True,False]  
[True,False] :: [Bool]
```

Typsynonymer

En typsynonym är en förkortning för en ofta använd typ.

`String` är t ex bara en förkortning för en lista av tecken:

```
type String = [Char]
```

Argument av typen `String` kan alltid förekomma där argument av typen `[Char]` förväntas.

Polymorfism

Språk som associerar endast en typ med ett värde kallas *monomorfa* (t ex C och Pascal).

Onödigt restriktivt. Exempel: räkna antalet element i en lista. I C måste man skriva en funktion för listor av heltal, en annan för listor av poster, osv.

Språk som tillåter värden att ha mer än en typ kallas *polymorfa*. I Gofer har `length` följande typ:

```
? :t length  
length :: [a] -> int
```

`a` kallas en typvariabel och kan instantieras till godtycklig typ.

```
? length(["a","b","c"])
```

```
3
```

```
? length([and,or])
```

```
2
```

```
? length(42)
```

```
ERROR: Type error in application
```

```
*** expression      : length 42
```

```
*** term            : 42
```

```
*** type            : Int
```

```
*** does not match : [a]
```

Att definiera nya datatyper

```
data Dir = North | South | East | West
```

Efter inladdning:

```
? :t North  
North :: Dir
```

Man kan tänka sig typer som mängder av element:

$$\text{Dir} = \{\text{North}, \text{South}, \text{East}, \text{West}\}$$

Varje element kallas en *konstruktör*.

Här är en funktion som opererar på en egendefinierad typ:

```
turn dir = case (dir) of
  North -> East
  South -> West
  East   -> South
  West   -> North
```

Efter inladdning:

```
? :t turn
turn :: Dir -> Dir

? turn West
North
```

Mönstermatchning

Kan skrivas mer elegant med mönstermatchning:

```
turn(North) = East
turn(South) = West
turn(East)  = South
turn(West)  = North
```

Fler exempel

```
fac(0) = 1      -- fac :: Int -> Int
fac(n) = n * fac(n-1)
```

```
fst(x,_) = x    -- fst :: (a,b) -> a
snd(_,y) = y    -- snd :: (a,b) -> b
```

”_” är en anonym (don’t-care) variabel.

Mer om nya datatyper

```
data Person = Male String | Female String
```

Definierar en ny typ `Person`.

Konstruktören `Male` tar ett argument, en sträng.

Konstruktören `Female` tar ett argument, en sträng.

```
? :t Male "kjell"  
Male "kjell" :: Person
```

```
? :t [Male "adam", Female "eva"]  
[Male "adam",Female "eva"] :: [Person]
```

Rekursiva datatyper

```
data Z = Zero | Succ Z
```

Det finns två sätt att skapa ett Z-objekt:

- Zero är ett Z-objekt
- Om n är ett Z-objekt så är $\text{Succ } n$ också det.

```
add(Zero,n) = n
```

```
add(Succ(m),n) = Succ(add(m,n))
```

```
? :t add
```

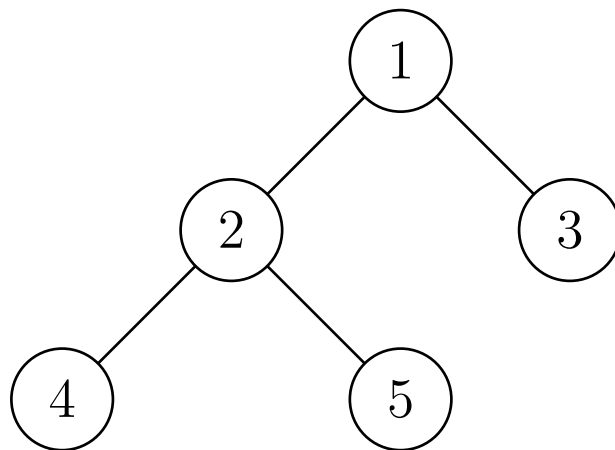
```
add :: (Z,Z) -> Z
```

```
? add(Succ(Succ(Zero)),Succ(Zero))
```

```
Succ (Succ (Succ Zero))
```

```
data IntTree = Null | Node Int IntTree IntTree
```

```
t = Node 1 (Node 2 (Node 4 Null Null)
                  (Node 5 Null Null))
      (Node 3 Null Null)
```



Hur ska vi göra om vi vill ha ett träd med strängar?

```
data Tree a = Null | Node a (Tree a) (Tree a)
```

Typen på noderna, `a`, är en parameter!

En polymorf funktion

```
fringe :: Tree a -> [a]
fringe(Null) = []
fringe(Node x Null Null) = [x]
fringe(Node _ l r) = fringe(l)++fringe(r)
```

- Typen på funktionen behöver ej deklareras.
- ++ konkatenerar två listor.

```
? fringe(t)
[4, 5, 3]
```

```
? fringe(Node "a" (Node "b" Null Null) Null)
["b"]
```

Problem: Summera alla talen i en lista.

```
add :: [Int] -> Int
add([]) = 0
add(h:t) = h + add(t)
```

(":" är infix cons.)

```
? add([1,2,3])
6
```

Problem: Multiplicera alla talen i en lista.

```
prod :: [Int] -> Int
prod([]) = 1
prod(h:t) = h * prod(t)
```

Mönstret är detsamma och kan generaliseras.

Högre ordningens funktioner

- Basfallet för add är 0
- Basfallet för prod är 1
- Funktionen som används i add är +
- Funktionen som används i prod är *

```
fold :: ([a], (a,b) -> b, b) -> b
fold([], f, base) = base
fold(h:t, f, base) = f(h,fold(t,f,base))
```

```
add(l) = fold(l, \ (x,y) -> x+y, 0)
prod(l) = fold(l, \ (x,y) -> x*y, 1)
conc(l) = fold(l, \ (x,y) -> x++y, [])
```

```
? conc([[1,2],[3,4],[5,6]])
[1, 2, 3, 4, 5, 6]
```

Argument till funktioner

I Gofer tar alla funktioner exakt *ett* argument.

```
nand :: (Bool,Bool) -> Bool  
nand(x,y) = not(x && y)
```

nand tar ett argument, en tupel (x,y).
Betrakta följande definition:

```
nand :: Bool -> Bool -> Bool  
nand x y = not(x && y)
```

vilket är detsamma som

```
nand :: Bool -> Bool -> Bool  
nand = \x y -> not(x && y)
```

Eftersom $\rightarrow$ är högerassociativ så gäller:

$$\begin{aligned} \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \\ \equiv \\ \text{Bool} \rightarrow (\text{Bool} \rightarrow \text{Bool}) \end{aligned}$$

Dvs, `nand` tar en `Bool` och returnerar en funktion!

```
neg = nand(true)
```

```
? :t neg
```

```
neg :: Bool -> Bool
```

```
? neg(True)
```

```
False
```

```
? (nand True) False
```

```
False
```

Applikation är vänsterassociativ så:

```
? nand True False
```

```
False
```

Andra finesser i Gofer

- Lat evaluering:

```
? let f x = 17 in f (1/0)
17
```

- Oändliga datastrukturer:

```
? take 5 [1..]
[1, 2, 3, 4, 5]
```

- Guarded equations:

```
fac n | (n == 0) = 1
      | otherwise = n * fac(n-1)
```

- List comprehension:

```
? [ (x,y) | x<-[1..3], y<-[1..2] ]
[(1,1),(1,2),(2,1),(2,2),(3,1),(3,2)]
```

3 Operationell Semantik

Inte bara vad resultatet är, utan *hur* program exekveras.

OS definieras med en abstrakt maskin.

Betydelsen av ett program är de tillstånd som besöks då programmet "körs".

Två typer av operationell semantik

- **Naturlig semantik:** (Big-step semantics)
Beskriver det totala resultatet av exekveringen.
- **Strukturell semantik:** (Small-step semantics)
Beskriver de enskilda stegen i exekveringen.

Betydelsen av aritmetiska och booleska uttryck var:

$$\mathcal{A} : \mathbf{Aexp} \rightarrow (\mathbf{State} \rightarrow \mathbf{Z})$$

$$\mathcal{B} : \mathbf{Bexp} \rightarrow (\mathbf{State} \rightarrow \mathbf{T})$$

Vad är betydelsen av en sats? En sats *ändrar* tillståndet:

$$\mathcal{S} : \mathbf{Stm} \rightarrow (\mathbf{State} \hookrightarrow \mathbf{State})$$

Betydelsen specificeras med ett övergångssystem:

$$\langle S_0, s_0 \rangle \triangleright \langle S_1, s_1 \rangle \triangleright \dots \triangleright \langle S_n, s_n \rangle \triangleright s$$

där $S_i \in \mathbf{Stm}$ är satser som exekveras i ett tillstånd $s_i \in \mathbf{State}$, och s är det slutgiltiga tillståndet.

Vi specificerar betydelsen med övergångsrelationen $\triangleright$. Detta görs på olika sätt i naturlig ($\triangleright = \rightarrow$) och strukturell ($\triangleright = \Rightarrow$) semantik.

Naturlig semantik

I naturlig semantik relaterar vi initiala och slutgiltiga tillståndet med varandra. En sådan övergång skrivs:

$$\langle S, s \rangle \rightarrow s'$$

”Exekveringen av S i s terminerar och resulterar i s' ”

Definitionen av $\rightarrow$ ges med regler på formen

$$\frac{\langle S_1, s_1 \rangle \rightarrow s'_1 \quad \cdots \quad \langle S_n, s_n \rangle \rightarrow s'_n}{\langle S, s \rangle \rightarrow s'} \quad \text{om } \dots$$

där $S_1, \dots, S_n$ är deluttryck i S . Regeln tolkas som:

$$\frac{\text{premiss}_1 \quad \cdots \quad \text{premiss}_n}{\text{slutsats}} \quad \text{om } \dots$$

Regler utan premisser kallas *axiom* och saknar streck.

Naturlig semantik för While

$$\langle x := a, s \rangle \rightarrow s[x \mapsto \mathcal{A}[[a]]s]$$

$$\langle \text{skip}, s \rangle \rightarrow s$$

$$\frac{\langle S_1, s \rangle \rightarrow s' \quad \langle S_2, s' \rangle \rightarrow s''}{\langle S_1; S_2, s \rangle \rightarrow s''}$$

$$\frac{\langle S_1, s \rangle \rightarrow s'}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s'} \quad \text{om } \mathcal{B}[[b]]s = \text{tt}$$

$$\frac{\langle S_2, s \rangle \rightarrow s'}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s'} \quad \text{om } \mathcal{B}[[b]]s = \text{ff}$$

$$\frac{\langle S, s \rangle \rightarrow s' \quad \langle \text{while } b \text{ do } S, s' \rangle \rightarrow s''}{\langle \text{while } b \text{ do } S, s \rangle \rightarrow s''} \quad \text{om } \mathcal{B}[[b]]s = \text{tt}$$

$$\langle \text{while } b \text{ do } S, s \rangle \rightarrow s \quad \text{om } \mathcal{B}[[b]]s = \text{ff}$$

Exempel: s_0 : initialt tillstånd, $\forall x \in \mathbf{Var}, s_0(x) = 0$

$$\langle x := x+1, s_0 \rangle \rightarrow s_0[x \mapsto 1] \qquad \text{ty } \mathcal{A}[\![x+1]\!]s_0 = 1$$

Exempel:

$$\langle \text{skip}, s_0 \rangle \rightarrow s_0 \qquad \text{ty skip ändrar inte } s_0.$$

Exempel: De två föregående instanserna ger:

$$\frac{\langle \text{skip}, s_0 \rangle \rightarrow s_0 \quad \langle x := x+1, s_0 \rangle \rightarrow s_0[x \mapsto 1]}{\langle \text{skip}; x := x+1, s_0 \rangle \rightarrow s_0[x \mapsto 1]}$$

Exempel: Eftersom $\mathcal{B}[\![x=0]\!]s_0 = \text{tt}$ så:

$$\frac{\langle \text{skip}, s_0 \rangle \rightarrow s_0}{\langle \text{if } x=0 \text{ then skip else } x := x+1, s_0 \rangle \rightarrow s_0}$$

Härledningsträd

För att härleda $\langle S, s \rangle \rightarrow s'$ konstrueras ett träd där $\langle S, s \rangle \rightarrow s'$ är rot och instanser av axiom är löv.

Exempel: $(z := x; x := y); y := z$

$$\frac{\frac{\langle z := x, s_0 \rangle \rightarrow s_1 \quad \langle x := y, s_1 \rangle \rightarrow s_2}{\langle z := x; x := y, s_0 \rangle \rightarrow s_2} \quad \langle y := z, s_2 \rangle \rightarrow s_3}{\langle (z := x; x := y); y := z, s_0 \rangle \rightarrow s_3}$$

där

$$\begin{aligned} s_0 &= \text{initialtillstånd, } s_0(x) = 5, s_0(y) = 7 \\ s_1 &= s_0[z \mapsto 5] \\ s_2 &= s_1[x \mapsto 7] \\ s_3 &= s_2[y \mapsto 5] = s_0[z \mapsto 5][x \mapsto 7][y \mapsto 5] \end{aligned}$$

Härledningsträd konstrueras enklast ”bottom-up”.
Om flera regler matchar måste man prova sig fram.

Terminering och loopar

Exekveringen av en sats S i ett tillstånd s sägs

- *terminera*, omm $\exists s', \langle S, s \rangle \rightarrow s'$.
- *loopa*, omm $\neg \exists s', \langle S, s \rangle \rightarrow s'$.
- *terminera alltid*, om den terminerar för alla s .
- *loopa alltid*, om den loopar för alla s .

Bevis av egenskaper mha härledningsträd

För att visa att två satser S_1 och S_2 är ekvivalenta:

$$\langle S_1, s \rangle \rightarrow s' \iff \langle S_2, s \rangle \rightarrow s'$$

$\Rightarrow$: Antag att det finns ett träd för $\langle S_1, s \rangle \rightarrow s'$.

Analysera hur det trädet måste se ut.

Försök sedan att bygga ett träd för $\langle S_2, s \rangle \rightarrow s'$.

$\Leftarrow$: Antag att det finns ett träd för $\langle S_2, s \rangle \rightarrow s'$, osv.

Analysera hur det trädet måste se ut.

Försök sedan att bygga ett träd för $\langle S_1, s \rangle \rightarrow s'$.

Uppgift 2.6: Visa att $S_1; (S_2; S_3) \iff (S_1; S_2); S_3$.

Bevis:

$\Rightarrow$: Antag $\langle S_1; (S_2; S_3), s_0 \rangle \rightarrow s_3$.

Då måste det finnas ett härledningsträd:

$$\frac{\frac{\vdots}{\langle S_1, s_0 \rangle \rightarrow s_1} \quad \frac{\frac{\vdots}{\langle S_2, s_1 \rangle \rightarrow s_2} \quad \frac{\vdots}{\langle S_3, s_2 \rangle \rightarrow s_3}}{\langle S_2; S_3, s_1 \rangle \rightarrow s_3}}{\langle S_1; (S_2; S_3), s_0 \rangle \rightarrow s_3}$$

Observera de tre beståndsdelarna:

$$\frac{\vdots}{\langle S_1, s_0 \rangle \rightarrow s_1} \quad \frac{\vdots}{\langle S_2, s_1 \rangle \rightarrow s_2} \quad \frac{\vdots}{\langle S_3, s_2 \rangle \rightarrow s_3}$$

Kan nu bygga ett träd för $\langle (S_1; S_2); S_3, s_0 \rangle \rightarrow s_3$.

$$\frac{\frac{\frac{\vdots}{\langle S_1, s_0 \rangle \rightarrow s_1} \quad \frac{\vdots}{\langle S_2, s_1 \rangle \rightarrow s_2}}{\langle S_1; S_2, s_0 \rangle \rightarrow s_2} \quad \frac{\vdots}{\langle S_3, s_2 \rangle \rightarrow s_3}}{\langle (S_1; S_2); S_3, s_0 \rangle \rightarrow s_3}$$

$\Leftarrow$: Tvärtom.

Eftersom semantiken för `while` är definierat mha sig själv,

$$\frac{\langle S, s \rangle \rightarrow s' \quad \langle \text{while } b \text{ do } S, s' \rangle \rightarrow s''}{\langle \text{while } b \text{ do } S, s \rangle \rightarrow s''} \quad \text{om } \mathcal{B}[[b]]s = \text{tt}$$

kan vi inte göra strukturinduktion på satser utan måste göra...

Induktionsbevis på härledningsträd

För att visa en egenskap P på ett träd

- **Basfall:** Visa att P gäller för enkla träd.
- **Induktivt:** För ett sammansatt träd:

$$\frac{T_1 \quad \dots \quad T_n}{T}$$

Visa $P(T_1) \wedge \dots \wedge P(T_n) \Rightarrow P(T)$.

Definition: Semantiken är *deterministisk* om

$$\langle S, s \rangle \rightarrow s' \wedge \langle S, s \rangle \rightarrow s'' \Rightarrow s' = s''$$

för alla tänkbara S, s, s' och s'' .

Sats 2.9: De regler vi gett är deterministiska.

Bevis: Induktion över alla härledningsträd.

Antag att $\langle S, s \rangle \rightarrow s'$.

Visa att om $\langle S, s \rangle \rightarrow s''$ så är $s' = s''$.

Basfall:

- **Tilldelning:** $\langle x := a, s \rangle \rightarrow s'$

Bara en regel, den ger $s' = s'' = s[x \mapsto \mathcal{A}[[a]]s]$.

- **Skip:** $\langle \text{skip}, s \rangle \rightarrow s'$

Bara en regel, den ger $s' = s'' = s$.

Induktivt: IH = $\langle S, s \rangle \rightarrow s' \wedge \langle S, s \rangle \rightarrow s'' \Rightarrow s' = s''$

- **Sekvens:** $\langle S_1; S_2, s \rangle \rightarrow s'$

Eftersom det bara finns en regel för ”;”:

$$\langle S_1; S_2, s \rangle \rightarrow s' \Rightarrow \langle S_1, s \rangle \rightarrow s_0 \wedge \langle S_2, s_0 \rangle \rightarrow s'$$

$$\langle S_1; S_2, s \rangle \rightarrow s'' \Rightarrow \langle S_1, s \rangle \rightarrow s_1 \wedge \langle S_2, s_1 \rangle \rightarrow s''$$

Applicera IH på $\langle S_1, s \rangle \rightarrow s_0$ vilket ger $s_0 = s_1$.

Applicera IH på $\langle S_2, s_0 \rangle \rightarrow s'$ vilket ger $s' = s''$.

- **If:** $\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s'$

- $\mathcal{B}[[b]]s = \text{tt}$ och $\langle S_1, s \rangle \rightarrow s'$:

Eftersom $\mathcal{B}[[b]]s = \text{tt}$ finns bara en regel för

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s''$$

Alltså måste $\langle S_1, s \rangle \rightarrow s''$. IH $\Rightarrow s' = s''$.

- $\mathcal{B}[[b]]s = \text{ff}$ och $\langle S_2, s \rangle \rightarrow s'$: Analogt.

- **while:** $\langle \text{while } b \text{ do } S, s \rangle \rightarrow s'$

- $\mathcal{B}[[b]]s = \text{tt}$, $\langle S, s \rangle \rightarrow s_0$, $\langle \text{while } b \text{ do } S, s_0 \rangle \rightarrow s'$:

Eftersom $\mathcal{B}[[b]]s = \text{tt}$ finns bara en regel för

$$\langle \text{while } b \text{ do } S, s \rangle \rightarrow s''$$

vilket betyder att

$$\langle S, s \rangle \rightarrow s_1 \quad \wedge \quad \langle \text{while } b \text{ do } S, s_1 \rangle \rightarrow s''$$

IH på $\langle S, s \rangle \rightarrow s_0$ och $\langle S, s \rangle \rightarrow s_1 \Rightarrow s_0 = s_1$.

$$\frac{\langle S, s \rangle \rightarrow s_0 \quad \langle \text{while } b \text{ do } S, s_0 \rangle \rightarrow s'}{\langle \text{while } b \text{ do } S, s_0 \rangle \rightarrow s'}$$

$$\frac{\langle S, s \rangle \rightarrow s_0 \quad \langle \text{while } b \text{ do } S, s_0 \rangle \rightarrow s''}{\langle \text{while } b \text{ do } S, s_0 \rangle \rightarrow s''}$$

Använd IH på högra premissen $\Rightarrow s' = s''$.

- $\mathcal{B}[[b]]s = \text{ff}$: Trivialt.

□

Semantiska funktionen $\mathcal{S}_{ns}$

Betydelsen av en sats ges nu av

$$\mathcal{S}_{ns} : \mathbf{Stm} \rightarrow (\mathbf{State} \hookrightarrow \mathbf{State})$$

dvs, för varje sats S har vi en partiell funktion

$$\mathcal{S}_{ns} \llbracket S \rrbracket \in \mathbf{State} \hookrightarrow \mathbf{State}$$

som definieras av

$$\mathcal{S}_{ns} \llbracket S \rrbracket s = \begin{cases} s' & \text{om } \langle S, s \rangle \xRightarrow{*} s' \\ \text{undef} & \text{annars} \end{cases}$$

Att observera:

- $\mathcal{S}_{ns}$ är verkligen en funktion tack vare sats 2.9.
- $\mathcal{S}_{ns} \llbracket S \rrbracket$ är partiell pga

$$\mathcal{S}_{ns} \llbracket \text{while true do skip} \rrbracket s = \text{undef}$$

Inlämningsuppgifter

- Uppgift 2.3 (studera exempel 2.2)
- Uppgift 2.7 (studera beviset av lemma 2.5)
- Lab-uppgift 1

Strukturell semantik

Studera istället de *enskilda stegen* i exekveringen

$$\langle S, s \rangle \Rightarrow \gamma$$

Relationen $\Rightarrow$ beskriver det första steget.

Två fall:

- $\gamma = \langle S', s' \rangle$

Exekveringen är inte färdig.

- $\gamma = s'$

Exekveringen har terminerat med sluttillståndet s' .

Exekveringen sägs ha *fastnat* om det inte finns ett γ så att $\langle S, s \rangle \Rightarrow \gamma$.

Strukturell semantik för While

$$\langle x := a, s \rangle \Rightarrow s[x \mapsto \mathcal{A}[a]s]$$

$$\langle \text{skip}, s \rangle \Rightarrow s$$

$$\frac{\langle S_1, s \rangle \Rightarrow \langle S'_1, s' \rangle}{\langle S_1; S_2, s \rangle \Rightarrow \langle S'_1; S_2, s' \rangle}$$

$$\frac{\langle S_1, s \rangle \Rightarrow s'}{\langle S_1; S_2, s \rangle \Rightarrow \langle S_2, s' \rangle}$$

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \Rightarrow \langle S_1, s \rangle \quad \text{om } \mathcal{B}[b]s = \text{tt}$$

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, s \rangle \Rightarrow \langle S_2, s \rangle \quad \text{om } \mathcal{B}[b]s = \text{ff}$$

$$\begin{aligned} \langle \text{while } b \text{ do } S, s \rangle \Rightarrow \\ \langle \text{if } b \text{ then } (S; \text{while } b \text{ do } S) \text{ else skip}, s \rangle \end{aligned}$$

Härledningssekvens

En *härledningssekvens* för $\langle S, s \rangle$ är antingen

- En ändlig sekvens

$$\gamma_0 = \langle S, s \rangle \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_k$$

där γ_k antingen fastnat eller är ett sluttillstånd.

- En oändlig sekvens

$$\gamma_0 = \langle S, s \rangle \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots$$

Notation

$\gamma_0 \xRightarrow{i} \gamma_i$ Exekvera i steg från γ_0 till γ_i .

$\gamma_0 \xRightarrow{*} \gamma_i$ Exekvera ett ändligt antal steg från γ_0 till γ_i .

Exempel: $s_0(x) = 5$, $s_0(y) = 7$ och $(z:=x; x:=y); y:=z$

Första steget:

$$\frac{\frac{\langle z:=x, s_0 \rangle \Rightarrow s_0[z \mapsto 5]}{\langle z:=x; x:=y, s_0 \rangle \Rightarrow \langle x:=y, s_0[z \mapsto 5] \rangle}}{\langle (z:=x; x:=y); y:=z, s_0 \rangle \Rightarrow \langle x:=y; y:=z, s_0[z \mapsto 5] \rangle}$$

Andra steget:

$$\frac{\langle x:=y, s_0[z \mapsto 5] \rangle \Rightarrow s_0[z \mapsto 5][x \mapsto 7]}{\langle x:=y; y:=z, s_0[z \mapsto 5] \rangle \Rightarrow \langle y:=z, s_0[z \mapsto 5][x \mapsto 7] \rangle}$$

Tredje steget:

$$\langle y:=z, s_0[z \mapsto 5][x \mapsto 7] \rangle \Rightarrow s_0[z \mapsto 5][x \mapsto 7][y \mapsto 5]$$

Dvs, vi har en härledningssekvens

$$\begin{aligned} & \langle (z:=x; x:=y); y:=z, s_0 \rangle \\ & \Rightarrow \langle x:=y; y:=z, s_0[z \mapsto 5] \rangle \\ & \Rightarrow \langle y:=z, s_0[z \mapsto 5][x \mapsto 7] \rangle \\ & \Rightarrow s_0[z \mapsto 5][x \mapsto 7][y \mapsto 5] \end{aligned}$$

Övningsuppgift: Visa exekveringen för

$\langle y:=1; \text{ while } \neg(x=1) \text{ do } (y:=y*x; x:=x-1), s \rangle$

i ett tillstånd s så att $s(x) = 3$.

Rätt svar: $s[y \mapsto 6][x \mapsto 1]$.

Definitioner för strukturell semantik

Exekveringen av en sats S i ett tillstånd s sägs

- *terminera*, om det finns en ändlig sekvens

$$\gamma_0 = \langle S, s \rangle \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_k$$

- *terminera lyckligt*, om

$$\langle S, s \rangle \xRightarrow{*} s'$$

för något s' . (dvs $\gamma_k = s'$)

- *terminera alltid*, om den terminerar för alla s .
- *loopa*, om det finns en oändlig sekvens

$$\gamma_0 = \langle S, s \rangle \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots$$

- *loopa alltid*, om den loopar för alla s .

While: "terminera" $\iff$ "terminera lyckligt"

Determinism

Som förut:

$$\langle S, s \rangle \Rightarrow \gamma \text{ och } \langle S, s \rangle \Rightarrow \gamma' \text{ implicerar } \gamma = \gamma'$$

SOS-reglerna för **While** är deterministiska.

Semantisk ekvivalens

Två satser S_1 och S_2 är *semantiskt ekvivalenta* om

- $\langle S_1, s \rangle \xRightarrow{*} \gamma \iff \langle S_2, s \rangle \xRightarrow{*} \gamma$
där γ antingen fastnat eller är ett sluttillstånd.
- $\langle S_1, s \rangle \text{ loopar} \iff \langle S_2, s \rangle \text{ loopar.}$

Semantiska funktionen $\mathcal{S}_{sos}$

Betydelsen av en sats i strukturell semantik ges av

$$\mathcal{S}_{sos} : \mathbf{Stm} \rightarrow (\mathbf{State} \hookrightarrow \mathbf{State})$$

som definieras av

$$\mathcal{S}_{sos} \llbracket S \rrbracket s = \begin{cases} s' & \text{om } \langle S, s \rangle \xRightarrow{*} s' \\ \text{undef} & \text{annars} \end{cases}$$

Sats 2.26: $\mathcal{S}_{ns} \llbracket S \rrbracket = \mathcal{S}_{sos} \llbracket S \rrbracket$.

Beviside':

- Visa med induktion över härledningsträd att varje träd i ns har en motsvarande ändlig härledning i sos.
- Visa med induktion över längden på härledningar i sos att varje sådan härledning har ett motsvarande härledningsträd i ns.

Inlämningsuppgifter

- Uppgift 2.16 (studera exempel 2.15)
- Uppgift 2.23
- Lab-uppgift 2

Utvidgningar av While

Abort

Utöka grammatiken: $S \rightarrow \dots \mid \text{abort}$

Kommandot `abort` avbryter exekveringen.

Implementeras genom att låta konfigurationen fastna, dvs, inga nya regler!

Jämförelse med `while true do skip`:

Naturlig semantik kan inte skilja på loopar och avbruten exekvering:

$$\langle \text{while true do skip}, s \rangle \rightarrow \text{fastnar}$$

Strukturell semantik: loopar $\Rightarrow$ oändlig härledning:

$$\langle \text{while true do skip}, s \rangle \Rightarrow^i \langle \text{while true do skip}, s \rangle \Rightarrow \dots$$

Parallellism

Utöka grammatiken: $S \rightarrow \dots \mid S_1 \text{ par } S_2$

Både S_1 och S_2 körs men exekveringen är interfolierad.

Exempel: $x:=1 \text{ par } (x:=2; x:=x+2)$ ger $x \in \{4, 1, 3\}$.

Naturlig semantik: omöjligt (?)

Strukturell semantik:

$$\frac{\langle S_1, s \rangle \Rightarrow \langle S'_1, s' \rangle}{\langle S_1 \text{ par } S_2, s \rangle \Rightarrow \langle S'_1 \text{ par } S_2, s' \rangle} \quad \frac{\langle S_1, s \rangle \Rightarrow s'}{\langle S_1 \text{ par } S_2, s \rangle \Rightarrow \langle S_2, s' \rangle}$$

$$\frac{\langle S_2, s \rangle \Rightarrow \langle S'_2, s' \rangle}{\langle S_1 \text{ par } S_2, s \rangle \Rightarrow \langle S_1 \text{ par } S'_2, s' \rangle} \quad \frac{\langle S_2, s \rangle \Rightarrow s'}{\langle S_1 \text{ par } S_2, s \rangle \Rightarrow \langle S_1, s' \rangle}$$

Block och Deklarationer

Utöka språket med block och deklARATIONER:

$$\begin{aligned} S &\rightarrow \dots \mid \text{begin } D_V \ S \ \text{end} \\ D_V &\rightarrow \text{var } x := a; \ D_V \mid \varepsilon \end{aligned}$$

Variablerna i en deklARATIONER ges av funktionen DV:

$$\begin{aligned} \text{DV}(\text{var } x := a; \ D_V) &= \{x\} \cup \text{DV}(D_V) \\ \text{DV}(\varepsilon) &= \emptyset \end{aligned}$$

Exempel: Globala x sätts till 4.

```
begin var y := 1;
  (x := 1;
   begin var x := 2; y := x+1 end;
   x := y+x)
end
```

Relationen $\rightarrow_D$

$$\frac{\langle D_V, s[x \mapsto \mathcal{A}[[a]]s] \rangle \rightarrow_D s'}{\langle \text{var } x := a; D_V, s \rangle \rightarrow_D s'}$$

$$\langle \varepsilon, s \rangle \rightarrow_D s$$

Block

$$\frac{\langle D_V, s \rangle \rightarrow_D s' \quad \langle S, s' \rangle \rightarrow s''}{\langle \text{begin } D_V \ S \ \text{end}, s \rangle \rightarrow s''[\text{DV}(D_V) \mapsto s]}$$

där

$$(s'[X \mapsto s])(x) = \begin{cases} s(x) & \text{om } x \in X \\ s'(x) & \text{om } x \notin X \end{cases}$$

Denna operation garanterar att lokala variabler återfår sina ursprungliga värden när man går ur ett block.

Dynamisk och statisk scoping

```
procedure P;  
  var y : integer;  
begin  
  y := x;  
end;
```

P refererar till en icke-lokal variabel x .

- **Dynamiska scoperegler:** Det x som avses finns i *anroparens* block (eller utåt).
- **Statisk scoperegler:** Det x som avses finns i P:s block (eller utåt).

De första LISP-systemen hade dynamiskt scope, men författaren McCarthy medgav sedan att det var ett misstag.

Procedurer

Utöka med deklarationer och anrop av procedurer:

$$\begin{aligned} S &\rightarrow \dots \mid \text{begin } D_V \ D_P \ S \ \text{end} \mid \text{call } p \\ D_P &\rightarrow \text{proc } p \text{ is } S; \ D_P \mid \varepsilon \end{aligned}$$

```
begin var x1 := 0;
      proc p1 is x1 := x1*2;
        proc q is call p1;
          begin var x2 := 5;
            proc p2 is x2 := x2+1;
              call q; y := x2
            end
          end
        end
      end
    (⇒ y = 5)
```

Statisk scoping: icke-lokala variabler tas från procedurdefinitionens omgivning, inte från anroparens.

Environment och Store

Istället för den enkla avbildningen $s \in \mathbf{State} = \mathbf{Var} \rightarrow \mathbf{Z}$ ska vi utföra mappningen i två steg:

$$\begin{aligned} env_V \in \mathbf{Env}_V &= \mathbf{Var} \rightarrow \mathbf{Loc} \\ sto \in \mathbf{Store} &= \mathbf{Loc} \cup \{\text{next}\} \rightarrow \mathbf{Z} \\ \mathbf{Loc} &= \mathbf{Z} \end{aligned}$$

Avsikten är att $s = sto \circ env_V$, där sto fungerar som ett minne och $env_V(x) \in \mathbf{Loc}$ är en minnesadress. Speciellt gäller att $sto(\text{next})$ ger nästa lediga minnescell.

Exempel: Effekt av $x := 17; y := 42$:

env_V		sto	
x	1	next	3
y	2	1	17
		2	42

Värdet av en variabel x : $(sto \circ env_V)(x) = sto(env_V(x))$

För att utföra $x := v$: $sto[env_V(x) \mapsto v]$.

Initialt minne : $sto(next) = 1$

Deklarationer av variabler

$$\langle \varepsilon, env_V, sto \rangle \rightarrow_D (env_V, sto)$$

$$\frac{\langle D_V, env_V[x \mapsto l], sto[l \mapsto v][next \mapsto l + 1] \rangle \rightarrow_D (env'_V, sto')}{\langle \text{var } x := a; D_V, env_V, sto \rangle \rightarrow_D (env'_V, sto')}$$

där $v = \mathcal{A}[a](sto \circ env_V)$ och $l = sto(next)$.

Kommentarer:

- En deklaration orsakar en förändring

$$\langle D_V, env_V, sto \rangle \rightarrow_D (env'_V, sto')$$

- För varje deklaration **var** $x := a$ binds x till $next$ – nästa lediga minnescell – som sedan fylls med värdet av a , varefter $next$ binds om till $next+1$.

Deklaration av procedurer

$$\text{upd}_P(\epsilon, \text{env}_V, \text{env}_P) = \text{env}_P$$

$$\begin{aligned} \text{upd}_P(\text{proc } p \text{ is } S; D_P, \text{env}_V, \text{env}_P) = \\ = \text{upd}_P(D_P, \text{env}_V, \text{env}_P[p \mapsto (S, \text{env}_V, \text{env}_P)]) \end{aligned}$$

Vi måste associera namnet på en procedur med dess kropp och den variabel- och proceduromgivning som råder vid definitionen.

$$\text{env}_P \in \mathbf{Env}_P = \mathbf{Pname} \hookrightarrow \mathbf{Stm} \times \mathbf{Env}_V \times \mathbf{Env}_P$$

Övergångsreglerna kommer nu att vara på formen:

$$\text{env}_V, \text{env}_P \vdash \langle S, \text{sto} \rangle \rightarrow \text{sto}'$$

Notationen är "standard" inom området; skulle lika gärna kunna skrivas som

$$\langle \text{env}_V, \text{env}_P, S, \text{sto} \rangle \rightarrow \text{sto}'$$

Block

$$\frac{\langle D_V, env_V, sto \rangle \rightarrow_D (env'_V, sto') \quad env'_V, upd_P(D_P, env'_V, env_P) \vdash \langle S, sto' \rangle \rightarrow sto''}{env_V, env_P \vdash \langle \text{begin } D_V \ D_P \ S \ \text{end}, sto \rangle \rightarrow sto''}$$

Kommentar:

- Först initieras de deklarerade variablerna vilket ger upphov till nya bindningar env' och sto' .
- Sedan utökas env_P med bindningar för deklarerade procedurer. Observera att dessa bindningar ”kommer ihåg” den omgivning där de skapades.
- I dessa nya bindingar körs S , vilket leder till sto'' .

Tilldelning

$$\begin{aligned} env_V, env_P \vdash \langle x := a, sto \rangle \\ \rightarrow sto[env_V(x) \mapsto \mathcal{A}[[a]](sto \circ env_V)] \end{aligned}$$

Skip

$$env_V, env_P \vdash \langle \text{skip}, sto \rangle \rightarrow sto$$

Sekvens

$$\frac{\begin{array}{l} env_V, env_P \vdash \langle S_1, sto \rangle \rightarrow sto' \\ env_V, env_P \vdash \langle S_2, sto' \rangle \rightarrow sto'' \end{array}}{env_V, env_P \vdash \langle S_1; S_2, sto \rangle \rightarrow sto''}$$

Proceduranrop

$$\frac{env'_V, env'_P[p \mapsto (S, env'_V, env'_P)] \vdash \langle S, sto \rangle \rightarrow sto'}{env_V, env_P \vdash \langle \text{call } p, sto \rangle \rightarrow sto'}$$

där $env_P(p) = (S, env'_V, env'_P)$.

Selektion

- $\mathcal{B}[[b]](sto \circ env_V) = \text{tt}$:

$$\frac{env_V, env_P \vdash \langle S_1, sto \rangle \rightarrow sto'}{env_V, env_P \vdash \langle \text{if } b \text{ then } S_1 \text{ else } S_2, sto \rangle \rightarrow sto'}$$

- $\mathcal{B}[[b]](sto \circ env_V) = \text{ff}$:

$$\frac{env_V, env_P \vdash \langle S_2, sto \rangle \rightarrow sto'}{env_V, env_P \vdash \langle \text{if } b \text{ then } S_1 \text{ else } S_2, sto \rangle \rightarrow sto'}$$

Iteration

- $\mathcal{B}[[b]](sto \circ env_V) = \text{tt}$:

$$\frac{\begin{array}{c} env_V, env_P \vdash \langle S, sto \rangle \rightarrow sto' \\ env_V, env_P \vdash \langle \text{while } b \text{ do } S, sto' \rangle \rightarrow sto'' \end{array}}{env_V, env_P \vdash \langle \text{while } b \text{ do } S, sto \rangle \rightarrow sto''}$$

- $\mathcal{B}[[b]](sto \circ env_V) = \text{ff}$:

$$env_V, env_P \vdash \langle \text{while } b \text{ do } S, sto \rangle \rightarrow sto$$

Inlämningsuppgifter

- Uppgift 2.42
- Uppgift 2.43

Antag att y har minnesadress 1.
Efter exekveringen bör $sto(1) = 5$.

4 Denotationssemantik

Princip

Avbilda programmet på en funktion $\text{State} \rightarrow \text{State}$.

- Varje *enkel* syntaktiskt konstruktion avbildas på en matematisk funktion som beskriver vad som händer om konstruktionen körs.
- Betydelsen av en *sammansatt* konstruktion definieras genom sammansättning av betydelsen för delarna.

Olika stilar

- Direct style semantics
- Continuation style semantics

Direct style semantics

Betydelsen av att köra satsen S definieras av funktionen

$$\mathcal{S}_{ds} : \mathbf{Stm} \rightarrow (\mathbf{State} \hookrightarrow \mathbf{State})$$

Första ansats

$$\mathcal{S}_{ds} \llbracket x := a \rrbracket s = s[x \mapsto \mathcal{A} \llbracket a \rrbracket s]$$

$$\mathcal{S}_{ds} \llbracket \mathbf{skip} \rrbracket s = s$$

$$\mathcal{S}_{ds} \llbracket S_1; S_2 \rrbracket s = \mathcal{S}_{ds} \llbracket S_2 \rrbracket (\mathcal{S}_{ds} \llbracket S_1 \rrbracket s)$$

$$\begin{aligned} \mathcal{S}_{ds} \llbracket \mathbf{if } b \mathbf{ then } S_1 \mathbf{ else } S_2 \rrbracket s &= \mathbf{if } \mathcal{B} \llbracket b \rrbracket s \\ &\quad \mathbf{then } \mathcal{S}_{ds} \llbracket S_1 \rrbracket s \\ &\quad \mathbf{else } \mathcal{S}_{ds} \llbracket S_2 \rrbracket s \end{aligned}$$

$$\mathcal{S}_{ds} \llbracket \mathbf{while } b \mathbf{ do } S \rrbracket s = ?$$

Hur ska loopar definieras?

$$\begin{aligned}\mathcal{S}_{ds}[\text{while } b \text{ do } S]s &= \\ &= \text{if } \mathcal{B}[b]s \text{ then } \mathcal{S}_{ds}[\text{while } b \text{ do } S](\mathcal{S}_{ds}[S]s) \text{ else } s\end{aligned}$$

Bryter mot principen ”betydelsen av en sammansatt konstruktion definieras genom sammansättning av betydelsen för delarna”!

Problemet med rekursiva definitioner

Sätt $X = \mathcal{S}_{ds}[\text{while } b \text{ do } S]$. Då har vi:

$$Xs = \text{if } \mathcal{B}[b]s \text{ then } X(\mathcal{S}_{ds}[S]s) \text{ else } s$$

Detta är ju ingen definition, utan en *ekvation*!
Vi söker det X som löser ekvationen.

Vad är en funktion?

- **Programmeringsspråk:** Inargument, en samling instruktioner och ett returvärde.
- **Matematik:** En avbildning $A \rightarrow B$.

Det vore trevligt om dessa två betydelser sammanföll.

En funktion kan representeras explicit med dess *graf*:

$$\text{graph}(\text{double}) = \{(0, 0), (1, 2), (2, 4), (3, 6), (4, 8), \dots\}$$

Lättare med en ändlig beskrivning:

$$\text{double}(x) = 2x$$

Två funktioner f och g är lika om $f(x) = g(x)$ för alla x .

Flera algoritmer har samma funktion som betydelse, t ex

$$\text{double}(x) = x + x$$

Rekursion

De flesta språk tillåter rekursiva definitioner:

$$fac(n) = \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * fac(n - 1)$$

Detta är en *ekvation* vars lösning vi hoppas är *fac*!

Alla ekvationer har dock inte lösningar:

$$f(x) = f(x) + 1$$

Hur ska dessa behandlas?

Betrakta följande ekvation:

$$\begin{aligned} f(x) = & \text{ if } x \equiv 0 \text{ then } 1 \\ & \text{ else if } x \equiv 1 \text{ then } f(3) \\ & \text{ else } f(x - 2) \end{aligned}$$

Vilken av följande lösningar är "bäst", tycker du?

$$\begin{aligned} f(x) &= \begin{cases} 1 & \text{om } x \geq 0 \text{ är jämnt} \\ \text{undef} & \text{annars} \end{cases} \\ f(x) &= 1 \quad \text{för alla } x \\ f(x) &= \begin{cases} 1 & \text{om } x \geq 0 \text{ är jämnt} \\ a & \text{om } x \geq 0 \text{ är udda} \\ b & \text{annars} \end{cases} \end{aligned}$$

Integritetskrav:

- Varje rekursiv definition måste ha en lösning.
- Om det finns flera lösningar måste vi ange vilken som avses.

λ -kalkyl

Studiet av funktioner som *regler*.

I vanlig matematik måste man vanligtvis namnge en funktion för att använda den:

$$f(x) = 2x$$

I andra sammanhang representeras en funktion av det uttryck som definierar den:

$$\frac{d}{dx}(x^2 + 2) = 2x$$

$$\int z \, dz = \frac{z^2}{2} + c$$

men man måste också veta vilken variabel som är oberoende.

Abstraktion

Alonzo Church införde beteckningen

$$\lambda x.E$$

för att beteckna en (anonym) funktion som tar ett argument x och returnerar värdet av E .

Exempel: $\lambda x.x^2 + 2$ betecknar funktionen

$$\{(1, 3), (2, 6), (3, 11), (4, 18), (5, 27), \dots\}$$

Exempel: Låt D beteckna derivering.

$$D(\lambda x.x^2 + 2) = \lambda x.2x$$

λ -kalkyl:	$\lambda x.E$
LISP:	(lambda (x) E)
Gofer:	\x -> E

Applikation

Givet en funktion f och ett argument a används

$$fa$$

för att beteckna f :s värde för a ("f appliceras på a ").

Exempel: $(\lambda x.x^2 + 2) 3 = 3^2 + 2$

Flera argument

Alla funktioner i λ -kalkyl tar ett argument. Man kan simulera en funktion som tar fler argument genom att returnera funktioner.

$$(\lambda x_1.(\dots(\lambda x_n.E)\dots))$$

Exempel: $plus = \lambda x.(\lambda y.x + y) \in \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$

$$plus\ 2 = \lambda y.2 + y \in \mathbb{N} \rightarrow \mathbb{N}$$

$$(plus\ 2)\ 3 = 2 + 3 \in \mathbb{N}$$

Paranteskonventioner

$(\lambda x_1.(\dots(\lambda x_n.E)\dots))$ skrivs $\lambda x_1 \dots x_n.E$

$(\dots((fa_1)a_2)\dots a_n)$ skrivs $fa_1a_2\dots a_n$

Observera:

- Applikation är vänsterassociativ:

$$xz(yz) = ((xz)(yz)) \neq (x(z(yz)))$$

- Ett λ når "så långt som möjligt":

$$\lambda x.xyz \neq (\lambda x.x)yz$$

Uppgift: Givet följande definitioner, visa att $SKK \equiv I$

$$\begin{aligned} I &= \lambda x.x \\ K &= \lambda xy.x \\ S &= \lambda xyz.xz(yz) \end{aligned}$$

Lösning:

$$\begin{aligned} SKK &= (\lambda xyz.xz(yz))KK \\ &= (\lambda yz.Kz(yz))K \\ &= \lambda z.Kz(Kz) \\ &= \lambda z.(\lambda xy.x)z(Kz) \\ &= \lambda z.(\lambda y.z)Kz \\ &= \lambda z.z = I \end{aligned}$$

Gofer:

$$\begin{aligned} k &= \lambda x y \rightarrow x \\ s &= \lambda x y z \rightarrow x z (y z) \\ i &= s k k \end{aligned}$$

I fortsättningen kommer vi inte att skilja mellan

$$\boxed{f(n) = E} = \boxed{f \ n = E} = \boxed{f = \lambda n. E}$$

Däremot så

$$\boxed{f(x, y) = E} \neq \boxed{f = \lambda x. \lambda y. E} = \boxed{f \ x \ y = E}$$

eftersom den första tar ett argument, en tupel (x, y) , medan de två andra tar två argument, ett i taget.

Om argumenten är heltal så har vi följande typer:

$$\boxed{\underbrace{f(x, y) = E}_{(\mathbb{N} \times \mathbb{N}) \rightarrow \mathbb{N}}} \neq \boxed{\underbrace{f = \lambda x. \lambda y. E}_{\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}}} = \boxed{\underbrace{f \ x \ y = E}_{\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}}}$$

Vi kommer också att använda ”utökningar” som **if-then-else** samt aritmetiska operationer. Dessa går med viss möda att simulera i ren λ -kalkyl.

Aritmetik

Inför följande beteckningar:

$$0 = \lambda f x. x$$

$$1 = \lambda f x. f x$$

$$2 = \lambda f x. f(f x)$$

$$\vdots$$

$$n = \lambda f x. \underbrace{f(f(\dots(f x)))}_n$$

$$true = \lambda x y. x$$

$$false = \lambda x y. y$$

$$succ = \lambda n s x. s(n s x)$$

$$pred = \lambda k. (k(\lambda p u. u(succ(p true))(p true))(\lambda u. u 0 0)) false$$

$$add = \lambda m n f x. m f(n f x)$$

$$mul = \lambda m n f x. m(n f) x$$

$$iszero = \lambda n. n(true false) true$$

Exempel:

$$\begin{aligned} succ\ 0 &= (\lambda n s x. s(n s x))(\lambda f x. x) \\ &= \lambda s x. s((\lambda f x. x) s x) \\ &= \lambda s x. s((\lambda x. x) x) \\ &= (\lambda s x. s x) = 1 \end{aligned}$$

$$\begin{aligned} true\ f_T\ f_F &= f_T \\ false\ f_T\ f_F &= f_F \end{aligned}$$

$$\begin{aligned} iszero\ 0 &= (\lambda n. (n(true\ false)\ true))\ 0 \\ &= 0\ (true\ false)\ true \\ &= (\lambda f x. x)(true\ false)\ true = true \end{aligned}$$

$$\begin{aligned} iszero\ 1 &= (\lambda n. (n(true\ false)\ true))\ 1 \\ &= 1\ (true\ false)\ true \\ &= (\lambda f x. f x)(true\ false)\ true \\ &= (true\ false)\ true = false \end{aligned}$$

Loopar

Man kan även simulera loopar med följande uttryck:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

Y har den intressanta egenskapen att

$$Yf = f(Yf)$$

Bevis:

$$\begin{aligned} Yf &= (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))f \\ &= (\lambda x.f(xx))(\lambda x.f(xx)) \\ &= f(\underbrace{(\lambda x.f(xx))(\lambda x.f(xx))}_{\text{rad 2}}) \\ &= f(Yf) \end{aligned}$$

Yf är tydligen en fixpunkt till f !

Rekursion och fixpunkter

Låt oss återvända till

$$fac = \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * fac(n - 1)$$

Antag att vi inte får referera till fac i högerledet.

En serie successiva approximationer till fac :

$$\begin{aligned} f_0 &= \lambda n. undef \\ f_1 &= \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * f_0(n - 1) \\ &\vdots \\ f_{i+1} &= \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * f_i(n - 1) \end{aligned}$$

Som synes är f_i likvärdig med fac för argument $n < i$.
Speciellt gäller $f_\infty = fac$. Inför nu följande funktion:

$$H = \lambda f. \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * f(n - 1)$$

så att $H f_i = f_{i+1}$. Speciellt gäller $H fac = fac$, dvs fac är en fixpunkt till H . Alltså:

$fac = Y \ H$

Kan detta verkligen fungera? Vi provar:

$$\begin{aligned} YH\ 3 &= H(YH)\ 3 \\ &= \text{if } 3 \equiv 0 \text{ then } 1 \text{ else } 3 * (YH)(2) \\ &= 3 * (YH)(2) \\ &\vdots \\ &= 3 * 2 * 1 * (YH)(0) \\ &= 3 * 2 * 1 * (H(YH)\ 0) \\ &= 3 * 2 * 1 * (\text{if } 0 \equiv 0 \text{ then } 1 \text{ else } \dots) \\ &= 3 * 2 * 1 * 1 = 6 \end{aligned}$$

OBS: Ingen rekursion, vare sig i beräkningen eller definitionen $fac = YH$.

Mer om Y

En *operator* är en funktion som tar funktioner som argument.

Y kallas *fixpunktsoperator* och betecknas ibland *fix*.

Gofer kan tyvärr inte typa $Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$ men man kan skriva

```
fix f = f(fix f)
```

Testa själv att köra

```
fix (\f n -> if n==0 then 1 else n*f(n-1)) 6
```

Svaret blir 720. (Fungerar bara i lata funktionella språk.)

Funktionaler

Betrakta återigen den rekursiva definitionen

$$fac = \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * fac(n - 1)$$

och den icke-rekursiva definitionen $fac = fix H$ där

$$H = \lambda f. \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * f(n - 1)$$

H kallas fac :s *funktional*.

Givet en rekursiv definition av en funktion $f(n)$

$$f = \lambda n. \dots f(g(n)) \dots$$

konstrueras funktionalen F genom att abstrahera ut f :

$$F = \lambda f. \lambda n. \dots f(g(n)) \dots$$

Och sedan erhålles lösningen

$$f = fix \ F$$

Direct Style Semantics

Ett par problem kvarstår dock:

- Vissa funktionaler har mer än en fixpunkt.

$$F = \lambda f.f$$

- Vissa funktionaler saknar fixpunkt.

$$F = \lambda f.\text{if } f \equiv f_2 \text{ then } f_1 \text{ else } f_2$$

Om $f_1 \neq f_2$ så finns inget f så att $Ff = f$.

Vi vill skriva $\mathcal{S}_{ds}[\text{while } b \text{ do } S] = \text{fix}(F)$ för något F .
Hur vet vi att F har en fixpunkt? Om det finns fler,
vilken ska vi välja?

Betrakta `while b do S` i ett tillstånd s_0 . Tre fall:

1. Konstruktionen terminerar.
2. Konstruktionen loopar i S .
3. Konstruktionen loopar pga b alltid är sant.

Antag att $\text{fix}(F) = g$.

1. **Terminering:**

$g^\infty(s_0) = s_n$ (sluttillstånd). Vilket g som helst duger.

2. **Loopar i S :**

$g^\infty(s_0) = \text{undef}$. Vilket g som helst duger.

3. **Loopar pga $\mathcal{B}[[b]] = \text{tt}$:**

$g(s_0) = s_i$ för alla i . Flera sätt att välja g .

Vilken fixpunkt ska vi välja? Studera

`while  $\neg(x=0)$  do skip`

Ger upphov till följande funktional:

$$F = \lambda g. \lambda s. \mathbf{if} \ s(x) \neq 0 \ \mathbf{then} \ g(s) \ \mathbf{else} \ s$$

Vilken funktion $g = \lambda f. f$ som helst är fixpunkt till F , ty

$$F(\lambda f. f) = \lambda s. \mathbf{if} \ s(x) \neq 0 \ \mathbf{then} \ s \ \mathbf{else} \ s$$

Men den fixpunkt som verkar mest intuitiv är

$$g_0 = \lambda s. \mathbf{if} \ s(x) \neq 0 \ \mathbf{then} \ \mathit{undef} \ \mathbf{else} \ s$$

Denna fixpunkt har en viktig egenskap:

$$g_0 \ s = s' \quad \Rightarrow \quad g \ s = s'$$

för varje annan fixpunkt g , men inte vice versa.

Beslut angående fixpunkter

Fixpunkten $fix(F)$ ska vara en funktion g_0 så att:

- $F(g_0) = g_0$
- För varje annan fixpunkt g gäller

$$g_0 \ s = s' \quad \Rightarrow \quad g \ s' = s'$$

(Om $g_0 \ s = \text{undef}$ finns inga krav på $g \ s$.)

5 Beräkningsmodeller

Turing-maskinen

- Maskinen förses med indata.
- Maskinen utför beräkningen.
- Om beräkningen terminerar så presenteras resultatet.

Ingen skillnad görs på

- `while true do skip;`
- `while true do write(1);`

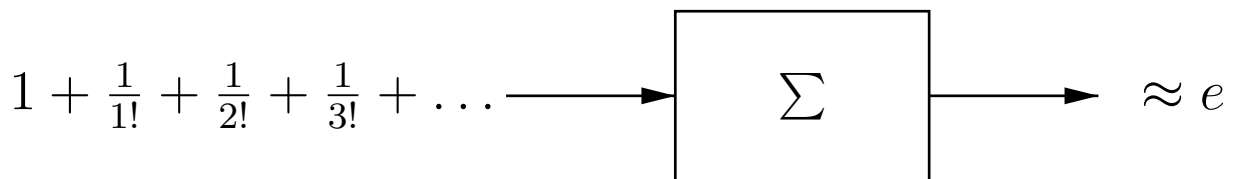
Känns inte helt korrekt, speciellt med dagens interaktiva system.

En modell för icke-terminerande program

Behandla ändliga approximationer till oändliga objekt.

Generalisering:

- En beräkning ger successivt allt bättre approximationer av utdata.
- I denna process behövs bättre och bättre approximationer av indata.



Varje program beräknar en *monoton* funktion:

- Bättre approximation av indata $\Rightarrow$
Bättre (inte sämre) approximation av utdata.

Domänteori

Partialordning

En relation $\preceq$ över en mängd $\mathbf{A}$ kallas *partialordning* om

- $\preceq$ är reflexiv, dvs

$$a \preceq a \text{ för alla } a \in \mathbf{A}$$

- $\preceq$ är transitiv, dvs

$$(a_1 \preceq a_2) \wedge (a_2 \preceq a_3) \Rightarrow a_1 \preceq a_3$$

- $\preceq$ är anti-symmetrisk, dvs,

$$(a_1 \preceq a_2) \wedge (a_2 \preceq a_1) \Rightarrow a_1 = a_2$$

$\langle \mathbf{A}, \preceq \rangle$ kallas ett *poset* (“partially ordered set”).

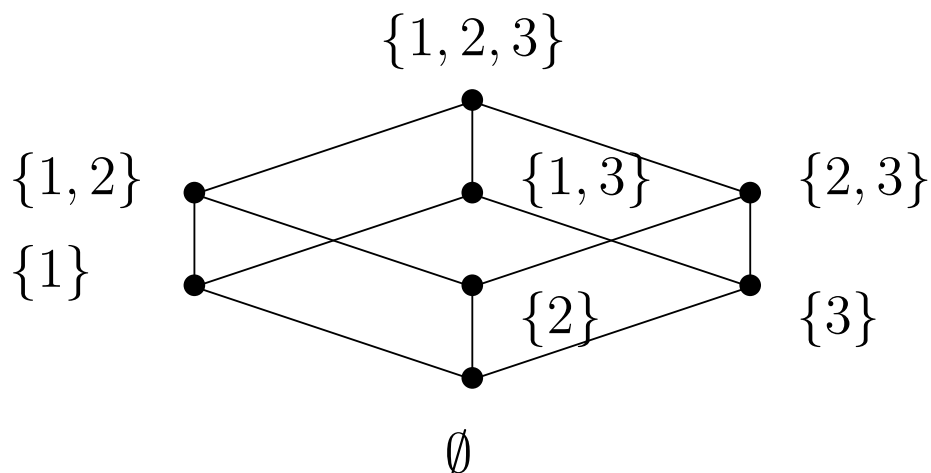
Ett poset kan åskådliggöras med ett Hasse-diagram:

Hasse-diagram: $a_1 \preceq a_2$ representeras av en båge så att a_1 ligger *under* a_2 . Reflexiva och transitiva bågar visas inte.

Exempel: $\langle \{1, 2, 3\}, \leq \rangle$



Exempel: $\langle \mathcal{P}(\{1, 2, 3\}), \subseteq \rangle$



Notation

Totalordning

Ett poset $\langle A, \preceq \rangle$ är en *totalordning* om

$$(\forall a_1, a_2 \in \mathbf{A}) \ a_1 \preceq a_2 \vee a_2 \preceq a_1$$

Exempel: $\langle \{1, 2, 3\}, \leq \rangle$ är en totalordning.

Exempel: $\langle \mathcal{P}(\{1, 2, 3\}), \subseteq \rangle$ är inte en totalordning.

Partialordning för funktioner

För två funktioner $g_1, g_2 \in \mathbf{A} \hookrightarrow \mathbf{B}$, definiera

$$g_1 \sqsubseteq g_2 \quad \Longleftrightarrow \quad (g_1 \ x = x' \Rightarrow g_2 \ x = x')$$

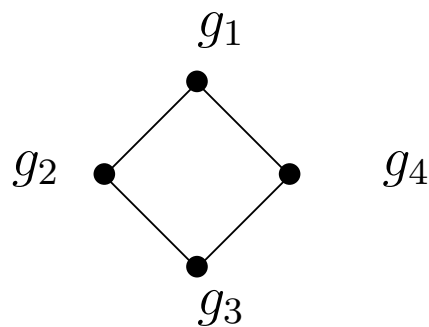
Exempel

$$g_1 = \lambda x.x$$

$$g_2 = \lambda x.\text{if } x \geq 0 \text{ then } x \text{ else } \textit{undef}$$

$$g_3 = \lambda x.\text{if } x = 0 \text{ then } x \text{ else } \textit{undef}$$

$$g_4 = \lambda x.\text{if } x \leq 0 \text{ then } x \text{ else } \textit{undef}$$



Karaktärisering med graph

Tidigare beskrev vi en funktion med dess graf:

$$\text{graph}(f) = \{\langle x, y \rangle \mid f\ x = y\}$$

Sats:

$$f_1 \sqsubseteq f_2 \quad \Longleftrightarrow \quad \text{graph}(f_1) \subseteq \text{graph}(f_2)$$

Bevis: $f_1 \sqsubseteq f_2$

$$\Longleftrightarrow (\forall x)(f_1\ x = y \Rightarrow f_2\ x = y)$$

$$\Longleftrightarrow (\forall x)(\langle x, f_1\ x \rangle \in \text{graph}(f_1) \Rightarrow \langle x, f_2\ x \rangle \in \text{graph}(f_2))$$

$$\Longleftrightarrow \text{graph}(f_1) \subseteq \text{graph}(f_2)$$

Exempel:

$$\text{graph}(g_1) = \{\langle 0, 0 \rangle, \langle 1, 1 \rangle, \langle -1, -1 \rangle, \langle 2, 2 \rangle, \dots\}$$

$$\text{graph}(g_2) = \{\langle 0, 0 \rangle, \langle 1, 1 \rangle, \langle 2, 2 \rangle, \dots\}$$

Minsta och största element

$\langle \mathbf{A}, \preceq \rangle$ — poset.

- $a \in \mathbf{A}$ är största element omm $(\forall a' \in \mathbf{A}) a' \preceq a$.
- $a \in \mathbf{A}$ är minsta element omm $(\forall a' \in \mathbf{A}) a \preceq a'$.

Sats 4.9: Om ett poset har ett minsta element är detta unikt.

Bevis: Antag $\langle \mathbf{A}, \preceq \rangle$ har två minsta element a_1 och a_2 .

Då gäller:

$$a_1 \preceq a_2 \text{ och } a_2 \preceq a_1$$

Men eftersom $\preceq$ är anti-symmetrisk måste $a_1 = a_2$.

Definition: Det minsta elementet i ett poset $\langle \mathbf{A}, \preceq \rangle$ betecknas $\perp_A$ (“bottom”).

Lemma 4.13

$$\langle \mathbf{State} \hookrightarrow \mathbf{State}, \sqsubseteq \rangle$$

är ett poset där

$$\perp = \lambda s. \mathit{undef}$$

är det minsta elementet.

Bevis: Betrakta funktionernas grafer.

- $\sqsubseteq$ är en partialordning, ty $\subseteq$ är en.
- $\perp$ är minsta elementet ty $\mathit{graph}(\perp) = \emptyset$.

Inlämningsuppgifter

- Uppgift 4.7
- Uppgift 4.11

Övre begränsningar och kedjor

$\langle D, \sqsubseteq \rangle$ — poset. Betrakta en delmängd $Y \subseteq D$.

- $d \in D$ kallas en *övre begränsning* av Y om

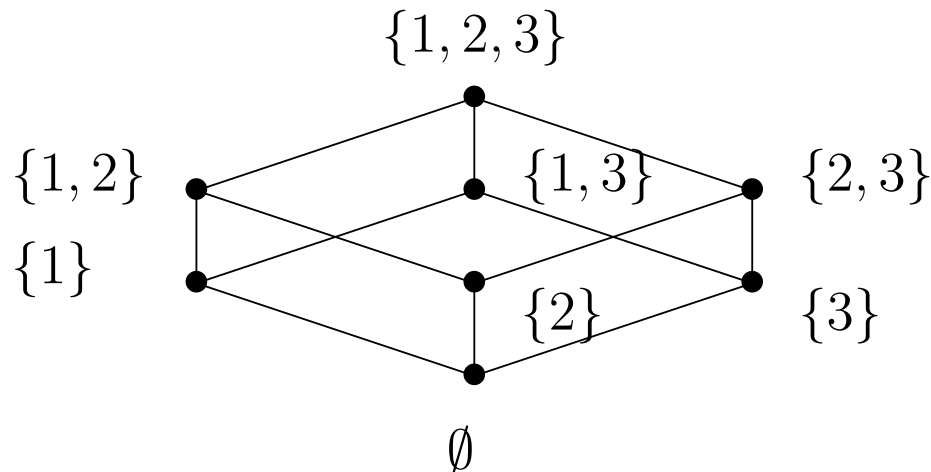
$$(\forall d' \in Y) d' \sqsubseteq d$$

- $d \in D$ kallas en *minsta* övre begränsning av Y om

$$d' \text{ är en övre begränsning av } Y \Rightarrow d \sqsubseteq d'.$$

- Om Y har en minsta övre begränsning betecknas denna $\sqcup Y$ ("möben av Y "). Denna är då unik.
- Om $\langle Y, \sqsubseteq \rangle$ är en totalordning kallas Y för en *kedja*.

Exempel: $\langle \mathcal{P}(\{1, 2, 3\}), \subseteq \rangle$



- $Y_0 = \{\emptyset, \{1\}, \{1, 2\}\}$ är en kedja.
- $\{1, 2, 3\}$ och $\{1, 2\}$ är övre begränsningar av Y_0 .
- $\{1, 2\}$ är minsta övre begränsning av Y_0 .
- $\{2, 3\}$ är ingen möb av Y_0 ty $\{1, 2\} \not\subseteq \{2, 3\}$.
- $Y_1 = \{\emptyset, \{1\}, \{3\}\}$ är ingen kedja ty $\{1\}$ och $\{3\}$ är orelaterade.
- Dock har Y_1 en möb, $\sqcup Y_1 = \{1, 3\}$.

Kedjefullständigt poset (ccpoer)

Ett poset $\langle D, \sqsubseteq \rangle$ är ett *kedjefullständigt poset* (ccpo¹) om $\bigsqcup Y$ existerar för alla kedjor $Y \subseteq D$.

Fullständigt lattice

Ett poset $\langle D, \sqsubseteq \rangle$ är ett *fullständigt lattice* om $\bigsqcup Y$ existerar för alla $Y \subseteq D$.

Alltså:

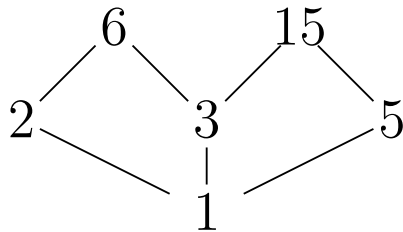
- Varje fullständigt lattice är en ccpo.
- Varje ccpo är ett poset.

¹“chain complete partially ordered set”

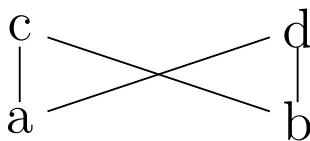
Exempel: Låt $F = \{x \mid x \text{ är en ändlig delmängd av } \mathbb{N}\}$.
 $\langle F, \subseteq \rangle$ är ett poset men ingen ccpo eftersom

$$\bigsqcup \{\emptyset, \{0\}, \{0, 1\}, \{0, 1, 2\}, \dots\} = \mathbb{N} \notin F$$

Exempel: $\langle \{1, 2, 3, 5, 6, 15\}, \mid \rangle^2$ är en ccpo men inget fullständigt lattice ty t ex $\bigsqcup \{6, 15\}$ existerar inte.



Exempel: Nedanstående ccpo är inget fullständigt lattice eftersom t ex både $\bigsqcup \{c, d\}$ och $\bigsqcup \{a, b\}$ saknas.



² $x|y$ är sant om x är en multipel av y .

Några resultat

Uppgift 4.18: $\langle \mathcal{P}(S), \subseteq \rangle$ och $\langle \mathcal{P}(S), \supseteq \rangle$ är fullständiga lattice för alla icke-tomma mängder S .

Uppgift 4.21: $\langle \text{State} \hookrightarrow \text{State}, \sqsubseteq \rangle$ är inget fullständigt lattice.

Lemma 4.25: $\langle \text{State} \hookrightarrow \text{State}, \sqsubseteq \rangle$ är en ccpo.
För varje kedja Y finns en möb:

$$\text{graph}(\bigsqcup Y) = \bigcup \{ \text{graph}(g) \mid g \in Y \}$$

Exempel

$$fac = \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * fac(n - 1)$$

En serie successiva approximationer till fac :

$$\begin{aligned} f_0 &= \lambda n. undef \\ f_1 &= \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * f_0(n - 1) \\ &\vdots \\ f_{i+1} &= \lambda n. \mathbf{if} \ n \equiv 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n * f_i(n - 1) \end{aligned}$$

$$\begin{aligned} graph(f_0) &= \emptyset \\ graph(f_1) &= \{\langle 0, 1 \rangle\} \\ &\vdots \\ graph(f_{i+1}) &= \{\langle 0, 1 \rangle, \dots, \langle i, i! \rangle\} \end{aligned}$$

Sekvensen $\{f_0, \dots, f_\infty\}$ är en kedja med fac som möb.

Monotona funktioner

$\langle D, \sqsubseteq \rangle, \langle D', \sqsubseteq' \rangle$ — ccpoer; $f : D \rightarrow D'$ — en funktion.

Definition: f är monoton omm för alla $d_1, d_2 \in D$

$$d_1 \sqsubseteq d_2 \quad \Rightarrow \quad f d_1 \sqsubseteq f d_2$$

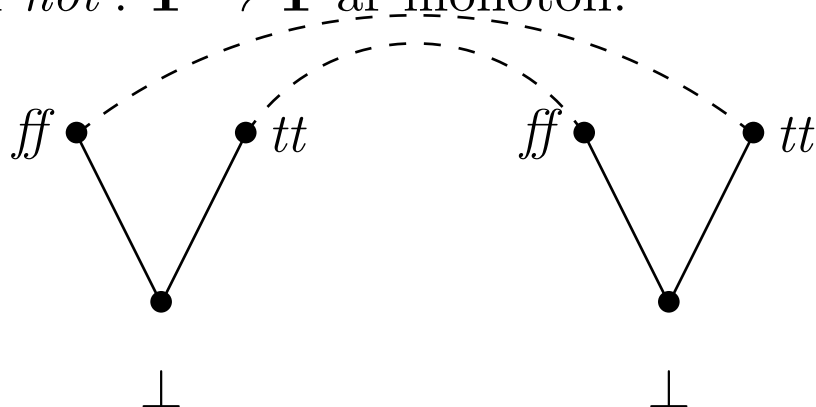
Exempel: Betrakta $\langle \mathcal{P}(\{a, b, c\}), \subseteq \rangle$ och $\langle \mathcal{P}(\{d, e\}), \subseteq \rangle$.

X	$\{a, b, c\}$	$\{a, b\}$	$\{a, c\}$	$\{b, c\}$	$\{a\}$	$\{b\}$	$\{c\}$	$\emptyset$
$f_1(X)$	$\{d, e\}$	$\{d\}$	$\{d, e\}$	$\{d, e\}$	$\{d\}$	$\{d\}$	$\{e\}$	$\emptyset$
$f_2(X)$	$\{d\}$	$\{d\}$	$\{d\}$	$\{e\}$	$\{d\}$	$\{e\}$	$\{e\}$	$\{e\}$

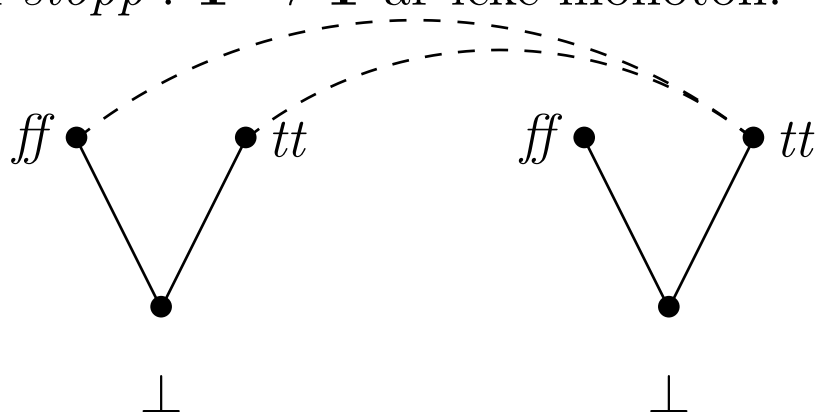
Funktionen f_1 är monoton, men f_2 är inte monoton.

Exempel

Funktionen $not : \mathbf{T} \rightarrow \mathbf{T}$ är monoton:



Funktionen $stopp : \mathbf{T} \rightarrow \mathbf{T}$ är icke monoton:



Lemma 4.30: $\langle D, \sqsubseteq \rangle, \langle D', \sqsubseteq' \rangle$ — ccpoer

$f : D \rightarrow D'$ — en monoton funktion.

Om Y är en kedja i D så gäller

1. $Y' = \{f\ d \mid d \in Y\}$ är en kedja i D'
2. $\sqcup' Y' \sqsubseteq' f(\sqcup Y)$

Bevis: Trivialt för $Y = \emptyset$. Betrakta nu $d_1, d_2 \in Y$:

Y är en kedja $\Rightarrow (d_1 \sqsubseteq d_2)$ eller $(d_2 \sqsubseteq d_1)$.

f är monoton $\Rightarrow (f\ d_1 \sqsubseteq f\ d_2)$ eller $(f\ d_2 \sqsubseteq f\ d_1)$.

Gäller för alla d_1, d_2 ; alltså är Y' också en kedja. □

Funktioner som uppfyller (2) med likhet är speciellt intressanta.

Kontinuerliga funktioner

$\langle D, \sqsubseteq \rangle, \langle D', \sqsubseteq' \rangle$ — ccpoer

Definition: $f : D \rightarrow D'$ sägs vara *kontinuerlig* om den är monoton och uppfyller likheten

$$\sqcup' \{ f \ d \mid d \in Y \} = f(\sqcup Y)$$

för alla icke-tomma kedjor Y .

En kontinuerlig funktion bevarar alltså möben på kedjor, dvs vi får samma svar oavsett om vi applicerar $\sqcup$ före eller efter applikationen av f .

Definition: Om f är kontinuerlig även för en tom kedja, dvs $\perp = f \ \perp$, sägs f vara *strikt*.

Observation: För ändliga ccpoer är alla monotona funktioner även kontinuerliga.

Exempel: Välj $D = \mathcal{P}(\mathbb{N} \cup \{a\})$ och $f : D \rightarrow D$:

$$f X = \begin{cases} X & \text{om } |X| \text{ är ändlig} \\ X \cup \{a\} & \text{om } |X| \text{ är oändlig} \end{cases}$$

f är monoton men inte kontinuerlig, ty:

- **Monoton:** Om $X_1 \subseteq X_2$ så gäller $f X_1 \subseteq f X_2$.
- **Icke kontinuerlig:** Studera mängden

$$Y = \{\{0, 1, \dots, n\} \mid n \geq 0\}$$

$$\begin{array}{ccccccc} \{0\} & \{0, 1\} & \{0, 1, 2\} & \dots & \Rightarrow & & \sqcup Y = \mathbb{N} \\ \downarrow & \downarrow & \downarrow & & & & \downarrow \\ \{0\} & \{0, 1\} & \{0, 1, 2\} & \dots & \Rightarrow & \sqcup Y = \mathbb{N} & \subseteq \mathbb{N} \cup \{a\} \end{array}$$

Inlämningsuppgifter

- Uppgift 4.27
- Uppgift 4.33. Ledtråd: två fall, $sx \neq 0$ och $sx = 0$.

Fixpunktssatsen

$\langle D, \sqsubseteq \rangle$ — ccpo.

$f : D \rightarrow D$ — en kontinuerlig funktion.

Sats 4.37:

$$\text{fix } f = \bigsqcup \{f^n \perp \mid n \geq 0\}$$

definierar en minsta fixpunkt av f , där

$$\begin{aligned} f^0 x &= x \\ f^{n+1} x &= f(f^n x) \end{aligned}$$

Bevis:

- Visa först att $\{f^n \perp \mid n \geq 0\}$ är en (icke-tom) kedja:
 - $f^0 \perp = \perp \sqsubseteq f \perp$
 - Med induktion: $f^n \perp \sqsubseteq f^m \perp, n \leq m$.
- $\text{fix } f$ existerar, ty $\langle D, \sqsubseteq \rangle$ är en ccpo.
- Visa sedan att $f(\text{fix } f) = \text{fix } f$:

$$\begin{aligned}
 f(\text{fix } f) &= f(\bigsqcup \{f^n \perp \mid n \geq 0\}) && (\text{enl. def.}) \\
 &= \bigsqcup f(\{f^n \perp \mid n \geq 0\}) && (f \text{ kont.}) \\
 &= \bigsqcup \{f^n \perp \mid n \geq 1\} \\
 &=^\dagger \bigsqcup (\{f^n \perp \mid n \geq 1\} \cup \{\perp\}) \\
 &= \bigsqcup \{f^n \perp \mid n \geq 0\} && (f^0 \perp = \perp) \\
 &= \text{fix } f
 \end{aligned}$$

$^\dagger \bigsqcup (Y \cup \{\perp\}) = \bigsqcup Y$ för alla kedjor Y .

- Visa att $\text{fix } f$ är den minsta fixpunkten:
 - Antag att d är en godtycklig fixpunkt, $\perp \sqsubseteq d$.
 - f monoton $\Rightarrow f^n \perp \sqsubseteq f^n d$ för $n \geq 0$.
 - Eftersom d är en fixpunkt: $f^n \perp \sqsubseteq d$ för $n \geq 0$.
 - Alltså är d en övre begränsning av kedjan

$$\{f^n \perp \mid n \geq 0\}$$

- Eftersom $\text{fix } f$ definierats som den *minsta* övre begränsning av denna kedja så:

$$\text{fix } f \sqsubseteq d$$

Exempel: Betrakta satsen `while  $\neg(x=0)$  do skip` med dess funktional

$$(F\ g)s = \begin{cases} g\ s & \text{om } s\ \mathbf{x} \neq 0 \\ s & \text{annars} \end{cases}$$

Vad är *fix* F ? Använd fixpunktssatsen.

- $(F^0 \perp)s = \perp\ s = \text{undef}$
- $(F^1 \perp)s = (F \perp)s = \begin{cases} \text{undef} & \text{om } s\ \mathbf{x} \neq 0 \\ s & \text{annars} \end{cases}$
- $(F^2 \perp)s = F(F \perp)s = \begin{cases} \text{undef} & \text{om } s\ \mathbf{x} \neq 0 \\ s & \text{annars} \end{cases}$
- $F^n \perp = F^{n+1} \perp$, för $n > 0$.

$$\begin{aligned} \text{fix } F &= \sqcup \{F^n \perp \mid n \geq 0\} = \sqcup \{F^0 \perp, F^1 \perp\} = F^1 \perp = \\ &= \begin{cases} \text{undef} & \text{om } s\ \mathbf{x} \neq 0 \\ s & \text{annars} \end{cases} \end{aligned}$$

Restriktioner

I fortsättningen kommer vi att begränsa oss till kontinuerliga funktioner på ccpoer eftersom de alltid har en minsta fixpunkt!

Inlämningsuppgifter

- Uppgift 4.39
- Uppgift 4.40

6 Direct style semantics (forts)

Vi återvänder nu till det ursprungliga problemet:

$$\begin{aligned}
 \mathcal{S}_{ds} \llbracket x := a \rrbracket &= \lambda s. s[x \mapsto \mathcal{A} \llbracket a \rrbracket s] \\
 \mathcal{S}_{ds} \llbracket \text{skip} \rrbracket &= \lambda s. s \\
 \mathcal{S}_{ds} \llbracket S_1; S_2 \rrbracket &= \mathcal{S}_{ds} \llbracket S_2 \rrbracket \circ \mathcal{S}_{ds} \llbracket S_1 \rrbracket \\
 \mathcal{S}_{ds} \llbracket \text{if } b \text{ then } S_1 \text{ else } S_2 \rrbracket &= \lambda s. \text{if } \mathcal{B} \llbracket b \rrbracket s \\
 &\quad \text{then } \mathcal{S}_{ds} \llbracket S_1 \rrbracket s \\
 &\quad \text{else } \mathcal{S}_{ds} \llbracket S_2 \rrbracket s \\
 \mathcal{S}_{ds} \llbracket \text{while } b \text{ do } S \rrbracket &= ?
 \end{aligned}$$

Nu har vi de verktyg som behövs för att definiera **while**!

Lösning till problemet

Sätt $g = \mathcal{S}_{ds}[\text{while } b \text{ do } S]$. Vi vill lösa ekvationen:

$$g = \lambda s. \text{if } \mathcal{B}[b]s \text{ then } g (\mathcal{S}_{ds}[S] s) \text{ else } s$$

Detta ger upphov till följande funktional:

$$G = \lambda g. \lambda s. \text{if } \mathcal{B}[b]s \text{ then } g s \text{ else } s$$

med den egenskapen att $G g = g$. Alltså:

$$g = \text{fix } G$$

och om G är kontinuerlig (vilket den är) så är g unik.

Resultatet

$$\begin{aligned}
\mathcal{S}_{ds}[[x := a]] &= \lambda s. s[x \mapsto \mathcal{A}[[a]]s] \\
\mathcal{S}_{ds}[[\text{skip}]] &= \lambda s. s \\
\mathcal{S}_{ds}[[S_1; S_2]] &= \mathcal{S}_{ds}[[S_2]] \circ \mathcal{S}_{ds}[[S_1]] \\
\mathcal{S}_{ds}[[\text{if } b \text{ then } S_1 \text{ else } S_2]] &= \lambda s. \text{if } \mathcal{B}[[b]]s \\
&\quad \text{then } \mathcal{S}_{ds}[[S_1]]s \\
&\quad \text{else } \mathcal{S}_{ds}[[S_2]]s \\
\mathcal{S}_{ds}[[\text{while } b \text{ do } S]] &= \text{fix } F
\end{aligned}$$

där

$$F = \lambda g. \lambda s. \text{if } \mathcal{B}[[b]]s \text{ then } g(\mathcal{S}_{ds}[[S]]s) \text{ else } s$$

Lemma 4.35, 4.43, 4.45:

F är kontinuerlig. (Beviset finns i boken.)

$\mathcal{S}_{ds}[\text{while} \dots]$ har alltså en unik lösning.

Sats 4.47

Ekvationerna för $\mathcal{S}_{ds}$ definierar en total funktion

$$\mathcal{S}_{ds} : \mathbf{Stm} \rightarrow (\mathbf{State} \hookrightarrow \mathbf{State})$$

Bevis: Strukturinduktion över satser.

Exempel: Givet $s_0(x) = 3$, studera betydelsen av

$$y := 1; \text{ while } \underbrace{\neg(x=1)}_b \text{ do } \underbrace{(y := y*x; x := x-1)}_S$$

$$\begin{aligned} \mathcal{S}_{ds} \llbracket y := 1; \text{ while } b \text{ do } S \rrbracket s_0 &= \\ &= \mathcal{S}_{ds} \llbracket \text{while } b \text{ do } S \rrbracket (\mathcal{S}_{ds} \llbracket y:=1 \rrbracket s_0) \\ &= \mathcal{S}_{ds} \llbracket \text{while } b \text{ do } S \rrbracket s_0[y \mapsto 1] \\ &= (fix F) s_0[y \mapsto 1] \end{aligned}$$

där

$$F g s = \begin{cases} g(\mathcal{S}_{ds} \llbracket S \rrbracket s) & \text{om } \mathcal{B} \llbracket b \rrbracket s = tt \\ s & \text{om } \mathcal{B} \llbracket b \rrbracket s = ff \end{cases}$$

Eftersom $S = (y:=y*x; x:=x-1)$ och $B = \neg(x=1)$:

$$F g s = \begin{cases} g(s[y \mapsto (s y) * (s x)][x \mapsto (s x) - 1]) & \text{om } s x \neq 1 \\ s & \text{om } s x = 1 \end{cases}$$

Beräkna nu $\text{fix } F \ s_0[y \mapsto 1]$ genom att växelvis

- Applicera fixpunktsregeln: $\text{fix } F = F (\text{fix } F)$
- Applicera definitionen av F

$$\begin{aligned} \text{fix } F \ s_0[y \mapsto 1] &= F (\text{fix } F) \ s_0[y \mapsto 1] \\ &= (\text{fix } F)(s_0[y \mapsto 3][x \mapsto 2]) \\ &= F (\text{fix } F) \ s_0[y \mapsto 3][x \mapsto 2] \\ &= (\text{fix } F)(s_0[y \mapsto 6][x \mapsto 1]) \\ &= F (\text{fix } F) \ s_0[y \mapsto 6][x \mapsto 1] \\ &= s_0[\underline{y \mapsto 6}][x \mapsto 1] \end{aligned}$$

Semantisk ekvivalens

Två satser S_1 och S_2 är *semantiskt ekvivalenta* om

$$\mathcal{S}_{ds}[\![S_1]\!] = \mathcal{S}_{ds}[\![S_2]\!]$$

Exempel: Visa att $\mathcal{S}_{ds}[\![S; \text{skip}]\!] = \mathcal{S}_{ds}[\![S]\!]$

$$\mathcal{S}_{ds}[\![S; \text{skip}]\!] = \mathcal{S}_{ds}[\![\text{skip}]\!] \circ \mathcal{S}_{ds}[\![S]\!] = id \circ \mathcal{S}_{ds}[\![S]\!] = \mathcal{S}_{ds}[\![S]\!]$$

Operationell och denotationell ekvivalens

Sats 4.55: För varje sats S i While gäller

$$\mathcal{S}_{sos}[\![S]\!] = \mathcal{S}_{ds}[\![S]\!]$$

Bevis: Se boken.

Inlämningsuppgifter

- Uppgift 4.49
(Antag ett initial tillstånd där $x=17$ och $y=5$)
- Uppgift 4.53
- Lab-uppgift 3

Block och deklarationer

Vi utökar återigen språket While med deklarationer av variabler och procedurer.

Environment och Store

Samma domäner som i den operationella semantiken:

$$\begin{aligned} env_V \in \mathbf{Env}_V &= \mathbf{Var} \rightarrow \mathbf{Loc} \\ sto \in \mathbf{Store} &= \mathbf{Loc} \cup \{\text{next}\} \rightarrow \mathbf{Z} \\ \mathbf{Loc} &= \mathbf{Z} \end{aligned}$$

OBS: $sto \circ env_V$ fungerar som gamla **State**.

(Boken kallar denna operation för *lookup*.)

Kommentarer:

- sto representerar det fysiska minnet.
- $env_V(x)$ är x :s minnesadress.
- $sto(next)$ är nästa lediga minnescell.

Exempel: Effekt av $x := 17; y := 42$:

env_V		sto	
x	1	next	3
y	2	1	17
		2	42

$$(sto \circ env_V) \mathbf{x} = sto(env_V \mathbf{x}) = sto \ 1 = 17$$

Betydelsen $\mathcal{S}'_{ds} \llbracket S \rrbracket$ av en sats S i en omgivning env_V är alltså en funktion $\mathbf{Store} \hookrightarrow \mathbf{Store}$.

$$\mathcal{S}'_{ds} : \mathbf{Stm} \rightarrow \mathbf{Env}_V \rightarrow (\mathbf{Store} \hookrightarrow \mathbf{Store})$$

$$\begin{aligned} \mathcal{S}'_{ds} \llbracket x := a \rrbracket env_V sto = \\ sto[env_V(x) \mapsto \mathcal{A} \llbracket a \rrbracket (sto \circ env_V)] \end{aligned}$$

$$\mathcal{S}'_{ds} \llbracket \mathbf{skip} \rrbracket env_V sto = sto$$

$$\mathcal{S}'_{ds} \llbracket S_1; S_2 \rrbracket env_V = (\mathcal{S}'_{ds} \llbracket S_2 \rrbracket env_V sto) \circ (\mathcal{S}'_{ds} \llbracket S_1 \rrbracket env_V sto)$$

$$\begin{aligned} \mathcal{S}'_{ds} \llbracket \mathbf{if } b \mathbf{ then } S_1 \mathbf{ else } S_2 \rrbracket env_V sto = \\ \mathbf{if } \mathcal{B} \llbracket b \rrbracket (sto \circ env_V) \mathbf{ then } \mathcal{S}'_{ds} \llbracket S_1 \rrbracket env_V sto \\ \mathbf{ else } \mathcal{S}'_{ds} \llbracket S_2 \rrbracket env_V sto \end{aligned}$$

$$\begin{aligned} \mathcal{S}'_{ds} \llbracket \mathbf{while } b \mathbf{ do } S \rrbracket env_V = \mathbf{fix } (\lambda g. \lambda sto. \\ \mathbf{if } \mathcal{B} \llbracket b \rrbracket (sto \circ env_V) \mathbf{ then } g(\mathcal{S}'_{ds} \llbracket S \rrbracket env_V sto) \\ \mathbf{ else } sto) \end{aligned}$$

Variabeldeklarationer

Utöka grammatiken:

$$\begin{array}{lll} S & \in \mathbf{Stm} & \rightarrow \dots \mid \text{begin } D_V \ S \ \text{end} \\ D_V & \in \mathbf{Dec}_V & \rightarrow \text{var } x := a; \ D_V \mid \varepsilon \end{array}$$

Exempel:

```
begin var y := 0; var x := 1;
  begin var x := 7; x := x+1 end;
  y := x
end
```

Introducera en ny funktion:

$$\mathcal{D}_{ds}^V : \mathbf{Dec}_V \rightarrow (\mathbf{Env}_V \times \mathbf{Store}) \rightarrow (\mathbf{Env}_V \times \mathbf{Store})$$

$$\mathcal{D}_{ds}^V[\varepsilon](env_V, sto) = (env_V, sto)$$

$$\begin{aligned} \mathcal{D}_{ds}^V[\mathbf{var } x := a; D_V](env_V, sto) = \\ \mathcal{D}_{ds}^V[D_V](env_V[x \mapsto l], sto[l \mapsto v][\text{next} \mapsto l + 1]) \end{aligned}$$

där $l = sto \text{ next}$, och $v = \mathcal{A}[a](sto \circ env_V)$.

Kommentar: $\mathcal{D}_{ds}^V[D_V]$ tar ett par (env_V, sto) och allokerar/initierar minne för variablerna i D_V .

$$\mathcal{S}'_{ds}[\mathbf{begin } D_V \ S \ \mathbf{end}](env_V, sto) = \mathcal{S}'_{ds}[S](env'_V, sto')$$

$$\text{där } (env'_V, sto') = \mathcal{D}_{ds}^V[D_V](env_V, sto)$$

Kommentar: Kör S i det allokerade/initierade minnet.

Procedurdeklarationer

Utöka grammatiken:

$$\begin{aligned} S &\in \mathbf{Stm} &\rightarrow & \dots \mid \mathbf{begin } D_V D_P S \mathbf{ end } \mid \mathbf{call } p \\ D_P &\in \mathbf{Dec}_P &\rightarrow & \mathbf{proc } p \mathbf{ is } S; D_P \mid \varepsilon \end{aligned}$$

Ett procedurnamn $p \in \mathbf{Pname}$ associeras med en funktion som ändrar minnet:

$$\mathbf{Env}_P = \mathbf{Pname} \rightarrow (\mathbf{Store} \rightarrow \mathbf{Store})$$

En proceduromgivning env_P uppdateras med $\mathcal{D}_{ds}^P$:

$$\mathcal{D}_{ds}^P : \mathbf{Dec}_P \rightarrow \mathbf{Env}_V \rightarrow \mathbf{Env}_P \rightarrow \mathbf{Env}_P$$

Anledningen till att $\mathbf{Env}_V$ är med beror på att proceduren måste komma ihåg variabelomgivningen.

Exempel:

```
begin var x1 := 7; proc p is x1 := 0;
    begin var x2 := 5; call p end
end
```

Statisk binding (det enda rätta): *p* refererar till *x*¹.

Icke-rekursiva procedurer

$$\mathcal{D}_{ds}^P[\varepsilon] \text{ env}_V \text{ env}_P = \text{env}_P$$

$$\begin{aligned} \mathcal{D}_{ds}^P[\text{proc } p \text{ is } S; D_P] \text{ env}_V \text{ env}_P = \\ \mathcal{D}_{ds}^P[D_P] \text{ env}_V (\text{env}_P[p \mapsto \mathcal{S}_{ds}[S] \text{ env}_V \text{ env}_P]) \end{aligned}$$

Kommentar: Uppdatera *env*_{*P*} genom att knyta varje procedurnamn till betydelsen av dess kropp.

Betydelsen av satser

$$\mathcal{S}_{ds} : \mathbf{Stm} \rightarrow \mathbf{Env}_V \rightarrow \mathbf{Env}_P \rightarrow (\mathbf{Store} \hookrightarrow \mathbf{Store})$$

De två nya fallen är anrop och deklARATIONER av procedurer:

$$\mathcal{S}_{ds}[\mathbf{call } p] \ env_V \ env_P = env_P \ p$$

$$\begin{aligned} \mathcal{S}_{ds}[\mathbf{begin } D_V \ D_P \ S \ \mathbf{end}] \ env_V \ env_P \ sto = \\ \mathcal{S}_{ds}[S] \ env'_V \ env'_P \ sto' \end{aligned}$$

där

$$\begin{aligned} (env'_V, sto') &= \mathcal{D}_{ds}^V[D_V](env_V, sto) \\ env'_P &= \mathcal{D}_{ds}^P[D_P] \ env'_V \ env_P \end{aligned}$$

Resten är enkelt (som förut): skicka bara runt env_P :

$$\mathcal{S}_{ds} \llbracket x := a \rrbracket env_V env_P sto = \\ sto[env_V x \mapsto \mathcal{A} \llbracket a \rrbracket (sto \circ env_V)]$$

$$\mathcal{S}_{ds} \llbracket \text{skip} \rrbracket env_V env_P sto = sto$$

$$\mathcal{S}_{ds} \llbracket S_1; S_2 \rrbracket env_V env_P = \\ (\mathcal{S}_{ds} \llbracket S_2 \rrbracket env_V env_P) \circ (\mathcal{S}_{ds} \llbracket S_1 \rrbracket env_V env_P)$$

$$\mathcal{S}_{ds} \llbracket \text{if } b \text{ then } S_1 \text{ else } S_2 \rrbracket env_V env_P sto = \\ \text{if } \mathcal{B} \llbracket b \rrbracket (sto \circ env_V) \text{ then } \mathcal{S}_{ds} \llbracket S_1 \rrbracket env_V env_P \\ \text{else } \mathcal{S}_{ds} \llbracket S_2 \rrbracket env_V env_P$$

$$\mathcal{S}_{ds} \llbracket \text{while } b \text{ do } S \rrbracket env_V env_P = \text{fix } (\lambda g. \lambda s. \\ \text{if } \mathcal{B} \llbracket b \rrbracket (sto \circ env_V) \text{ then } g(\mathcal{S}_{ds} \llbracket S \rrbracket env_V env_P s) \\ \text{else } s)$$

Exempel 4.66: Applicera $\mathcal{S}_{ds}$ på satsen

$$\begin{array}{c} \text{begin } \overbrace{\text{var } x := 7;}^{D_V} \overbrace{\text{proc } p \text{ is } x := 0;}^{D_P} \\ \quad \underbrace{\text{begin var } x := 5; \text{ call } p \text{ end}}_S \\ \text{end} \end{array}$$

Antag initialt $sto \text{ next} = 12$.

$\mathcal{S}_{ds}[\text{begin } D_V \ D_P \ S \ \text{end}] \ env_V \ env_P \ sto =$
 $\mathcal{S}_{ds}[S] \ env'_V \ env'_P \ sto'$, där

$$\begin{aligned} (env'_V, sto') &= \mathcal{D}_{ds}^V[\text{var } x := 7](env_V, sto) \\ &= (env_V[x \mapsto 12], sto[12 \mapsto 7][\text{next} \mapsto 13]) \\ env'_P &= \mathcal{D}_{ds}^P[\text{proc } p \text{ is } x:=0] env'_V env_P \\ &= env_P[p \mapsto \mathcal{S}_{ds}[x:=0] env'_V env_P] \\ &= env_P[p \mapsto (\lambda sto. sto[12 \mapsto 0])] \end{aligned}$$

$\mathcal{S}_{ds}[\text{call } p] \text{ env}_V'' \text{ env}_P'' \text{ sto}'', \text{ d}{\ddot{a}}\text{r}$

$$\begin{aligned} (\text{env}_V'', \text{sto}'') &= \mathcal{D}_{ds}^V[\text{var } x := 5](\text{env}_V', \text{sto}') \\ &= (\text{env}_V'[\mathbf{x} \mapsto 13], \text{sto}'[13 \mapsto 5][\text{next} \mapsto 14]) \\ \text{env}_P'' &= \text{env}_P' \end{aligned}$$

$$\begin{aligned} (\text{env}_P'' \text{ p}) \text{ sto}'' &= (\lambda \text{sto}. \text{sto}[12 \mapsto 0]) \text{ sto}'' \\ &= \text{sto}''[12 \mapsto 0] \\ &= \text{sto}'[13 \mapsto 5][12 \mapsto 0] \\ &= \text{sto}[12 \mapsto 7][13 \mapsto 5][12 \mapsto 0] \\ &= \text{sto}[13 \mapsto 5][12 \mapsto 0] \end{aligned}$$

Rekursiva procedurer

Hur behandlade vi en icke-rekursiv procedurdeklaration?

$$\begin{aligned} \mathcal{D}_{ds}^P[\text{proc } p \text{ is } S; D_P] \text{ env}_V \text{ env}_P = \\ \mathcal{D}_{ds}^P[D_P] \text{ env}_V \left(\text{env}_P[p \mapsto \underbrace{\mathcal{S}_{ds}[S] \text{ env}_V \text{ env}_P}_{g : \text{Store} \hookrightarrow \text{Store}}] \right) \end{aligned}$$

Om p är rekursiv vill vi egentligen knyta p till en annan funktion h som uppfyller följande ekvation:

$$h = \mathcal{S}_{ds}[S] \text{ env}_V (\text{env}_P[p \mapsto h])$$

Lösning: $h = \text{fix}(\lambda f. \mathcal{S}_{ds}[S] \text{ env}_V (\text{env}_P[p \mapsto f]))$

Bevis: Lita på mig!

Inlämningsuppgifter

- Uppgift 4.67

7 Fortsättningar (continuations)

En användbar teknik för att modellera hopp, korutiner, processer, backtracking och andra kontrollstrukturer.

Bakgrund: Ett svansrekursivt anrop är ett rekursivt anrop som inte pushar sin returnadress. Kan användas om inget mer ska utföras efter anropet.

Den anropade funktionen återvänder direkt till anroparens anropare. Ett svansrekursivt anrop kan implementeras med ett hopp:

CALL foo		JUMP foo	
RET			
foo:	⋮	foo:	⋮
RET		RET	

Exempel: Följande definition är inte svansrekursiv ty efter det rekursiva anropet måste man tillbaks och göra *

```
fac 0 = 1
fac n = n*fac(n-1)
```

Exempel: Den här koden är däremot svansrekursiv:

```
fac n = fac' n 1
fac' 0 acc = acc
fac' n acc = fac' (n-1) (n*acc)
```

Fortsättningsbaserad kodning

- Alla anrop ser svansrekursiva ut men kommer inte tillbaks.
- Skicka alltid med en extra funktion (en continuation) C .
- När den anropade funktionen har räknat ut sitt värde, applicera C på värdet.

Funktionen C representerar vad som ska göras härnäst.

Exempel:

```
fac 0 c = c 1
fac n c = fac (n-1) (\f -> c (n*f))
```

Körexempel:

$$\begin{aligned}\text{fac } 3 \text{ id} &= \text{fac } 2 \underbrace{(\lambda f. \text{id } (3 * f))}_{C_1} \\ &= \text{fac } 1 \underbrace{(\lambda f. C_1(2 * f))}_{C_2} \\ &= \text{fac } 0 \underbrace{(\lambda f. C_2(1 * f))}_{C_3} \\ &= C_3 \ 1 \\ &= C_2 \ (1 * 1) \\ &= C_1 \ (2 * 1 * 1) \\ &= \text{id } (3 * 2 * 1 * 1) \\ &= 6\end{aligned}$$

While med exceptionhantering

Vi inför nu en goto-liknande möjlighet i While:

$$S \rightarrow \dots \mid \text{begin } S_1 \text{ handle } e : S_2 \text{ end} \mid \text{raise } e$$

där $e \in \mathbf{Exception}$ är en identifierare.

Exempel:

```
begin
  while true do
    if x <= 0 then raise exit else x := x-1
  handle exit:
    y := 7
  end
```

När x blir 0 "hoppa" programmet till satsen $y := 7$.

Motivation

Betrakta en sats

$$(\text{if } b \text{ then } S_1 \text{ else } S_2); S_3$$

- Normalt kommer S_1/S_2 att fortsätta med S_3 .
- Om S_1/S_2 gör **raise** e hoppar vi förbi S_3 .

Vi måste skriva om semantiken så att exekveringstråden ("det som ska göras härnäst") blir explicit.

Definition: En *fortsättning* c är ett element i

$$\mathbf{Cont} = \mathbf{State} \hookrightarrow \mathbf{State}$$

och representerar vad som händer om man exekverar *resten* av programmet.

Fortsättningsbaserad semantik

En fortsättning byggs normalt upp bakifrån från *id*.

Betrakta en sekvens av satser $\dots; S_{i-1}; S_i; S_{i+1}; \dots$ och en fortsättning c_{i+1} som beskriver resultatet av att exekvera $S_{i+1}; \dots$

$$\dots; S_{i-1}; S_i; \underbrace{S_{i+1}; \dots}_{c_{i+1}}$$

Dvs, om s_{i+1} är tillståndet efter exekvering av S_i så är $c_{i+1} s_{i+1}$ det slutgiltiga tillståndet.

Vi vill bestämma c_i , resultatet av att köra $S_i; S_{i+1}, \dots$

$$\dots; S_{i-1}; \underbrace{S_i; S_{i+1}; \dots}_{c_i}$$

Den funktion som utför detta är

$$\mathcal{S}'_{cs} : \text{Stm} \rightarrow (\text{Cont} \rightarrow \text{Cont})$$

Antag att vi befinner oss i ett tillstånd s .

Tilldelning

$$\dots S_{i-1}; \quad \underbrace{x := a; S_{i+1}; \dots}_c$$

$$\mathcal{S}'_{cs} \llbracket x := a \rrbracket c \ s = c(s[x \mapsto \mathcal{A} \llbracket a \rrbracket s])$$

Skip

$$\dots S_{i-1}; \quad \underbrace{\text{skip}; S_{i+1}; \dots}_c$$

$$\mathcal{S}'_{cs} \llbracket \text{skip} \rrbracket c \ s = c \ s$$

Sekvens

$$\mathcal{S}'_{cs}[[S_1; S_2]] = \mathcal{S}'_{cs}[[S_1]] \circ \mathcal{S}'_{cs}[[S_2]]$$

Kompositionen är ”baklänges” eftersom vi drar fortsättningen bakåt först genom S_2 , sedan S_1 .

Selektion

$$\begin{aligned} \mathcal{S}'_{cs}[\text{if } b \text{ then } S_1 \text{ else } S_2] \ c \ s = \\ \text{if } \mathcal{B}[[b]]s \text{ then } \mathcal{S}'_{cs}[[S_1]] \ c \ s \text{ else } \mathcal{S}'_{cs}[[S_2]] \ c \ s \end{aligned}$$

Iteration

$$\begin{aligned} \mathcal{S}'_{cs}[\text{while } b \text{ do } S] = \\ \text{fix}(\lambda g. \lambda c. \lambda s. \text{if } \mathcal{B}[[b]]s \text{ then } g \ c \ (\mathcal{S}'_{cs}[[S]] \ c \ s) \text{ else } c \ s) \end{aligned}$$

$$\begin{aligned}
\text{Exempel: } \mathcal{S}'_{cs} \llbracket z := x; x := y; y := z \rrbracket id &= \\
&= (\mathcal{S}'_{cs} \llbracket z := x \rrbracket \circ \mathcal{S}'_{cs} \llbracket x := y \rrbracket \circ \mathcal{S}'_{cs} \llbracket y := z \rrbracket) id \\
&= (\mathcal{S}'_{cs} \llbracket z := x \rrbracket \circ \mathcal{S}'_{cs} \llbracket x := y \rrbracket) \underbrace{(\lambda s. id \ s[y \mapsto (s \ z)])}_{c_1} \\
&= \mathcal{S}'_{cs} \llbracket z := x \rrbracket \underbrace{(\lambda s. c_1 \ s[x \mapsto (s \ y)])}_{c_2} \\
&= \lambda s. c_2 \ s[z \mapsto (s \ x)] \\
&= \lambda s. c_1 \ s[z \mapsto (s \ x)][x \mapsto (s \ y)] \\
&= \lambda s. id \ s[z \mapsto (s \ x)][x \mapsto (s \ y)][y \mapsto (s \ z)] \\
&= \lambda s. s[z \mapsto (s \ x)][x \mapsto (s \ y)][y \mapsto (s \ z)]
\end{aligned}$$

Semantiska funktioner för exceptions

Introducera först en exception-omgivning:

$$env_E \in \mathbf{Env}_E = \mathbf{Exception} \rightarrow \mathbf{Cont}$$

så att $env_E\ e$ representerar effekten av att köra från e :s handler.

Denna omgivning skickas alltid med så semantiken beskrivs med

$$\mathcal{S}_{cs} : \mathbf{Stm} \rightarrow \mathbf{Env}_E \rightarrow (\mathbf{Cont} \rightarrow \mathbf{Cont})$$

Tilldelning, osv, är som förut – skicka bara med env_E :

$$\mathcal{S}_{cs}[[x := a]] \ env_E \ c \ s = c \ (s[x \mapsto \mathcal{A}[[a]]s])$$

$$\mathcal{S}_{cs}[[\text{skip}]] \ env_E \ c \ s = c \ s$$

$$\mathcal{S}_{cs}[[S_1; S_2]] \ env_E = (\mathcal{S}_{cs}[[S_1]] \ env_E) \circ (\mathcal{S}_{cs}[[S_2]] \ env_E)$$

$$\begin{aligned} \mathcal{S}_{cs}[[\text{if } b \text{ then } S_1 \text{ else } S_2]] \ env_E \ c \ s = \\ \text{if } \mathcal{B}[[b]]s \text{ then } \mathcal{S}_{cs}[[S_1]] \ env_E \ c \ s \text{ else } \mathcal{S}_{cs}[[S_2]] \ env_E \ c \ s \end{aligned}$$

$$\begin{aligned} \mathcal{S}_{cs}[[\text{while } b \text{ do } S]] \ env_E = \\ \text{fix}(\lambda gcs. \text{if } \mathcal{B}[[b]]s \text{ then } g \ c \ (\mathcal{S}_{cs}[[S]] \ env_E \ c \ s) \text{ else } c \ s) \end{aligned}$$

Nya fall

$$\mathcal{S}_{cs}[\text{begin } S_1 \text{ handle } e : S_2 \text{ end}] \text{ env}_E c s = \\ \mathcal{S}_{cs}[S_1](\text{env}_E[e \mapsto \mathcal{S}_{cs}[S_2] \text{ env}_E c]) c s$$

Här körs S_1 i en exception-omgivning där e knutits till S_2 :s fortsättning.

$$\mathcal{S}_{cs}[\text{raise } e] \text{ env}_E c s = \text{env}_E e s$$

Observera hur **raise** e struntar i aktuell fortsättning (c) och hoppar vidare till e :s fortsättning.

Inlämningsuppgifter

- Räkna ut $\mathcal{S}_{cs}[[S]] \text{ env}_E \text{ id } s$ där S är följande sats:

```
begin
  if x < 0 then skip else raise exit
  handle exit: x := 0
end
```

och s uppfyller $s \models x > 0$.

- Lab-uppgift 4

8 Statisk Analys

Samla information om programmet vid kompileringen.

Användningsområden:

- **Optimeringar:**
 - Constant propagation
 - Reduction in strength
 - Eliminering av död kod
- **Diagnostik:**
 - Hitta typfel
 - Funktionell analys

Problem: Exakt analys är dyrt, ofta omöjlig!

Non-standard semantics

- Baserat på (vanligtvis) denotationssemantik.
- Arbetar med *abstrakta* värden istf konkreta.

Exempel: Vad har -3×5 för tecken?

Behöver inte räkna ut svaret (-15), räcker med regeln

”Minus gånger plus blir minus”

Varje tal n har ett *abstrakt värde*:

$$abs(n) = \begin{cases} - & \text{om } n < 0 \\ \mathbf{0} & \text{om } n = 0 \\ + & \text{om } n > 0 \end{cases}$$

Abstrakta värden representerar konkreta mängder:

$$\begin{aligned} conc(-) &= \{\dots, -3, -2, -1\} \\ conc(\mathbf{0}) &= \{0\} \\ conc(+) &= \{1, 2, 3, \dots\} \end{aligned}$$

Konkreta operationer, t ex $\times$, har en motsvarande abstrakt operation $\hat{\times}$

$\hat{\times}$	$-$	$\mathbf{0}$	$+$
$-$	$+$	$\mathbf{0}$	$-$
$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$
$+$	$-$	$\mathbf{0}$	$+$

Tecknet på -3×5 blir alltså

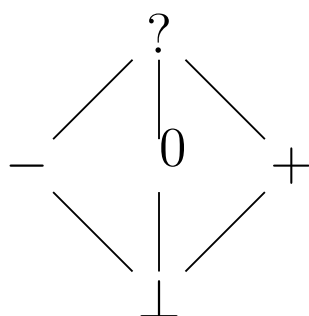
$$abs(-3) \hat{\times} abs(5) = - \hat{\times} + = -$$

Vad blir tecknet på $-3 + 5$?

$\hat{+}$	$-$	$\mathbf{0}$	$+$
$-$	$-$	$-$	$?$
$\mathbf{0}$	$-$	$\mathbf{0}$	$+$
$+$	$?$	$+$	$+$

”?” representerar ”vilket heltal som helst”.

Abstrakta värden är alltså partialordnade:



så att t ex $- \sqsubseteq ?$.

Tecknet på $-3 + 5$ blir alltså $abs(-3) \hat{+} abs(5) = ?$

Rekursion

Betrakta den konkreta funktionen

$$fac(n) = \mathbf{if} \ n = 0 \ \mathbf{then} \ 1 \ \mathbf{else} \ n \times fac(n - 1)$$

Abstrakt version:

$$\hat{fac}(a) = \mathbf{if} \ a \hat{=} \mathbf{0} \ \mathbf{then} \ + \ \mathbf{else} \ a \hat{\times} fac(a \hat{-} +)$$

Studera återigen en sekvens av approximationer till fac :

$$\hat{fac}_0(a) = \perp$$

$$\hat{fac}_1(a) = \mathbf{if} \ a \hat{=} \mathbf{0} \ \mathbf{then} \ + \ \mathbf{else} \ a \hat{\times} fac_0(a \hat{-} +)$$

$$\vdots$$

$$\hat{fac}_{i+1}(a) = \mathbf{if} \ a \hat{=} \mathbf{0} \ \mathbf{then} \ + \ \mathbf{else} \ a \hat{\times} fac_i(a \hat{-} +)$$

Tabulera $\hat{fac}_i(a)$ för $a \in \{\perp, -, \mathbf{0}, +, ?\}$.

	$\perp$	$-$	$\mathbf{0}$	$+$	$?$
$\hat{fac}_0$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$\hat{fac}_1$	$\perp$	$\perp$	$+$	$\perp$	$+$
$\hat{fac}_2$	$\perp$	$\perp$	$+$	$+$	$?$
$\hat{fac}_3$	$\perp$	$\perp$	$+$	$?$	$?$
$\hat{fac}_4$	$\perp$	$\perp$	$+$	$?$	$?$

$\leftarrow$ fixpunkt

Kommentarer:

- $\hat{fac}_1(?) = + \sqcup \hat{fac}_0(?) = +$
- $\hat{fac}_2(+) = + \hat{\times} \hat{fac}_1(?) = +$
- $\hat{fac}_2(?) = + \sqcup (? \hat{\times} \hat{fac}_1(?)) = ?$

Sats: Ändliga abstrakta domäner $\Rightarrow$ terminering.

- **Konkret program:** Programmet som ska analyseras.
- **Abstrakt program:** Ett program där operationerna ersatts med sin abstrakta motsvarighet och där variabler innehåller abstrakta värden.
- **Abstrakt exekvering:** Att köra det abstrakta programmet. En ändlig abstrakt domän garanterar terminering.
- **Utdata:** Något av
 1. "Ja, programmet har egenskapen si-och-så"
 2. "Det är möjligt att programmet har den egenskapen, men det kan jag inte avgöra"

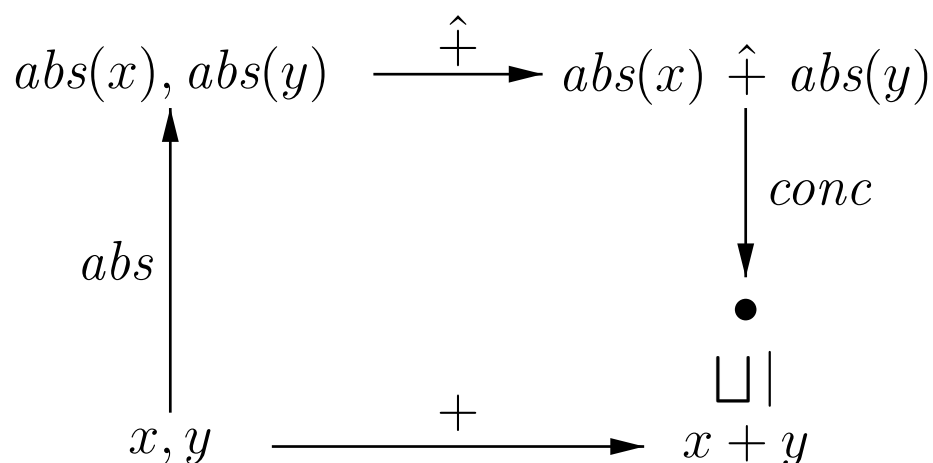
Svar (2) är priset vi får betala för att vi insisterar på att alltid terminera.

En fullständig analys är i regel oavgörbart.

Sundhet

För varje abstrakt operation (t ex $\hat{+}$) måste man visa

$$x + y \sqsubseteq \text{conc}(\text{abs}(x) \hat{+} \text{abs}(y))$$



Funktionella beroenden (kap 5)

Givet följande

- En sats S med en mängd variabler V .
- En delmängd $V_i \subseteq V$, taggade som *input*.
- En delmängd $V_o \subseteq V$, taggade som *output*.

Vi vill avgöra om V_i (och V_i endast) styr kontrollflödet i programmet och bestämmer slutgiltiga värden på V_o .

I så fall: V_o sägs vara *funktionellt beroende* av V_i .

Om V_o inte är FB av V_i kan det bero på två saker:

1. $\exists v_o \in V_o$ som beror (ev. transitivt) på ngt $v \notin V_i$.
2. Kontrollflödet beror (ev. transitivt) på ngt $v \notin V_i$.

Exempel:

	Input	Sats	Output	FB?
1	x	y := x*x+25	y	Ja
2		y := x*x+25	y	Nej
3	x	if (x < 0) ...		Ja
4		if (x < 0) ...		Nej

I fall (2) och (4) bör kompilatorn utfärda en varning:

“Warning, use of an un-initialized variable **x**”

Exempel: Betrakta följande sats S :

$y := 1; \text{ while } \neg(x=1) \text{ do } (y := y*x; x := x-1)$

Input	Sats	Output	FB?
x	S	y	Ja

Här ser man att kontrollflödet och slutgiltiga värdet av y endast beror av x .

Exempel: Betrakta följande sats S' :

$\text{while } \neg(x=1) \text{ do } (y := y*x; x := x-1)$

Input	Sats	Output	FB?
x	S'	y	Nej

Här beror slutgiltiga värdet av y inte bara på x utan även på y 's initiala värde.

En analysmetod för funktionella beroenden

- Avgör om en sats S definitivt har ett funktionellt beroende mellan sina *input*- och *output*-variabler.
- Abstrakt domän: en variabel v innehåller

OK— om v :s värde beror endat på *input*-variabler.

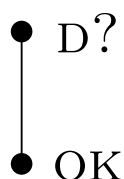
D?— om v :s värde kan bero på en icke-*input*-variabel.

- Initialt:
 - Alla *input*-variabler har värdet OK
 - Alla icke-*input*-variabler har värdet D?
 - *on-track* är OK.
- När analysen är klar, svara
 - ”Ja”, om alla *output*-variabler innehåller OK.
 - ”Nej?”, i annat fall. (Konservativt svar)

Abstrakt domän

$$p \in \mathbf{P} = \{\text{OK}, \text{D?}\}$$

Elementet OK innehåller mer information än D?.



$$\text{OK} \sqsubseteq \text{D?} \quad \text{OK} \sqsubseteq \text{OK} \quad \text{D?} \sqsubseteq \text{D?}$$

OBS: D?, som inte innehåller någon information alls, är toppelementet — tvärt emot domänteorin!

Definition: $p_1 \sqcup p_2 = \sqcup\{p_1, p_2\}$.

$\sqcup$	OK	D?
OK	OK	D?
D?	D?	D?

Abstrakta tillstånd

I operationell och denotationssemantik studerade vi tillstånd

$$s \in \mathbf{State} : \mathbf{Var} \rightarrow \mathbf{Z}$$

I analysen studerar vi abstrakta tillstånd:

$$ps \in \mathbf{PState} : \mathbf{Var} \cup \{on-track\} \rightarrow \mathbf{P}$$

- $on-track = \text{OK} \Rightarrow$ kontrollflödet endast beroende av *input*-variabler.
- $on-track = \text{D?} \Rightarrow$ kontrollflödet *kan vara* beroende av icke-*input*-variabler.

Definitioner för abstrakta tillstånd

Initialt tillstånd: För alla variabler och *on-track*:

$$\text{INIT}(x) = \text{OK}$$

Borttappat tillstånd: För alla variabler och *on-track*:

$$\text{LOST}(x) = \text{D?}$$

Korrekt tillstånd: *ps* är *korrekt* (eng. proper) om

$$ps(\textit{on-track}) = \text{OK}$$

I annat fall sägs *ps* vara *inkorrekt* (eng. improper).

Hur analysen är tänkt att fungera

En konstant kan inte bero på en *input*-variabel, så:

	Tillstånd Före	Sats	Tillstånd Efter
Konkret:	s	$x := 1$	$s[x \mapsto 1]$
Abstrakt:	ps	$x := 1$	$ps[x \mapsto \text{OK}]$

Betrakta nu satsen

$$S' : \text{if } x=1 \text{ then } x:=1 \text{ else } x:=2$$

Om $ps \ x = D?$ kan vi inte bestämma kontrollflödet och följaktligen blir ps inkorrekt.

	Tillstånd Före	Sats	Tillstånd Efter
Abstrakt:	ps	S'	LOST

Betrakta nu satsen

$$S : \text{if } x=1 \text{ then } x:=1 \text{ else } x:=1$$

Eftersom S är semantiskt ekvivalent med $x:=1$ borde man kunna fortsätta med $ps[x \mapsto \text{OK}]$ men pga kompositionsprincipen klarar vi inte detta:

	Tillstånd Före	Sats	Tillstånd Efter
Abstrakt:	ps	S	LOST

Analys av aritmetiska uttryck

$$\mathcal{PA} : \mathbf{Aexp} \rightarrow (\mathbf{Pstate} \rightarrow \mathbf{P})$$

$$\mathcal{PA}[[n]] \ ps = \begin{cases} \text{OK} & \text{om } ps(\text{on-track}) = \text{OK} \\ \text{D?} & \text{annars} \end{cases}$$

$$\mathcal{PA}[[x]] \ ps = \begin{cases} ps(x) & \text{om } ps(\text{on-track}) = \text{OK} \\ \text{D?} & \text{annars} \end{cases}$$

$$\mathcal{PA}[[a_1 + a_2]] \ ps = (\mathcal{PA}[[a_1]] \ ps) \sqcup_P (\mathcal{PA}[[a_2]] \ ps)$$

$$\mathcal{PA}[[a_1 * a_2]] \ ps = (\mathcal{PA}[[a_1]] \ ps) \sqcup_P (\mathcal{PA}[[a_2]] \ ps)$$

$$\mathcal{PA}[[a_1 - a_2]] \ ps = (\mathcal{PA}[[a_1]] \ ps) \sqcup_P (\mathcal{PA}[[a_2]] \ ps)$$

Kommentar: Om ps är inkorrekt, dvs om kontrollen tappats bort, blir resultatet alltid D?.

Analys av booleska uttryck

$$\mathcal{PB} : \mathbf{Bexp} \rightarrow (\mathbf{Pstate} \rightarrow \mathbf{P})$$

$$\mathcal{PB}[\mathbf{true}] \ ps = \begin{cases} \text{OK} & \text{om } ps(\text{on-track}) = \text{OK} \\ \text{D?} & \text{annars} \end{cases}$$

$$\mathcal{PB}[\mathbf{false}] \ ps = \begin{cases} \text{OK} & \text{om } ps(\text{on-track}) = \text{OK} \\ \text{D?} & \text{annars} \end{cases}$$

$$\mathcal{PB}[a_1 = a_2] \ ps = (\mathcal{PA}[a_1] \ ps) \sqcup_P (\mathcal{PA}[a_2] \ ps)$$

$$\mathcal{PB}[a_1 \leq a_2] \ ps = (\mathcal{PA}[a_1] \ ps) \sqcup_P (\mathcal{PA}[a_2] \ ps)$$

$$\mathcal{PB}[\neg b] \ ps = \mathcal{PB}[b] \ ps$$

$$\mathcal{PB}[b_1 \wedge b_2] \ ps = (\mathcal{PB}[b_1] \ ps) \sqcup_P (\mathcal{PB}[b_2] \ ps)$$

Exempel: $\mathcal{PB}[\mathbf{x}=\mathbf{x}] = \text{D?}$ om $ps(\mathbf{x}) = \text{D?}$ trots att man ”vet” att resultatet alltid blir *tt* oavsett $\mathbf{x}$:s värde.

Analys av satser

$$\mathcal{PS} : \text{Stm} \rightarrow (\text{PState} \rightarrow \text{PState})$$

$$\mathcal{PS}[[x := a]] = \lambda ps.ps[x \mapsto \mathcal{PA}[[a]] ps]$$

$$\mathcal{PS}[[\text{skip}]] = \lambda ps.ps$$

$$\mathcal{PS}[[S_1; S_2]] = \mathcal{PS}[[S_2]] \circ \mathcal{PS}[[S_1]]$$

$$\begin{aligned} \mathcal{PS}[[\text{if } b \text{ then } S_1 \text{ else } S_2]] &= \lambda ps.\text{if } \mathcal{PB}[[b]] ps = \text{OK} \\ &\quad \text{then } (\mathcal{PS}[[S_1]] ps) \\ &\quad \sqcup_{PS} (\mathcal{PS}[[S_2]] ps) \\ &\quad \text{else LOST} \end{aligned}$$

$$\mathcal{PS}[[\text{while } b \text{ do } S]] = \text{fix } H$$

där $H = \lambda h.\lambda ps.\text{if } \mathcal{PB}[[b]] ps = \text{OK}$
 $\quad \text{then } h(\mathcal{PS}[[S]] ps) \sqcup_{PS} ps$
 $\quad \text{else LOST}$

och $ps_1 \sqcup_{PS} ps_2 = \lambda x.(ps_1 x) \sqcup_P (ps_2 x)$

Exempel:

ps	
x	OK
y	OK
z	D?
<i>on-track</i>	OK

$(\mathcal{PS}[\text{if } x=x \text{ then } z:=y \text{ else } y:=z] \ ps) \ z =$

$$\begin{aligned}
 \mathcal{PB}[\mathbf{x}=\mathbf{x}] \ ps &= \mathcal{PA}[\mathbf{x}] \ ps \sqcup_P \mathcal{PA}[\mathbf{x}] \ ps \\
 &= \text{OK} \sqcup_P \text{OK} \\
 &= \text{OK}
 \end{aligned}$$

$$= (\mathcal{PS}[z:=y] \ ps \sqcup_{PS} \mathcal{PS}[y := z] \ ps) \ z =$$

$$= (ps[z \mapsto \text{OK}] \sqcup_{PS} ps[y \mapsto \text{D?}]) \ z =$$

$$= (\lambda x. ps[z \mapsto \text{OK}](x) \sqcup_P ps[y \mapsto \text{D?}](x)) \ z =$$

$$= (\text{OK} \sqcup_P \text{D?}) = \text{D?}$$

Exempel:

ps_0	
x	OK
y	D?
$on-track$	OK

Analysera satsen

$y := 1; \text{ while } \neg(x = 1) \text{ do } \underbrace{(y := y*x; x := x-1)}_S$

$\mathcal{PS}[\![y:=1; \text{ while } \neg(x=1) \text{ do } S]\!] ps_0$

$= \text{fix } H \underbrace{ps_0[y \mapsto \text{OK}]}_{ps'} \quad \text{där } H \ h \ ps =$

$$\begin{cases} h(\mathcal{PS}[\![S]\!] ps) \sqcup_{PS} ps & \text{om } \mathcal{PB}[\![\neg(x=1)]\!] ps = \text{OK} \\ \text{LOST} & \text{om } \mathcal{PB}[\![\neg(x=1)]\!] ps = \text{D?} \end{cases}$$

- $(H^0 \perp) ps' = \perp ps' = \text{INIT}$

- $(H^1 \perp) ps' = H \perp ps' =$

$$\mathcal{PB}[\neg(\mathbf{x}=1)] ps' = ps'(\mathbf{x}) = \text{OK}$$

$$= \perp(\dots) \sqcup_{PS} ps' = \text{INIT} \sqcup_{PS} ps' = ps'$$

- $(H^2 \perp) ps' = H(H^1 \perp) ps' =$

$$= (H^1 \perp)(\mathcal{PS}[y:=y*x; x:=x-1] ps' \sqcup_{PS} ps') =$$

$$= (H^1 \perp)(\mathcal{PS}[x:=x-1](\mathcal{PS}[y:=y*x] ps')) \sqcup_{PS} ps')$$

$$= (H^1 \perp)(\mathcal{PS}[x:=x-1](ps'[y \mapsto \text{OK}]) \sqcup_{PS} ps')$$

$$= (H^1 \perp)(ps'[y \mapsto \text{OK}][x \mapsto \text{OK}]) \sqcup_{PS} ps')$$

$$= (H^1 \perp)(ps' \sqcup_{PS} ps') = (H^1 \perp)(ps')$$

$$\Rightarrow \text{fix } H \text{ } ps' = ps' \text{ och speciellt } ps'(y) = \text{OK}.$$

Inlämningsuppgift

- Undersök satsen

$z:=0; \text{ while } y \leq x \text{ do } (z:=z+1; x:=x-y)$

för att se om det finns ett funktionellt beroende mellan z och invariablerna x och y .

Antag följande abstrakta initialtillstånd:

ps_0	
x	OK
y	OK
$on-track$	OK

Gör sedan om analysen för $ps_0[y \mapsto D?]$.

9 Axiomatisk semantik, programbevis

Översikt

- Partiella korrekthetsbevis
- Totala korrekthetsbevis

Partiell korrekthet

En *partiell* korrekthetsspecifikation är ett uttryck

$$\{P\} S \{Q\}$$

P och Q kallas *pre-* respektive *post-*villkor.

- om P är sant i starttillståndet, och
- om exekveringen av S terminerar
- så kommer Q att vara sant i sluttillståndet.

P och Q är *predikat*, dvs av typen **State** $\rightarrow$ **T**.

Exempel: Följande specifikation är sann:

$$\{\lambda s.sx = 1\} \quad x := x + 1 \quad \{\lambda s.sx = 2\}$$

Exempel: Följande specifikation är sann:

$$\{\lambda s.sx = 1\} \quad y := x \quad \{\lambda s.sy = 1\}$$

Exempel: Följande specifikation är falsk:

$$\{\lambda s.sx = X \wedge sy = Y\}$$

$$x := y; \quad y := x$$

$$\{\lambda s.sy = X \wedge sx = Y\}$$

Extensionella predikat

Variablerna X och Y i predikatet

$$\lambda s. s\mathbf{x} = X \wedge s\mathbf{y} = Y$$

kallas *logiska variabler* och förekommer inte i program.

Predikat har typen $\mathbf{State} \rightarrow \mathbf{T}$. Förkortningar:

$$\begin{aligned} \mathbf{x} &\equiv s(\mathbf{x}) \\ \text{true} &\equiv \lambda s. tt \\ \text{false} &\equiv \lambda s. ff \\ P_1 \wedge P_2 &\equiv \lambda s. (P_1 s) \text{ och } (P_2 s) \\ P_1 \vee P_2 &\equiv \lambda s. (P_1 s) \text{ eller } (P_2 s) \\ \neg P &\equiv \lambda s. \text{icke } (P s) \\ P[x \mapsto \mathcal{A}[[a]]] &\equiv \lambda s. P(s[x \mapsto \mathcal{A}[[a]]]) \\ P_1 \Rightarrow P_2 &\equiv \lambda s. P_1 s \text{ implicerar } P_2 s \end{aligned}$$

Exempel

Med hjälp av förkortningarna kan vi skriva:

- $\{x = 1\} \quad x := x + 1 \quad \{x = 2\}$
- $\{x = 1\} \quad y := x \quad \{y = 1\}$
- $\{x = X \wedge y = Y\} \quad x := y; \quad y := x \quad \{y = X \wedge x = Y\}$

Ibland kommer vi också att slopa $\mathcal{A}[\dots]$ och $\mathcal{B}[\dots]$.

Inferenssystem för partiell korrekthet

$$[\text{ass}] \quad \{P[x \mapsto \mathcal{A}[[a]]]\} \ x := a \ \{P\}$$

$$[\text{skip}] \quad \{P\} \ \text{skip} \ \{P\}$$

$$[\text{comp}] \quad \frac{\{P\} \ S_1 \ \{Q\} \quad \{Q\} \ S_2 \ \{R\}}{\{P\} \ S_1; S_2 \ \{R\}}$$

$$[\text{if}] \quad \frac{\{\mathcal{B}[[b]] \wedge P\} \ S_1 \ \{Q\} \quad \{\neg \mathcal{B}[[b]] \wedge P\} \ S_2 \ \{Q\}}{\{P\} \ \text{if } b \text{ then } S_1 \ \text{else } S_2 \ \{Q\}}$$

$$[\text{while}] \quad \frac{\{\mathcal{B}[[b]] \wedge P\} \ S \ \{P\}}{\{P\} \ \text{while } b \text{ do } S \ \{\neg \mathcal{B}[[b]] \wedge P\}}$$

$$[\text{cons}] \quad \frac{\{P'\} \ S \ \{Q'\}}{\{P\} \ S \ \{Q\}} \quad \text{om } P \Rightarrow P' \text{ och } Q' \Rightarrow Q$$

Kommentarer

- $[\text{ass}] \quad \{P[x \mapsto \mathcal{A}[[a]]]\} x := a \{P\}$

Många tycker att detta är helt baklänges och borde skrivas

$$\{P\} x := a \{P[x \mapsto \mathcal{A}[[a]]]\}$$

Men en instans av denna regel är t ex:

$$\{x = 0\} x := 1 \{1 = 0\}$$

vilket ju inte är sant. Med $[\text{ass}]$ får vi istället

$$\{P[x \mapsto 1]\} x := 1 \{P\}$$

vilket är korrekt.

- $$\frac{\{ \mathcal{B}[[b]] \wedge P \} \ S \ \{ P \}}{\{ P \} \ \text{while } b \ \text{do } S \ \{ \neg \mathcal{B}[[b]] \wedge P \}}$$
- [while]

P kallas en *invariant* och är sann både före och efter loopen.

- $$\frac{\{ P' \} \ S \ \{ Q' \}}{\{ P \} \ S \ \{ Q \}} \quad \text{om } P \Rightarrow P' \text{ och } Q' \Rightarrow Q$$
- [cons]

Vi kan alltså alltid förstärka P' eller försvaga Q' .

Kom ihåg att $P \Rightarrow Q \equiv \neg P \vee Q$.

Inferensträd

Ett inferensträd är ett bevis för egenskapen i roten och konstrueras från löven till roten.

Vi skriver

$$\vdash \{ P \} \ S \ \{ Q \}$$

om $\{ P \} \ S \ \{ Q \}$ kan visas mha ett inferens-träd.

Exempel

$$\frac{\frac{\frac{\{true\} \text{ skip } \{true\}}{\{true \wedge true\} \text{ skip } \{true\}} [\text{cons}^*]}{\{true\} \text{ while true do skip } \{\neg true \wedge true\}} [\text{while}]}{\{true\} \text{ while true do skip } \{true\}} [\text{const}^\dagger]$$

Här har vi använt $[\text{cons}]$ två ggr:

* $true \wedge true \Rightarrow true$

† $\neg true \wedge true \Rightarrow true$

Exempel (utdelat material)

$$\begin{array}{l}
\vdash \{x = n\} \\
\text{while } \neg(x=1) \text{ do } (y:=y*x; x:=x-1) \\
\{y = n! \wedge n > 0\}
\end{array}$$

Bevisligen ekvivalent

Två satser S_1 och S_2 är bevisligen ekvivalenta om

$$\vdash \{P\} S_1 \{Q\} \quad \Longleftrightarrow \quad \vdash \{P\} S_2 \{Q\}$$

gäller för alla P och Q

Giltighet

$\{P\} S \{Q\}$ är *giltig* omm för alla tillstånd s :

$$\text{om } P(s) \text{ och } \langle S, s \rangle \rightarrow s' \text{ så } Q(s')$$

Detta betecknas

$$\models \{P\} S \{Q\}$$

och är alltså en operationell tolkning av beviset.

Sundhet och fullständighet

Följande gäller (se boken för bevis):

Sundhet: $\vdash \{P\} S \{Q\} \Rightarrow \models \{P\} S \{Q\}.$

Fullständighet: $\models \{P\} S \{Q\} \Rightarrow \vdash \{P\} S \{Q\}.$

Frivillig inlämningsuppgift

- Uppgift 6.10: Visa med ett inferensträd

$$\vdash \{x = n \wedge x > 0 \wedge y > 0\}$$
$$z := 0; \text{ while } y \leq x \text{ do } (z := z + 1; x := x - y)$$
$$\{x = \lfloor n/y \rfloor \wedge x = n \bmod y\}$$

Förslag på invariant:

$$(x \geq 0 \wedge y \geq 0) \Rightarrow (x + z \cdot y = n)$$

Total korrekthet

En *total* korrekthetsspecifikation är ett uttryck

$$\{P\} S \{\Downarrow Q\}$$

- om P är sant i starttillståndet, och
- när exekveringen av S terminerar
- så kommer Q att vara sant i sluttillståndet.

Total korrekthet = Partiell korrekthet + Terminering.

$$\models \{P\} S \{\Downarrow Q\}$$

$$\Longleftrightarrow$$

$$(\forall s)(P(s) \Rightarrow (\exists s')(\langle S, s \rangle \rightarrow s' \wedge Q(s'))))$$

Inferenssystem för total korrekthet

$$[\text{ass}] \quad \{P[x \mapsto \mathcal{A}[[a]]]\} \ x := a \ \{\Downarrow P\}$$

$$[\text{skip}] \quad \{P\} \ \text{skip} \ \{\Downarrow P\}$$

$$[\text{comp}] \quad \frac{\{P\} \ S_1 \ \{\Downarrow Q\} \quad \{Q\} \ S_2 \ \{\Downarrow R\}}{\{P\} \ S_1; S_2 \ \{\Downarrow R\}}$$

$$[\text{if}] \quad \frac{\{\mathcal{B}[[b]] \wedge P\} \ S_1 \ \{\Downarrow Q\} \quad \{\neg \mathcal{B}[[b]] \wedge P\} \ S_2 \ \{\Downarrow Q\}}{\{P\} \ \text{if } b \text{ then } S_1 \ \text{else } S_2 \ \{\Downarrow Q\}}$$

$$[\text{while}] \quad \frac{\{P(z+1)\} \ S \ \{\Downarrow P(z)\}}{\{\exists z. P(z)\} \ \text{while } b \ \text{do } S \ \{\Downarrow P(0)\}}$$

$$[\text{cons}] \quad \frac{\{P'\} \ S \ \{\Downarrow Q'\}}{\{P\} \ S \ \{\Downarrow Q\}} \quad \text{om } P \Rightarrow P' \text{ och } Q' \Rightarrow Q$$

där $P(0) \Rightarrow \neg \mathcal{B}[[b]]$, $P(z+1) \Rightarrow \mathcal{B}[[b]]$

Kommentarer

Allt är som förut bortsett från [while]:

$$\frac{\{P(z+1)\} S \{\Downarrow P(z)\}}{\{\exists z.P(z)\} \text{ while } b \text{ do } S \{\Downarrow P(0)\}} \quad \begin{array}{l} P(0) \Rightarrow \neg \mathcal{B}[[b]] \\ P(z+1) \Rightarrow \mathcal{B}[[b]] \end{array}$$

$P(z)$ är invarianten, z är antalet varv som behövs.

Bevisbarhet: $\vdash \{P\} S \{\Downarrow Q\}$

om det finns ett inferensträd med $\{P\} S \{\Downarrow Q\}$ som rot.

Exempel (utdelat material)

$$\begin{array}{l} \vdash \{x > 0 \wedge x = n\} \\ \quad \text{while } \neg(x=1) \text{ do } (y:=y*x; x:=x-1) \\ \quad \{\Downarrow y = n!\} \end{array}$$

Sundhet och fullständighet

Inferenssystemet för total korrekthet är både sunt och fullständigt:

$$\vdash \{P\} S \{\Downarrow Q\} \quad \Longleftrightarrow \quad \models \{P\} S \{\Downarrow Q\}$$

Dessutom:

$$\vdash \{P\} S \{\Downarrow Q\} \quad \Longrightarrow \quad \vdash \{P\} S \{Q\}$$

Proceduranrop och rekursion

$$\frac{\{P\} S \{Q\}}{\{P\} \text{ call } p \{Q\}} \quad \text{och} \quad \frac{\{P\} S \{\Downarrow Q\}}{\{P\} \text{ call } p \{\Downarrow Q\}}$$

där p definieras av `proc  $p$  is  $S$` .

Problem: Antag att S innehåller `call  $p$` .

För att visa $\{P\} \text{ call } p \{Q\}$:

1. Visa $\{P\} S \{Q\}$

vilket leder till att vi efter ett tag måste visa

2. $\{P'\} \text{ call } p \{Q'\}$

och nu är vi tillbaks där vi började.

Vi behöver en ny regel!

För partiell korrekthet:

$$\frac{\{P\} \text{ call } p \{Q\} \vdash \{P\} S \{Q\}}{\{P\} \text{ call } p \{Q\}}$$

Förklaring: För att visa $\{P\} \text{ call } p \{Q\}$:

- Antag att $\{P\} \text{ call } p \{Q\}$ är sant.
- Visa att $\{P\} S \{Q\}$ är sant under detta antagande.

För total korrekthet: ($z > 0$ och $\neg P(0)$ är sant)

$$\frac{\{P(z)\} \text{ call } p \{\Downarrow Q\} \vdash \{P(z+1)\} S \{\Downarrow Q\}}{\{\exists z.P(z)\} \text{ call } p \{\Downarrow Q\}}$$

Förklaring: Antalet rekursiva anrop är begränsade.

- Antag att

$$\{P(z)\} \text{ call } p \{\Downarrow Q\}$$

gäller för alla rekursiva anrop med djup $\leq z$.

- Visa att

$$\{P(z+1)\} S \{\Downarrow Q\}$$

dvs, att det gäller för ännu en anropsnivå.

- Slutsatsen

$$\{\exists z.P(z)\} \text{ call } p \{\Downarrow Q\}$$

säger att rekursionen fungerar för ett visst djup z .

Sista sidan

- Glöm inte att fylla i kursvärderingen!
- God jul! Kom ihåg att skinka $\sqsubseteq$ rödkål.

10 Bilagor

Följande blad delades ut under kursen som komplement till bilderna.

Grammatik

$$\begin{aligned}
a \in \mathbf{Aexp} &\rightarrow n \mid x \mid a_1 + a_2 \mid a_1 * a_2 \mid a_1 - a_2 \\
b \in \mathbf{Bexp} &\rightarrow \mathbf{true} \mid \mathbf{false} \mid a_1 = a_2 \mid a_1 \leq a_2 \mid \neg b \mid b_1 \wedge b_2 \\
S \in \mathbf{Stm} &\rightarrow x := a \mid \mathbf{skip} \mid S_1; S_2 \mid \mathbf{if } b \mathbf{ then } S_1 \mathbf{ else } S_2 \mid \mathbf{while } b \mathbf{ do } S \mid \mathbf{call } p \mid \mathbf{begin } D_V \ D_P \ S \mathbf{ end} \\
D_V \in \mathbf{Dec}_V &\rightarrow \mathbf{var } x := a; D_V \mid \varepsilon \\
D_P \in \mathbf{Dec}_P &\rightarrow \mathbf{proc } p \mathbf{ is } S; D_P \mid \varepsilon
\end{aligned}$$

Domäner

$$\begin{aligned}
env_P \in \mathbf{Env}_P &= \mathbf{Pname} \hookrightarrow \mathbf{Stm} \times \mathbf{Env}_V \times \mathbf{Env}_P \\
env_V \in \mathbf{Env}_V &= \mathbf{Var} \rightarrow \mathbf{Loc} \\
sto \in \mathbf{Store} &= \mathbf{Loc} \cup \{\mathbf{next}\} \rightarrow \mathbf{Z} \\
\mathbf{Loc} &= \mathbf{Z}
\end{aligned}$$

Deklaration av variabler

$$\begin{aligned}
&\langle \varepsilon, env_V, sto \rangle \rightarrow_D \langle env_V, sto \rangle \\
&\frac{\langle D_V, env_V[x \mapsto l], sto[l \mapsto v][\mathbf{next} \mapsto l + 1] \rangle \rightarrow_D \langle env'_V, sto' \rangle}{\langle \mathbf{var } x := a; D_V, env_V, sto \rangle \rightarrow_D \langle env'_V, sto' \rangle} \quad \text{där } v = \mathcal{A}[a](sto \circ env_V), \ l = sto(\mathbf{next})
\end{aligned}$$

Deklaration av procedurer

$$\begin{aligned}
&\mathbf{upd}_P(\varepsilon, env_V, env_P) = env_P \\
&\mathbf{upd}_P(\mathbf{proc } p \mathbf{ is } S; D_P, env_V, env_P) = \mathbf{upd}_P(D_P, env_V, env_P[p \mapsto (S, env_V, env_P)])
\end{aligned}$$

Naturlig semantik för While med block

$$\begin{aligned}
&\frac{\langle D_V, env_V, sto \rangle \rightarrow_D \langle env'_V, sto' \rangle \quad env'_V, \mathbf{upd}_P(D_P, env'_V, env_P) \vdash \langle S, sto' \rangle \rightarrow sto''}{env_V, env_P \vdash \langle \mathbf{begin } D_V \ D_P \ S \mathbf{ end}, sto \rangle \rightarrow sto''} \\
&env_V, env_P \vdash \langle x := a, sto \rangle \rightarrow sto[env_V(x) \mapsto \mathcal{A}[a](sto \circ env_V)] \\
&env_V, env_P \vdash \langle \mathbf{skip}, sto \rangle \rightarrow sto \\
&\frac{env_V, env_P \vdash \langle S_1, sto \rangle \rightarrow sto' \quad env_V, env_P \vdash \langle S_2, sto' \rangle \rightarrow sto''}{env_V, env_P \vdash \langle S_1; S_2, sto \rangle \rightarrow sto''} \\
&\frac{env'_V, env'_P[p \mapsto (S, env'_V, env'_P)] \vdash \langle S, sto \rangle \rightarrow sto'}{env_V, env_P \vdash \langle \mathbf{call } p, sto \rangle \rightarrow sto'} \quad \text{där } env_P(p) = (S, env'_V, env'_P) \\
&\frac{env_V, env_P \vdash \langle S_1, sto \rangle \rightarrow sto'}{env_V, env_P \vdash \langle \mathbf{if } b \mathbf{ then } S_1 \mathbf{ else } S_2, sto \rangle \rightarrow sto'} \quad \text{om } \mathcal{B}[b](sto \circ env_V) = \mathbf{tt} \\
&\frac{env_V, env_P \vdash \langle S_2, sto \rangle \rightarrow sto'}{env_V, env_P \vdash \langle \mathbf{if } b \mathbf{ then } S_1 \mathbf{ else } S_2, sto \rangle \rightarrow sto'} \quad \text{om } \mathcal{B}[b](sto \circ env_V) = \mathbf{ff} \\
&\frac{env_V, env_P \vdash \langle S, sto \rangle \rightarrow sto' \quad env_V, env_P \vdash \langle \mathbf{while } b \mathbf{ do } S, sto' \rangle \rightarrow sto''}{env_V, env_P \vdash \langle \mathbf{while } b \mathbf{ do } S, sto \rangle \rightarrow sto''} \quad \text{om } \mathcal{B}[b](sto \circ env_V) = \mathbf{tt} \\
&env_V, env_P \vdash \langle \mathbf{while } b \mathbf{ do } S, sto \rangle \rightarrow sto \quad \text{om } \mathcal{B}[b](sto \circ env_V) = \mathbf{ff}
\end{aligned}$$

```
begin var x := 0;
```

```
  proc p is x := x * 2;
  proc q is call p;
```

```
  begin var x := 5;
    proc p is x := x + 1;
    call q; y := x
  end
```

```
end
```

$$\frac{\frac{\langle \epsilon, env_V^1, sto^1 \rangle \rightarrow_D (env_V^1, sto^1)}{\langle \text{var } x := 0, env_V^0, sto^0 \rangle \rightarrow_D (env_V^1, sto^1)} \quad \frac{\frac{\langle \epsilon, env_V^2, sto^2 \rangle \rightarrow_D (env_V^2, sto^2)}{\langle \text{var } x := 5, env_V^1, sto^1 \rangle \rightarrow_D (env_V^2, sto^2)} \quad \frac{\frac{env_V^1, env_P^0 \vdash \langle x := x * 2, sto^2 \rangle \vdash sto^3}{env_V^1, env_P^3 \vdash \langle \text{call } p, sto^2 \rangle \rightarrow sto^3} \quad \frac{env_V^2, env_P^2 \vdash \langle \text{call } q, sto^2 \rangle \rightarrow sto^3}{env_V^2, env_P^2 \vdash \langle \text{call } q; y := x, sto^2 \rangle \rightarrow sto^4} \quad env_V^2, env_P^2 \vdash \langle y := x, sto^3 \rangle \rightarrow sto^4}{env_V^1, env_P^1 \vdash \langle \text{begin var } x := 5; \text{proc } p \text{ is } x := x + 1; \text{call } q; y := x \text{ end}, sto^1 \rangle \rightarrow sto^4}}{env_V^0, env_P^0 \vdash \langle \text{begin var } x := 0; \text{proc } p \text{ is } \dots \text{end}, sto^0 \rangle \rightarrow sto^4}$$

Initial environment: $env_V^0(y) = 0, env_V^0(\text{next}) = 1$.

$$\begin{aligned} env_V^1 &= env_V^0[x \mapsto 1][\text{next} \mapsto 2] \\ sto^1 &= sto^0[1 \mapsto 0] \\ env_P^1 &= upd_P(\text{proc } p \text{ is } x := x * 2; \text{proc } q \text{ is call } p, env_V^1, env_P^0) \\ &= upd_P(\text{proc } q \text{ is call } p, env_V^1, env_P^0[p \mapsto (x := x * 2, env_V^1, env_P^0)]) \\ &= env_P^0[p \mapsto (x := x * 2, env_V^1, env_P^0)][q \mapsto (\text{call } p, env_V^1, env_P^0[p \mapsto (x := x * 2, env_V^1, env_P^0)])] \\ env_V^2 &= env_V^1[x \mapsto 2][\text{next} \mapsto 3] \\ sto^2 &= sto^1[2 \mapsto 5] \\ env_P^2 &= upd_P(\text{proc } p \text{ is } x := x + 1, env_V^2, env_P^1) \\ &= env_P^1[p \mapsto (x := x + 1, env_V^2, env_P^1)] \\ env_P^3 &= env_P^2[q \mapsto (\text{call } p, env_V^1, env_P^2[p \mapsto (x := x * 2, env_V^1, env_P^2)])] \\ sto^3 &= sto^2[env_V^1(x) \mapsto \mathcal{A}[\mathbf{x}](sto^2 \circ env_V^1) * 2] = sto^2[1 \mapsto 0] \\ sto^4 &= sto^3[env_V^2(y) \mapsto \mathcal{A}[\mathbf{x}](sto^3 \circ env_V^2) = sto^3[0 \mapsto 5]] \end{aligned}$$

Grammatik

$$\begin{aligned}
a \in \mathbf{Aexp} &\rightarrow n \mid x \mid a_1 + a_2 \mid a_1 * a_2 \mid a_1 - a_2 \\
b \in \mathbf{Bexp} &\rightarrow \text{true} \mid \text{false} \mid a_1 = a_2 \mid a_1 \leq a_2 \mid \neg b \mid b_1 \wedge b_2 \\
S \in \mathbf{Stm} &\rightarrow x := a \mid \text{skip} \mid S_1; S_2 \mid \text{if } b \text{ then } S_1 \text{ else } S_2 \mid \text{while } b \text{ do } S \mid \text{call } p \mid \text{begin } D_V \ D_P \ S \text{ end} \\
D_V \in \mathbf{Dec}_V &\rightarrow \text{var } x := a; D_V \mid \varepsilon \\
D_P \in \mathbf{Dec}_P &\rightarrow \text{proc } p \text{ is } S; D_P \mid \varepsilon
\end{aligned}$$

Domäner

$$\begin{aligned}
env_P \in \mathbf{Env}_P &= \mathbf{PName} \rightarrow (\mathbf{Store} \rightarrow \mathbf{Store}) \\
env_V \in \mathbf{Env}_V &= \mathbf{Var} \rightarrow \mathbf{Loc} \\
sto \in \mathbf{Store} &= \mathbf{Loc} \cup \{\text{next}\} \rightarrow \mathbf{Z} \\
\mathbf{Loc} &= \mathbf{Z}
\end{aligned}$$

Deklaration av variabler

$$\begin{aligned}
\mathcal{D}_{ds}^V : \mathbf{Dec}_V &\rightarrow (\mathbf{Env}_V \times \mathbf{Store}) \rightarrow (\mathbf{Env}_V \times \mathbf{Store}) \\
\mathcal{D}_{ds}^V[\varepsilon](env_V, sto) &= (env_V, sto) \\
\mathcal{D}_{ds}^V[\text{var } x := a; D_V](env_V, sto) &= \\
&\mathcal{D}_{ds}^V[D_V](env_V[x \mapsto l], sto[l \mapsto v][\text{next} \mapsto l + 1]), \text{ där } l = sto.\text{next} \text{ och } v = \mathcal{A}[a](sto \circ env_V).
\end{aligned}$$

Deklaration av procedurer

$$\begin{aligned}
\mathcal{D}_{ds}^P : \mathbf{Dec}_P &\rightarrow \mathbf{Env}_V \rightarrow \mathbf{Env}_P \rightarrow \mathbf{Env}_P \\
\mathcal{D}_{ds}^P[\varepsilon] env_V &= id \\
\mathcal{D}_{ds}^P[\text{proc } p \text{ is } S; D_P] env_V env_P &= \\
&\mathcal{D}_{ds}^P[D_P] env_V (env_P[p \mapsto \text{fix}(\lambda f. \mathcal{S}_{ds}[S] env_V (env_P[p \mapsto f]))])
\end{aligned}$$

Denotationssemantik för While med block

$$\begin{aligned}
\mathcal{S}_{ds} : \mathbf{Stm} &\rightarrow \mathbf{Env}_V \rightarrow \mathbf{Env}_P \rightarrow (\mathbf{Store} \hookrightarrow \mathbf{Store}) \\
\mathcal{S}_{ds}[\text{call } p] env_V env_P &= env_P p \\
\mathcal{S}_{ds}[\text{begin } D_V \ D_P \ S \text{ end}] env_V env_P sto &= \mathcal{S}_{ds}[S] env'_V env'_P sto', \\
&\text{där } (env'_V, sto') = \mathcal{D}_{ds}^V[D_V](env_V, sto), \text{ } env'_P = \mathcal{D}_{ds}^P[D_P] env'_V env_P. \\
\mathcal{S}_{ds}[x := a] env_V env_P sto &= sto[env_V x \mapsto \mathcal{A}[a](sto \circ env_V)] \\
\mathcal{S}_{ds}[\text{skip}] env_V env_P sto &= sto \\
\mathcal{S}_{ds}[S_1; S_2] env_V env_P &= (\mathcal{S}_{ds}[S_2] env_V env_P) \circ (\mathcal{S}_{ds}[S_1] env_V env_P) \\
\mathcal{S}_{ds}[\text{if } b \text{ then } S_1 \text{ else } S_2] env_V env_P sto &= \\
&\text{if } \mathcal{B}[b](sto \circ env_V) \text{ then } \mathcal{S}_{ds}[S_1] env_V env_P \text{ else } \mathcal{S}_{ds}[S_2] env_V env_P \\
\mathcal{S}_{ds}[\text{while } b \text{ do } S] env_V env_P &= \\
&\text{fix } (\lambda g. \lambda s. \text{if } \mathcal{B}[b](sto \circ env_V) \text{ then } g(\mathcal{S}_{ds}[S] env_V env_P s) \text{ else } s)
\end{aligned}$$

Invariant: $\text{INV} \equiv (x > 0 \Rightarrow (y * x! = n! \wedge n \geq x))$

$$\frac{\frac{\frac{\frac{\text{INV}[x \mapsto x-1][y := y * x] \quad y := y * x \quad \text{INV}[x \mapsto x-1] \quad \text{INV}[x \mapsto x-1] \quad x := x-1 \quad \text{INV}}{\text{INV}[x \mapsto x-1][y := y * x] \quad y := y * x; \quad x := x-1 \quad \text{INV}} \quad 3}{\neg(x=1) \wedge \text{INV} \quad y := y * x; \quad x := x-1 \quad \text{INV}} \quad 2}{\frac{\text{INV}[y \mapsto 1] \quad y := 1 \quad \text{INV}}{x = n \quad y := 1 \quad \text{INV}} \quad 1 \quad \frac{\text{INV} \quad \text{while } \neg(x=1) \text{ do } (y := y * x; x := x-1) \quad \neg \neg(x=1) \wedge \text{INV}}{\text{INV} \quad \text{while } \neg(x=1) \text{ do } (y := y * x; x := x-1) \quad \{y = n! \wedge n > 0\}} \quad 2} \quad \{x = n\} \quad y := 1; \text{ while } \neg(x=1) \text{ do } (y := y * x; x := x-1) \quad \{y = n! \wedge n > 0\}$$

Kommentarer till inferensstegen:

1) $x = n \Rightarrow \text{INV}[y \mapsto 1]$ är sant eftersom

$$\begin{aligned} \text{INV}[y \mapsto 1] &= (x > 0 \Rightarrow x! = n! \wedge n \geq x) \\ &= (x > 0 \Rightarrow x = n \wedge n \geq x) \\ &= (x > 0 \Rightarrow x = n) \end{aligned}$$

och $p \Rightarrow (q \Rightarrow p) = (\neg p \vee (\neg q \vee p)) = \text{true}$

2) $\neg \neg(x=1) \wedge \text{INV} \Rightarrow y = n! \wedge n > 0$ är sant eftersom

$$\begin{aligned} \neg \neg(x=1) \wedge \text{INV} &= (x=1) \wedge (1 > 0 \Rightarrow y = n! \wedge n \geq 1) \\ &= (x=1) \wedge (y = n! \wedge n > 0) \end{aligned}$$

3) $\neg(x=1) \wedge \text{INV} \Rightarrow \text{INV}[x \mapsto x-1][y \mapsto y * x]$ är sant eftersom

$$\begin{aligned} \text{INV}[x \mapsto x-1][y \mapsto y * x] &= (x-1) > 0 \Rightarrow (y * x * (x-1)! = n! \wedge n \geq (x-1)) \\ &= (x-1) > 0 \Rightarrow (y * x! = n! \wedge n \geq (x-1)) \end{aligned}$$

Tre fall:

- $x = 0$: Då får vi $\text{false} \Rightarrow \text{false} = \text{true}$.
- $x = 1$: Då får vi $\text{false} \Rightarrow \text{INV}[x \mapsto x-1][y \mapsto y * x] = \text{true}$.
- $x > 1$: Då är både $\neg(x=1)$ och $(x-1 > 0)$ sant, dvs $(\text{INV} \Rightarrow \text{INV}) = \text{true}$.

Programmeringsspråket Gofer

Kjell Post

13 september 2026

1 Introduktion

- **Gofer** är ett *funktionellt* språk. Funktioner kan skickas som argument, returneras som resultat och lagras i variabler. Den huvudsakliga kontrollstrukturen är rekursion.
- **Gofer** är *interaktivt*. Uttryck analyseras och körs varefter värdet skrivs ut.
- **Gofer** är *starkt typat*. Varje giltigt uttryck har en typ som härleds av systemet. Stark typning garanterar att typfel inte uppstår under körning.
- **Gofer** har *lat evaluering* vilket innebär att argument till funktioner inte evalueras förrän de behövs.

Gofer påminner mycket om Haskell, ett annat funktionellt språk med lat evaluering. Till skillnad från Haskell kan **Gofer** köras på en mängd datorer, t ex hemma på din PC eller Mac. **Gofer** är gratis och implementationer för olika maskiner kan hämtas med ftp från <ftp.cs.chalmers.se> i katalogen `pub/haskell/gofer`.

Denna rapport är avsedd som en informell introduktion till **Gofer** och är absolut ingen fullständig definition av språket. För mer information, se [1] som finns på ovan nämnda ftp-arkiv.

2 Interaktion med Gofer

Gofer startas i UNIX med kommandot

```
$ gofer
```

Om detta inte fungerar, kontakta systemansvarig.

När systemet startat får du en prompt där du kan mata in uttryck:

```
? 3+2
5
(3 reductions, 7 cells)
```

Systemet evaluerar uttrycket och visar svaret, tillsammans med lite information om hur mycket resurser som krävdes.

Andra kommandon, för att t ex ladda in filer och avsluta **Gofer**, inleds med ":". En sammanställning av dessa kommandon fås genom att skriva ":"? vid prompten. Vi kommer att ta upp några av dessa senare.

Olika fel kan uppstå under evalueringen. De vanligaste felen är syntaxfel, typfel och runtime-fel. Du är säkert bekant med syntax- och runtime-fel från andra språk, men ett typfel är kanske mer obekant. Vi kommer att diskutera typer längre fram, så vi nöjer oss med att konstatera att ett typfel uppstår när ett värde används på fel sätt, t ex genom att addera 3 och True:

```
? 3+True
```

```
ERROR: Type error in application
*** expression      : 3 + True
*** term            : 3
*** type             : Int
*** does not match  : Bool
```

3 Uttryck, värden och typer

Låt oss börja med en översikt av de grundläggande typerna. I **Gofer** är en typ en mängd av värden. Till exempel så är heltalen en typ, liksom tecken och booleska värden. Givet två typer σ och τ så är mängden av ordnade par, med den vänstra komponenten av typen σ och den högra av typen τ , också en typ. En annan viktig typ är mängden av funktioner som mappar en typ till en annan. Utöver detta tillåter **Gofer** definition av egen-definierade typer, något som vi kommer att studera senare. Vi kommer att illustrera typsystemet med olika exempel.

3.1 Booleska

Typen Bool består av typerna False och True. Funktionerna not, && och || implementerar logisk negation, konjunktion och disjunktion, respektive.

```
? not False
True
? True && False
False
```

Också värt att nämna är det villkorliga uttrycket if b then e_1 else e_2 , där b måste vara av typen Bool. Dessutom måste typen på e_1 och e_2 vara lika, ty denna typ definierar resultat av hela if-uttrycket.

```
? if True then 1 else False
```

```
ERROR: Type error in conditional
*** expression      : if True then 1 else False
```

```
*** term          : False
*** type          : Bool
*** does not match : Int
```

3.2 Heltal

Typen `Int` omfattar alla positiva och negativa heltal.

```
? if 4*5 == 60/3 then 17 else -42
17
```

De vanliga aritmetiska operationerna `+`, `-`, `*`, `/` samt unärt `-` finns naturligtvis.

Referenser

[1] Mark Jones. *An Introduction to Gofer*, February 1995.

Programspråkssemantik
Laborationsmaterial 98/99



Introduktion till Gofer-Implementationerna

Vi börjar med några grundläggande definitioner för att kunna implementera de olika semantikstilarna i Gofer. Definitionerna baserar sig på materialet i kapitel 1 och finns på hemsidan under namnet `appB.gs`.

Abstrakt syntax och parsning

För **Num** använder vi den inbyggda typen `Int` i Gofer. För **Var** väljer vi en lista av tecken och definierar så följande typsynonym:

```
type Var = [Char]
```

För var och en av de syntaktiska kategorierna **Aexp**, **Bexp** och **Stm** definierar vi en algebraisk datatyp som täcker de olika produktionerna i While:s syntax.

```
data Aexp = N Int | V Var | Add Aexp Aexp |
           Mult Aexp Aexp | Sub Aexp Aexp

data Bexp = TRUE | FALSE | Eq Aexp Aexp | Le Aexp Aexp |
           Neg Bexp | And Bexp Bexp

data Stm  = Ass Var Aexp | Skip | Comp Stm Stm |
           If Bexp Stm Stm | While Bexp Stm | Repeat Stm Bexp
           Block Stm Var Stm | Raise Var
```

Exempel 1: Fakultetssatsen representeras av följande abstrakta datatyp i Gofer:

```
factorial = Comp (Ass "y" (N 1))
               (While (Neg (Eq (V "x") (N 1)))
                     (Comp (Ass "y" (Mult (V "y") (V "x")))
                           (Ass "x" (Sub (V "x") (N 1)))))
```

Observera att detta är en representation av den abstrakta syntaxen för satsen

```
y := 1; while ~(x = 1) do (y := y * x; x := x - 1)
```

□

Evaluering av uttryck

Vi börjar med att representera värden och tillstånd. De naturliga talen $\mathbf{Z}$ har sin motsvarighet i typen `Int`, så betydelsefunktionen $\mathcal{N}$ blir trivial. Booleska värden i $\mathbf{T}$ representeras av typen `Bool`. Detta leder till följande typsnyponyer:

```
type Z = Int
type T = Bool
```

Mängden **State** definieras som mängden av funktioner från variabler till naturliga tal:

```
type State = Var -> Z
```

Exempel 2: Tillståndet `s_init` avbildar `x` på **3** och alla andra variabler utom `x` på **0**.

```
s_init "x" = 3
s_init y   = 0
```

Observera att vi innesluter ett specifikt variabelnamn med "... " medan `y` kan vara vilken variabel som helst. $\square$

Uppdateringsoperationen $s[x \mapsto v]$ implementeras med följande funktion:

```
update s x v y = v,   if x == y
               = s y, otherwise
```

Betydelsefunktionerna $\mathcal{A}$ och $\mathcal{B}$ döps i implementationen till `a_val` och `b_val`, respektive. De definieras genom direkt översättning av tabell 1.1 och 1.2 i boken:

```
a_val :: Aexp -> State -> Z
b_val :: Bexp -> State -> T

a_val (N n) s      = n
a_val (V x) s      = s x
a_val (Add a1 a2) s = (a_val a1 s) + (a_val a2 s)
a_val (Mult a1 a2) s = (a_val a1 s) * (a_val a2 s)
a_val (Sub a1 a2) s = (a_val a1 s) - (a_val a2 s)

b_val TRUE s       = True
b_val FALSE s      = False
b_val (Eq a1 a2) s  = True,  if a_val a1 s == a_val a2 s
                   = False, if a_val a1 s /= a_val a2 s
b_val (Le a1 a2) s  = True,  if a_val a1 s <= a_val a2 s
                   = False, if a_val a1 s > a_val a2 s
b_val (Neg b) s     = True,  if b_val b s == False
                   = False, if b_val b s == True
b_val (And b1 b2) s = True,  if b_val b1 s == True &&
                           b_val b2 s == True
                   = False, if b_val b1 s == False ||
                           b_val b2 s == False
```

Operationell Semantik i Gofer

I detta avsnitt implementerar vi den naturliga semantiken och den strukturellt operationella semantiken ur kapitel 2. Vi behöver definitionerna från föregående avsnitt men eftersom Gofer inte har något `include`-direktiv måste `appB.gs` antingen laddas in manuellt eller via en projektfil. Förslagsvis skapar ni en projektfil `appC.gp` med följande innehåll:

```
appB.gs
appC.gs
```

För att ladda in filerna kan ni antingen skriva `:p appC.gp` i Gofer eller `gofer + appC.gp` från kommandoraden. Den kod som utvecklas i detta avsnitt placeras i `appC.gs`. Ett labskelett finns att hämta på kursens hemsida.

I kapitel 2 skiljde vi mellan två typer av konfigurationer, nämligen intermediära och slutgiltiga. Detta beskrivs med följande datatyp:

```
data Config = Inter Stm State | Final State
```

Vi börjar med att studera den naturliga semantiken och fortsätter sedan med den strukturella.

Naturlig semantik

Relationen $\rightarrow$ i avsnitt 2.1 i boken beskrivs med en funktion `ns_stm`:

```
ns_stm :: Config -> Config
```

Argumentet till denna funktion motsvarar den vänstra sidan till $\rightarrow$ medan det resultat som produceras motsvarar höger sida. Detta är möjligt eftersom sats 2.9 visade att relationen var deterministisk. Definitionen av `ns_stm` är en rent mekanisk översättning av tabell 2.1 i boken.

```
ns_stm (Inter (Ass x a) s)
    = Final (update s x (a_val a s))
```

```
ns_stm (Inter (Skip) s)
    = Final s
```

```
ns_stm (Inter (Comp ss1 ss2) s)
    = Final s''
```

```

    where Final s'  = ns_stm (Inter ss1 s)
          Final s'' = ns_stm (Inter ss2 s')

ns_stm (Inter (If b ss1 ss2) s)
  = Final s', if b_val b s
  where Final s' = ns_stm (Inter ss1 s)

ns_stm (Inter (If b ss1 ss2) s)
  = Final s', if not(b_val b s)
  where Final s' = ns_stm (Inter ss2 s)

ns_stm (Inter (While b ss) s)
  = Final s'', if b_val b s
  where Final s'  = ns_stm (Inter ss s)
        Final s'' = ns_stm (Inter (While b ss) s')

ns_stm (Inter (While b ss) s)
  = Final s, if not(b_val b s)

```

Observera att i axiomet för tilldelning används `update s x v` för att realisera $s[x \mapsto v]$. Den semantiska funktionen $\mathcal{S}_{\text{ns}}$ kan nu definieras som

```

s_ns ss s = s'
  where Final s' = ns_stm (Inter ss s)

```

Exempel 3: Vi kan exekvera fakultetssatsen från tillståndet `s_init` som avbildade `x` på `3` och alla andra variabler på `0`. Det slutgiltiga tillståndet `s_fac` erhålles på följande vis:

```

s_fac = s_ns factorial s_init

```

För att ta reda på värdet av `y` kan vi evaluera `s_fac "y"`. □

Uppgift 1: Utöka definitionen av `ns_stm` till att innehålla `repeat`-konstruktionen. Redovisa genom att köra följande program

```

y := 1;
repeat (y := y * x;
        x := x - 1)
until (x <= 0)

```

på samma sätt som i föregående exempel. □

Strukturell Operationell Semantik

När vi specificerar den strukturella semantiken måste vi avgöra om $\Rightarrow$ har producerat en intermediär eller slutgiltig konfiguration. För detta ändamål introducerar vi funktionen `is_final`:

```
is_Final (Inter ss s) = False
is_Final (Final s)    = True
```

Relationen $\Rightarrow$ implementeras sedan med funktionen `sos_stm`. Den har följande typ:

```
sos_stm :: Config -> Config
```

Precis som i den naturliga semantiken motsvarar vänster- och högersida till $\Rightarrow$ argument och resultat från funktionen. Återigen ska det betonas att denna implementation endast är möjlig eftersom semantiken är deterministisk (uppgift 2.22 i boken). En översättning av tabell 2.2 leder till följande definition av `sos_stm`:

```
sos_stm (Inter (Ass x a) s)
    = Final (update s x (a_val a s))

sos_stm (Inter (Skip) s) = Final s

sos_stm (Inter (Comp ss1 ss2) s)
    = Inter (Comp ss1' ss2) s', if not(is_Final (sos_stm (Inter ss1 s)))
    where Inter ss1' s' = sos_stm (Inter ss1 s)

sos_stm (Inter (Comp ss1 ss2) s)
    = Inter ss2 s', if is_Final(sos_stm (Inter ss1 s))
    where Final s' = sos_stm (Inter ss1 s)

sos_stm (Inter (If b ss1 ss2) s)
    = Inter ss1 s, if b_val b s

sos_stm (Inter (If b ss1 ss2) s)
    = Inter ss2 s, if not(b_val b s)

sos_stm (Inter (While b ss) s)
    = Inter (If b (Comp ss (While b ss)) Skip) s
```

Funktionen `sos_stm` implementerar ett steg i beräkningen. Den fullständiga härledningssekvensen beräknas av funktionen `deriv_seq` nedan (även om den sekvensen är oändlig!).

```
deriv_seq (Inter ss s)
    = (Inter ss s) : (deriv_seq (sos_stm (Inter ss s)))
deriv_seq (Final s) = [Final s]
```

Den semantiska funktionen $\mathcal{S}_{\text{SOS}}$ definieras nu med Gofer-funktionen `s_sos`:

```
s_sos ss s = s'
    where Final s' = last (deriv_seq (Inter ss s))
```

Exempel 4: Härledningssekvensen för fakultetssatsen i tillståndet `s_init` erhålles med

```
fac_seq = deriv_seq (Inter factorial s_init)
```

Det slutgiltiga tillståndet i härledningssekvensen fås fram med

```
s_fac' = s_sos factorial s_init
```

och värdet av y fås genom att evaluera `s_fac "y"`. □

Uppgift 2: Utöka definitionen av `sos_stm` till att innehålla `repeat`-konstruktionen. Redovisa som i föregående uppgift. □

Denotationssemantik i Gofer

I detta avsnitt implementerar vi den denotationssemantik som beskrivs i bokens kapitel 4. Vi behöver återigen filen `appB.gs` och skapar därför en projektfil `appD.gp` med följande innehåll:

```
appB.gs
appD.gs
```

Hämta labskelettet för `appD.gs` på kursens hemsida.

Direct style semantics

I den implementation som följer kommer vi att använda den inbyggda funktionen `”.”` som representerar funktionskomposition.

De teoretiska grunderna för Gofer bygger till stor del på det material som presenteras i kapitel 4, även om utrymme saknas i detta kompendium för att gå in djupare på detta. En konsekvens är emellertid det faktum att fixpunktsberäkningen kan implementeras på följande enkla vis:

```
fix ff = ff (fix ff)
```

Funktionen $\mathcal{S}_{ds}$ kan nu implementeras med funktionen

```
s_ds :: Stm -> State -> State
```

och de definitioner som vi tagit upp på föreläsningen (och som i något omskriven form återfinns i bokens tabell 4.1) kan översättas nästan mekaniskt till

```
s_ds (Ass x a)      = \s -> update s x (a_val a s)
s_ds Skip           = \s -> s
s_ds (Comp ss1 ss2) = (s_ds ss2) . (s_ds ss1)
s_ds (If b ss1 ss2) = \s -> if b_val b s then s_ds ss1 s else s_ds ss2 s
s_ds (While b ss)   = fix (\g s -> if b_val b s then g (s_ds ss s) else s)
```

Exempel 5: Vi kan återigen illustrera semantiken genom att ”köra” fakultetssatsen i det initiala tillståndet `s_init`:

```
s_final = s_ds factorial s_init
```

□

Uppgift 3: Utöka `s_ds` till att klara av `repeat`-konstruktionen. Redovisa som förut. □

Fortsättningssemantik

För att representera en fortsättning introducerar vi följande typsynonym:

```
type Cont = State -> State
```

En fortsättningsomgivning (boken, s. 130) är en avbildning från identifierare till en fortsättning och leder till följande typsynonym:

```
type Env_E = Var -> Cont
```

Produktionerna

$$S \rightarrow \dots \mid \text{begin } S_1 \text{ handle } e : S_2 \text{ end} \mid \text{raise } e$$

har sin motsvarighet i den abstrakta syntaxen, datatypen `Stm` i `appB.gs`

```
data Stm = Ass Var Aexp | Skip | Comp Stm Stm |  
         If Bexp Stm Stm | While Bexp Stm | Repeat Stm Bexp  
         Block Stm Var Stm | Raise Var
```

där komponenten `Var` i `Block` och `Raise` representerar identifieraren e i produktionerna.

Uppgift 4: Skriv en funktion `s_cs` för att realisera fortsättningssemantiken $\mathcal{S}_{cs}$ i boken. Testa er kod med programmet

```
begin  
  if x < 0 then skip else raise exit  
  handle exit: x := 0  
end
```

genom att köra följande:

```
test = Block (If (Le (V "x") (N 0)) Skip (Raise "exit"))  
        "exit" (Ass "x" (N 0))  
  
e_init x = id -- initial exception environment  
  
c_init = id   -- initial continuation  
  
s_final' = s_cs test e_init c_init s_init
```

Vad har `x` för värde i `s_final'`?

□

Projekt för Programspråkssemantik

Ni kan antingen välja att läsa och recensera en artikel, eller göra en implementation. Allt arbete utförs individuellt och lämnas in innan kursens slut.

A Artikel

Här följer en lista på föreslagna artiklar. Har ni en egen artikel som ni tycker verkar intressant kan ni försöka övertala mig.

1. Andrew W. Appel: "Semantics-Directed Code Generation";
2. William Clinger: "The Scheme 311 Compiler; An Exercise in Denotational Semantics";
3. Ravi Sethi: "Control Flow Aspects of Semantics-Directed Compiling";
4. Dana Scott, Christopher Strachey: "Toward a Mathematical Semantics for Computer Languages";
5. Uwe F. Pleban: "Compiler Prototyping Using Formal Semantics";
6. Jukka Mäki-Turja *et al*: "An Operational Semantics for Smalltalk";
7. Jan Gustafsson *et al*: "Automatic Derivation of Path and Loop Annotations in Object-Oriented Real-Time Programs";
8. Patrick Cousot *et al*: "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints".

Er recension ska innehålla en kortfattad beskrivning av vad som presenteras samt för- och nackdelar med systemet. Vilka begränsningar har författarna gjort? Är innehållet endast av akademiskt intresse, eller kan det användas till att lösa befintliga problem, osv, osv.

B Implementation

Implementera While-språket med procedurdeklarationer och block såsom reglerna beskriver under flik 3 eller 7 i kurspärmen. Ni får alltså själva välja om ni vill utgå från naturlig semantik eller denotationssemantik. Visa med vettiga körexempel att din prototyp fungerar.

Appendices of “Semantics with Applications” with Gofer implementations

Jan Bækgaard Pedersen
matt@daimi.aau.dk

Morten Rhiger
mrhiger@daimi.aau.dk

(Revised and condensed by Kjell Post — kjell.post@mdh.se)

Appendix B

Introduction to Gofer Implementations

In this appendix we give the basic definitions needed to implement the various semantics definitions in **Gofer**. Essentially, this amounts to an implementation of the material of Chapter 1. The material developed in the this appendix are placed in a file “appB.gs” so that we can use it in the following appendices.

B.1 Abstract syntax and parsing

For **Num** we choose the primitive type **Int** of **Gofer**. For **Var** we choose lists of characters and so define the type synonym:

```
type Var = [Char]
```

For each of the syntactic categories **Aexp**, **Bexp** and **Stm** we define an algebraic data type taking into account the various possibilities mentioned by the BNF syntax of Section 1.2:

```
data Aexp =  N Int | V Var | Add  Aexp Aexp |
             Mult Aexp Aexp | Sub  Aexp Aexp

data Bexp = TRUE | FALSE | Eq Aexp Aexp | Le Aexp Aexp |
             Neg Bexp | And Bexp Bexp

data Stm  = Ass Var Aexp | Skip | Comp Stm Stm |
             If Bexp Stm Stm | While Bexp Stm | Repeat Stm Bexp
             Block [(Var,Aexp)] [(Var,Stm)] Stm | Call Var
```

Example B.1 The factorial statement is represented by the following abstract data type (which may be entered directly in **Gofer**):

```
factorial = Comp (Ass "y" (N 1))
               (While (Neg (Eq (V "x") (N 1)))
```

```
(Comp (Ass "y" (Mult (V "y") (V "x")))
      (Ass "x" (Sub (V "x") (N 1)))))
```

Note that this is a representation of the *abstract syntax* of the statement

```
y := 1; while ~(x = 1) do (y := y * x; x := x - 1)
```

□

Exercise B.2 Specify an element of **Stm** that represents the statement constructed in Exercise 1.2 for computing n to the power of m . □

B.2 Evaluation of expressions

We shall first be concerned with the representation of values and states. The natural numbers **Z** will be represented by the type **Int** meaning that the semantic function $\mathcal{N}$ becomes trivial. The truth values **T** will be represented by the type **Bool** of booleans. So we define the type synonyms:

```
type Z = Int
type T = Bool
```

The set **State** is defined as the set of functions from variables to natural numbers so we define:

```
type State = Var -> Z
```

Example B.3 The state **s_init** that maps all variables except **x** to **0** and that maps **x** to **3** can be defined by

```
s_init "x" = 3
s_init y   = 0
```

Note that we encapsulate the specific variable name **x** in quotes whereas **y** can be any variable. □

The functions $\mathcal{A}$ and $\mathcal{B}$ will be called **a_val** and **b_val** in the implementation and they are defined by directly translating Table 1.1 and 1.2 into **Gofer**:

```
a_val :: Aexp -> State -> Z
b_val :: Bexp -> State -> T

a_val (N n) s      = n
a_val (V x) s      = s x
a_val (Add a1 a2) s = (a_val a1 s) + (a_val a2 s)
a_val (Mult a1 a2) s = (a_val a1 s) * (a_val a2 s)
a_val (Sub a1 a2) s = (a_val a1 s) - (a_val a2 s)
```

```

b_val TRUE s      = True
b_val FALSE s     = False
b_val (Eq a1 a2) s = True,  if a_val a1 s == a_val a2 s
                  = False, if a_val a1 s /= a_val a2 s
b_val (Le a1 a2) s = True,  if a_val a1 s <= a_val a2 s
                  = False, if a_val a1 s > a_val a2 s
b_val (Neg b) s    = True,  if b_val b s == False
                  = False, if b_val b s == True
b_val (And b1 b2) s = True,  if b_val b1 s == True &&
                           b_val b2 s == True
                  = False, if b_val b1 s == False ||
                           b_val b2 s == False

```

Exercise B.4 Construct an algebraic data type for the binary numerals considered in Section 1.3. Define a function `n_val` that associates a number (in the decimal system) to each numeral. □

Exercise B.5 Define functions

```

fv_aexp :: Aexp -> [Var]
fv_bexp :: Bexp -> [Var]

```

computing the set of free variables occurring in an expression. Ensure that each variable occurs at most once in the resulting list. □

Exercise B.6 Define functions

```

subst_aexp :: Aexp -> Var -> Aexp -> Aexp
subst_bexp :: Bexp -> Var -> Aexp -> Bexp

```

implementing the substitution operation, that is `subst_aexp a y a0` constructs $a[y \mapsto a_0]$ and `subst_bexp b y a0` constructs $b[y \mapsto a_0]$. □

Appendix C

Operational Semantics in Gofer

In this section we implement the natural semantics and the structural operational semantics of Chapter 2 in **Gofer** and show how similar techniques can be used to implement an interpreter for the abstract machine and the code generation of Chapter 3.

We shall need the definitions from last section. Since the syntax of **Gofer** does not contain an `include` command the file must either be included manually or a project-file must be written. We make a project file “`appC.gp`” containing

```
appB.gs  
appC.gs
```

To use the material one must enter

```
:p appC.gp
```

The material developed in this appendix must be placed in the file “`appC.gs`”.

In Chapter 2 we distinguish between two kinds of configurations, intermediate configurations and final configurations. This is captured by the algebraic data type:

```
data Config = Inter Stm State | Final State
```

In the next section we shall show how the natural semantics can be implemented and after that we shall turn to the structural operational semantics.

C.1 Natural semantics

Corresponding to the relation $\rightarrow$ in Section 2.1 we shall introduce a function `ns_stm` of type

```
ns_stm :: Config -> Config
```

The argument of this function corresponds to the left-hand side of $\rightarrow$ whereas the result produced corresponds to the right-hand side of the relation. This is possible because Theorem 2.9 shows that the relation is deterministic. The definition of `ns_stm` follows closely the definition of $\rightarrow$ in Table 2.1:

```

update s x v y = v,    if x == y
                = s y, otherwise

ns_stm (Inter (Ass x a) s)
    = Final (update s x (a_val a s))

ns_stm (Inter (Skip) s)
    = Final s

ns_stm (Inter (Comp ss1 ss2) s)
    = Final s''
    where Final s'  = ns_stm (Inter ss1 s)
          Final s'' = ns_stm (Inter ss2 s')

ns_stm (Inter (If b ss1 ss2) s)
    = Final s', if b_val b s
    where Final s' = ns_stm (Inter ss1 s)

ns_stm (Inter (If b ss1 ss2) s)
    = Final s', if not(b_val b s)
    where Final s' = ns_stm (Inter ss2 s)

ns_stm (Inter (While b ss) s)
    = Final s'', if b_val b s
    where Final s'  = ns_stm (Inter ss s)
          Final s'' = ns_stm (Inter (While b ss) s')

ns_stm (Inter (While b ss) s)
    = Final s, if not(b_val b s)

```

Note that in the axiom for assignment, `update s x v` corresponds to $s[x \mapsto v]$. The semantic function $\mathcal{S}_{\text{ns}}$ can now be defined by

```

s_ns ss s = s'
    where Final s' = ns_stm (Inter ss s)

```

Example C.1 We can execute the factorial statement (see Example B.1) from the state `s_init` mapping `x` to **3** and all other variables to **0** (see Example B.3). The final state `s_fac` is obtained as follows:

```
s_fac = s_ns factorial s_init
```

To get the value of `y` we evaluate `s_fac "y"`. □

Exercise C.2 Extend the definition of `ns_stm` to include the `repeat`-construct. □

Exercise C.3 Define an algebraic data type `Deriv_tree` representing the derivation trees of the natural semantics. Construct a variant of the function `s_ns` of type

```
s_ns :: Stm -> State -> Deriv_tree
```

that constructs the *derivation tree* for a given statement and state rather than just the final state. Apply the function to some example statements. $\square$

C.2 Structural operational semantics

When specifying the structural operational semantics we shall need to test whether $\Rightarrow$ produces an intermediate configuration or a final configuration. So we shall introduce the function `is_final` defined by:

```
is_Final (Inter ss s) = False
is_Final (Final s)    = True
```

Corresponding to the relation $\Rightarrow$ we define the function `sos_stm` of type:

```
sos_stm :: Config -> Config
```

As in the previous section the argument of this function will correspond to the configuration on the left-hand side of the relation $\Rightarrow$ and the result will correspond to the right-hand side. Again this implementation technique is only possible because the semantics is deterministic (Exercise 2.22). The definition of `sos_stm` follows Table 2.2 closely:

```
sos_stm (Inter (Ass x a) s)
    = Final (update s x (a_val a s))

sos_stm (Inter (Skip) s) = Final s

sos_stm (Inter (Comp ss1 ss2) s)
    = Inter (Comp ss1' ss2) s', if not(is_Final (sos_stm (Inter ss1 s)))
    where Inter ss1' s' = sos_stm (Inter ss1 s)

sos_stm (Inter (Comp ss1 ss2) s)
    = Inter ss2 s', if is_Final(sos_stm (Inter ss1 s))
    where Final s' = sos_stm (Inter ss1 s)

sos_stm (Inter (If b ss1 ss2) s)
    = Inter ss1 s, if b_val b s

sos_stm (Inter (If b ss1 ss2) s)
    = Inter ss2 s, if not(b_val b s)

sos_stm (Inter (While b ss) s)
    = Inter (If b (Comp ss (While b ss)) Skip) s
```

The function `sos_stm` implements one step of the computation. The function `deriv_seq` defined below will determine the complete derivation sequence (*even if it is infinite!*).

```
deriv_seq (Inter ss s)
  = (Inter ss s) : (deriv_seq (sos_stm (Inter ss s)))
deriv_seq (Final s) = [Final s]
```

The semantic function $\mathcal{S}_{\text{SOS}}$ can now be defined by the **Gofer** function `s_sos`:

```
s_sos ss s = s'
  where Final s' = last (deriv_seq (Inter ss s))
```

Example C.4 The derivation sequence obtained by executing the factorial statement on the state `s_init` can now be obtained as follows:

```
fac_seq = deriv_seq (Inter factorial s_init)
```

We may want to inspect this in more detail and in particular we may be interested in the values of the variables `x` and `y` in the various intermediate states. To facilitate this we use the function

```
show_seq fv l = layn (map show_config l)
  where show_config (Final s) =
    "final state:\n" ++ layn (map (show_val s) fv)
    show_config (Inter ss s) =
      (show ss) ++ "\n" ++ layn (map (show_val s) fv)
    show_val s x = " s(" ++ x ++ ")=" ++ show (s x)
```

The function call `show_seq ["x",y"] fac_seq` will for each configuration in the derivation sequence `fac_seq` list the statement part and the values of `x` and `y` in the state part. The final state of the derivation sequence can be obtained from

```
s_fac' = s_sos factorial s_init
```

and the value obtained for `y` is obtained by executing `s_fac' "y"`. □

Exercise C.5 Extend the definition of `sos_stm` to include the `repeat`-construct. □

C.3 Extensions of While

The implementation of the natural semantics of **While** in Section C.1 will now be extended to the procedure language **Proc** of Section 2.5. Rather than presenting a fully worked out implementation we shall give detailed instructions for how to construct it. We shall pay attention to the semantics of **Proc** with static scope rules for variables as well as procedures.

Exercise C.6 The first step will be to define the data types needed to represent the semantics of **Proc**.

- Define algebraic datatypes `Dec_V` and `Dec_P`

```
type Dec_V = [(Var, Aexp)]
type Dec_P = [(Var, Stm)]
```

- Define the algebraic type `Loc` to be `Int` such that locations will be integers. Define the function

```
new :: Loc -> Loc
```

such that `new` increments its argument by one.

- define algebraic types `Env_V` and `Env_P` corresponding to $\mathbf{Env}_v$ and $\mathbf{Env}_p$. Define the function

```
upd_P :: (Dec_P, Env_V, Env_P) -> Env_P
```

corresponding to `upd_P`.

- Finally, we need a type `Store` corresponding to $\mathbf{Store}$. There are at least three possibilities: One possibility is to define

```
data Loc' = Loc loc | Next
type Store = Loc' -> Z
```

as this will correspond closely to the definition of $\mathbf{Store}$. Alternatively, one may identify the special token 'next' with location `0` and then simply define

```
type Store = Loc -> Z
```

The third possibility is to define

```
type Store = (Loc -> Z, Loc)
```

where the second component corresponds to the value of 'next'. Choose a method that seems appropriate to you. $\square$

Exercise C.7 Finally we turn towards the transition system. We begin by implementing the transition system for variable declarations:

- Define an abstract data type `Config_D` for the configurations of the transition system for variable declarations.
- Then define a function

```
ns_dec_V :: Config_D -> Config_D
```

corresponding to the relation $\rightarrow_D$.

Now we turn to the transition relation for statements:

- Define an abstract data-type `Config_P` corresponding to the configurations $\langle S, sto \rangle$ and sto of the transition system.
- Next define a function

```
ns_stm :: (Env_V, Env_P) -> Config_P -> Config_P
```

Corresponding to the relation $\rightarrow$.

Finally define a function

```
s_ns :: Stm -> Store -> Store
```

that calls `ns_stm` with appropriately initialized environments. Use the function on various example statements in order to ensure that the implementation works as intended. $\square$

Exercise C.8 Modify the implementation above to use dynamic scope rules for variable declarations as well as procedure declarations. $\square$

It is more problematic to extend the implementation to handle the constructs of Section 2.4:

Exercise C.9 Discuss how to extend the implementation of the natural semantics in Section C.1 to incorporate the construct considered in Section 2.4. $\square$

Exercise C.10 Discuss how to extend the implementation of the structural operational semantics of Section C.2 to incorporate the constructs considered in Section 2.4. $\square$

C.4 Provably correct implementation

Rather than presenting a fully worked out **Gofer** script we shall provide exercises showing how to develop an implementation corresponding to Chapter 3.

Exercise C.11 We need some data types to represent the configurations of the abstract machine:

- Define an algebraic data type `Am_ins` for representing instructions and define the type synonym

```
type Am_code = [Am_ins]
```

for representing code.

- Define an algebraic data type **Stack_values** representing the elements that may be on the evaluation stack and define the type synonym

```
type Stack = [Stack_values]
```

- Define a type **Storage** representing the storage.

Finally define

```
type Am_config = (Am_code, Stack, Storage)
```

for the configurations of **AM**. □

Exercise C.12 We can then turn to the semantics of the abstract machine instructions. For this we proceed in three stages:

- First define a function **am_step** of type

```
am_step :: Am_config -> Am_config
```

implementing Table 3.1.

- We shall also be interested in the computation sequence of **AM** so define a function

```
am_comp_seq :: Am_code -> Storage -> [Am_config]
```

that given a sequence of instructions and an initial storage will construct the corresponding computation sequence.

- Finally define a function **run** corresponding to the function $\mathcal{M}$ of Chapter 3.

This provides us with an interpreter for **AM**. What happens if we enter a stuck configuration? □

Exercise C.13 Finally, we implement the code generation functions:

- Define functions corresponding to $\mathcal{CA}$, $\mathcal{CB}$ and $\mathcal{CS}$.
- Define a function **am_stm** corresponding to the function $\mathcal{S}_{\text{am}}$.

Apply the construction to a couple of examples to verify that everything works as expected. □

Exercise C.14 Modify the implementation to use the abstract machine **AM₂** of Exercise 3.8 and 3.17 rather than **AM**. □

Appendix D

Denotational semantics in Gofer

In this section we implement the denotational semantics of Chapter 4 in **Gofer** and show how similar techniques can be used to implement the static program analysis of Chapter 5. We shall need the definitions from Appendix B so we begin by writing a project file “appD.gp” containing

```
appB.gs
appD.gs
```

The material developed in this appendix must then be placed in the file “appD.gs”.

D.1 Direct style semantics

In the implementation we shall rely on the built-in function ‘.’ which denotes function composition.

The theoretical foundation of **Gofer** is closely related to the theory developed in Chapter 4 (although it is outside the scope of this book to go further into this). One of the consequences of this is that the fixed point operation can be implemented in a very simple way:

```
fix ff = ff (fix ff)
```

The function $\mathcal{S}_{ds}$ can now be implemented by the function

```
s_ds :: Stm -> State -> State
```

A straightforward rewriting of Table 4.1 gives:

```
s_ds (Ass x a)      = \s -> update s x (a_val a s)
                      where update s x v y = v,   if x == y
                      = s y, otherwise

s_ds Skip           = \s -> s

s_ds (Comp ss1 ss2) = (s_ds ss2) . (s_ds ss1)
```

```

s_ds (If b ss1 ss2) = \s -> if b_val b s then s_ds ss1 s else s_ds ss2 s

s_ds (While b ss)    = fix (\g s -> if b_val b s then g (s_ds ss s) else s)

```

Example D.1 Returning to the factorial statement we can apply its denotation to the initial state `s_init` as follows:

```

s_final = s_ds factorial s_init

```

□

Exercise D.2 We may be interested in the various iterands of the fixed point. Rewrite the semantics equations above so that each fixed point is unfolded at most n times where n is an additional parameter to the functions. Give examples showing that if the value of n is sufficiently large then we get the same result as above. □

Exercise D.3 Extend the definition above to handle the `repeat`-construct. □

D.2 Extensions of While

It is fairly straightforward to extend the implementation to handle the procedure language and the exception language of Section 4.5.

Exercise D.4 Modify the above implementation to use environments and stores and extend it to implement the semantics of the language **Proc** of Section 4.5. □

Exercise D.5 Modify the above implementation to use continuations and extend it to handle the language **Exc** of Section 4.5. □

D.3 Static program analysis

Rather than presenting a fully worked out **Gofer** script performing the dependency analysis we shall provide a rather detailed list of instructions for how to develop such an implementation.

Exercise D.6 The first step will be to implement the complete lattices **P** and **PState** and the operations on them:

- Define an algebraic data type **Property** representing the set **P** of properties and define a function `p_lub` corresponding to $\sqcup_P$.
- Define a type synonym **Pstate** representing the property states. Define the special property states `INIT` and `LOST`. Define a function `pstate_lub` corresponding to $\sqcup_{Ps}$. □

Exercise D.7 We can turn to the semantic equations defining the analysis:

- Define the functions

`p_aexp :: Aexp -> Pstate -> Property`

corresponding to $\mathcal{PA}$ and

`p_bexp :: Bexp -> Pstate -> Property`

corresponding to $\mathcal{PB}$.

- Define the auxiliary function `cond_P` corresponding to cond_P .
- Define the function

`p_stm :: Stm -> Pstate -> Pstate`

corresponding to $\mathcal{PS}$ of Table 5.3. (You may use the result of Section 5.4 for this).

□

Exercise D.8 Implement the algorithm of Section 5.2 and apply the implementation to a couple of examples to verify that everything works as expected. □

Appendix E

From Miranda to Gofer by hand

In this appendix we give an informal description of the translation from **Miranda**-programs to **Gofer**-programs. In particular we describe what was necessary to change the appendices. We do not catch all differences but mainly those that are relevant to the previous appendices. For deeper explanations see [2] (Appendix D), [1] and [3].

E.1 Scripts and projects

In **Gofer** it is not possible to include a file from another file. If two or more files are mutually dependent a *project*-file must be made to be able to access them. Loading a **Miranda**-script may cause a number of files “`file1.m`”, “`file2.m`”, ..., “`filen.m`” to be read. These files are those found in the dependency graph. When translating the files to **Gofer** one must make a project file “`file.gp`” containing

```
file1.gs  
file2.gs  
...  
filen.gs
```

where each “`filei.gs`” is a **Gofer**-script corresponding to the **Miranda**-script “`filei.m`”.

E.2 Types and data types

In **Miranda** type synonym names as well as data type names must begin with a lower-case letter. Declarations of new type synonyms starts with the name of the type synonym, followed by the formal arguments (if any), a double-equal sign “`==`” and the type expression. Declaration of data types starts with the name of the data type, followed by the formal arguments (if any), followed by “`::=`” and the constructors which are separated by “`|`”. Constructor names must be capitalized.

In **Gofer** all type synonym names and data type names must start with a capital letter. This includes the basic types. Declaration of new type synonyms must start with the keyword **type**, the name of the new synonym, the possible formal arguments, an equal sign “`=`” and the type expression. Declaration of new abstract data types starts with

the keyword **data** followed by the name of the data type, the possible formal arguments, an equal sign “=” and the constructors separated by “|”. Constructor names must be capitalized too.

The syntax for type expressions are almost the same in the two languages. The differences has not influenced the translation of the appendices. We have therefore the correspondence

Miranda	Gofer
num	Int
bool	Bool
char	Char
string == [char]	type String = [Char]
sex ::= Male Female	data Sex = Male Female
set a == [a]	type Set a = [a]

E.3 Functions and expressions

Definition of functions have almost the same syntax in the two languages. They both allow mutually recursive functions in a syntax close to “mathematical notation” with pattern matching of actual function arguments. One difference is that **Gofer** requires the number of patterns (on the left-hand side) to be the same for all defining clauses for each particular function. Thus

```
binary      :: Int -> Int -> Int
binary 0    = id
binary n x  = n * x
```

is not a legal function declaration. There are however no problems if one rewrites the above to the equivalent

```
binary      :: Int -> Int -> Int
binary 0    = id
binary n    = \x -> n * x
```

This restriction has only influenced the development of the function **s_ds** in Appendix D. Both languages allow the **where** clause as a **let**-construct and they both allow guarded **if**-expression. **Gofer** accepts two kinds of syntax for the guarded **if** expression. In this paper we have chosen the one similar to **Miranda**.

Expressions have the same syntax in the two languages. They only differ on the set of predefined functions. This has only influenced the development of the function **show_seq** in Section C.2 where we use the **Gofer**-function **layn** as a substitute for the **Miranda**-function **lay**.

Bibliography

- [1] Ian Holyer. *Functional programming with Miranda*. UCL Press, 1993.
- [2] Mark Jones. *An Introduction to Gofer*, February 1995.
- [3] Jan Bækgaard Pedersen and Morten Rhiger. An introduction to the gofer functional programming environment. Report, DAIMI, Computer Science Department, Aarhus University, Ny Munkegade, 8000 Aarhus C, Denmark, 1995.
- [4] Torben Mogensen. Ratatosk. a parser generator and scanner generator for gofer. Anonymos ftp: `ftp.diku.dk` file `pub/diku/dists/Ratatosk.tar.Z`, 1993.
- [5] Hanne Riis Nielson and Flemming Nielson. *Semantics with Applications, a formal introduction*. Wiley Professional Computing. John Wiley and Sons, 1992.